

NORMATIVE / RELEASE-BLOCKING TARGET

ARCHIE SACROSANCT

Normative architecture and execution constitution for Archie + Forge

DOCUMENT	Archie Sacrosanct
VERSION	3.1-draft
DATE	2026-08-28
REPOSITORY	satvikOS/Forge
PRIMARY TARGET	Mac Studio / Apple M4 Max / 36 GB unified memory
STATUS	Target specification - implementation claims require archived proof

Probabilistic in exploration. Exact in specification. Deterministic in commitment. Verified in execution. Explicit in failure. Unlimited in logical feature-tree length.

Contents

Document control and reading map	8
0. The Elastic Exact Feature DAG Constitution	8
0.1 Definition	8
0.2 Exact length law	9
0.3 Completeness contract	10
0.4 Cardinality reconciliation	11
0.5 No opaque loops or macros	12
0.6 Patterns	12
0.7 Sketch expansion	13
0.8 Feature expansion	13
0.9 Assemblies and hierarchy	14
0.10 Stable topological naming	14
0.11 Chunked generation and unlimited logical length	15
0.12 Graph-size forecasting	16
0.13 Edits	16
0.14 Exactness metrics	16
0.15 Audit finding from supplied examples	17
0.16 Feature-DAG compilation is an incremental domain compile	17
1. The governing product axiom	18
2. Non-negotiable laws	18
3. What "pure C++" means	19
3.1 Native LangGraph-style behavior	20
3.2 Zero-JavaScript repository and non-C++ allowlist	20
4. Hardware truth and resource envelope	22
4.1 Consequences of the 36 GB envelope	22
4.2 Memory budget gate	22
4.3 Throughput and latency	23
5. Model architecture	23
5.1 Recommended foundation	23

5.2 Production inference	23
5.3 Dual-mode behavior	23
5.4 Specialist system, not one undifferentiated prompt	24
6. End-to-end execution pipeline	24
7. Multimodal intake and ambiguity handling	25
7.1 Accepted evidence	25
7.2 Normalization	25
7.3 Ambiguity ledger	25
7.4 No hallucinated precision	26
8. Engineering Specification Graph	26
8.1 Required entity families	26
8.2 Traceability	26
8.3 Requirement conflict policy	26
9. Native C++ feature IR and compiler	27
9.1 IR design principles	27
9.2 Logical C++ type model	27
9.3 Compiler stages	28
9.4 Determinism envelope	29
9.5 Sandboxing and resource guards	29
9.6 Prebuilt operator ABI and incremental execution cache	29
10. Forge geometry kernel architecture	30
10.1 Kernel roles	30
10.2 Capability matrix, not fictional parity	30
10.3 Geometric robustness doctrine	31
10.4 Sketch constraint system	31
10.5 Import and export	31
10.6 Fully local kernel and solver dependency plane	32
11. Durable native agent architecture	33
11.1 Typed workflow state	34
11.2 Node contract	34

11.3 Candidate reasoning	34
11.4 Error taxonomy and repair	35
11.5 Memory architecture	35
12. SearXNG retrieval and evidence architecture	35
12.1 Search request contract	36
12.2 Evidence record	36
12.3 Retrieval is not execution	37
12.4 Offline behavior	37
13. Verification, evidence, and bounded repair	37
13.1 Verification ladder	37
13.2 Visual verification set	38
13.3 Repair budget and convergence	39
13.4 Evidence bundle	39
14. Full engineering scope and closure	39
14.1 Domain map	39
14.2 Manufacturing intelligence	40
14.3 GD&T;, tolerance, and inspection	40
14.4 Assembly intelligence	41
14.5 Physics in the loop	41
14.6 Optimization	44
14.7 PLM/PDM and engineering work products	44
14.8 Physical feedback loop	44
14.9 High-end visualization, DCC, and digital-scene scope	44
15. Training architecture and curriculum	45
15.1 Competence decomposition	45
15.2 Training stages	46
15.3 Dataset record and sampling	47
15.4 Synthetic data factory	47
15.5 Long feature-DAG curriculum	48
15.6 Reasoning supervision policy	48
15.7 Adapter strategy	48

16. Data and source registry	48
16.1 Admission states	48
16.2 Parametric CAD, sketches, parts, and assemblies	49
16.3 Text/drawing-to-CAD and CAD editing sources	49
16.4 Supplemental 3D vision, articulation, and material-learning candidates	50
16.5 Engineering physics and surrogate-learning candidates	50
16.6 Materials, manufacturing, standards, and public technical knowledge	51
16.7 Mathematics, code, and technical language	51
16.8 Data quality gates	51
16.9 High-yield source program for industrial geometry and physics	51
17. Benchmark and evaluation constitution	57
17.1 Public benchmark matrix	57
17.2 System-level evaluation	57
17.3 Contamination firewall	58
17.4 Hidden Forge evaluation suite	58
17.5 Engineering quality scorecard	59
18. Local inference, model storage, and weight delivery	59
18.1 Preferred package	59
18.2 Local mapped weights; network inference and weight streaming are forbidden	59
18.3 Context management	60
18.4 Performance harness	60
19. Product, workflow, and user-interface architecture	60
19.1 Product lessons from the market	60
19.2 Desktop technology	61
19.3 Workspace anatomy	63
19.4 Feature-tree UI for enormous graphs	63
19.5 Archie workstream UI	64
19.6 Visualization rules	64
20. Security, privacy, safety, and governance	64
20.1 Threat model	64

20.2 Local-only execution and single-tool network policy	65
20.3 Data governance	65
20.4 Engineering safety classes	65
20.5 Supply-chain governance	65
21. Build, test, observability, and release engineering	66
21.1 Repository architecture target	66
21.2 Local build, activation, and fast execution contract	66
21.3 Standing storage-health and proof-based garbage collection	68
21.4 Test pyramid	70
21.5 Observability	70
21.6 Release gates	71
22. Implementation sequence	71
Phase 0 - truth, governance, and reproducibility	71
Phase 1 - native C++ spine	72
Phase 2 - manufacturing-grade parts	72
Phase 3 - long-DAG and benchmark leadership	72
Phase 4 - assemblies, drawings, and lifecycle	73
Phase 5 - physics and multidisciplinary engineering	73
Phase 6 - enterprise and broad physical-product platform	73
23. Definition of done for Archie	73
Appendix A - Canonical long-graph example	74
A.1 Human plan	74
A.2 Graph hierarchy	74
A.3 Illustrative native C++ builder calls	75
Appendix B - Mandatory exact-length acceptance tests	76
Appendix C - Source register	77
Models and runtimes	77
Agent and retrieval architecture	77
Geometry, numerics, and simulation	77
CAD and engineering benchmarks	78

Data and public engineering references	78
High-yield capability catalogs and product references	79
Public product references	79
Appendix D - Open decisions that require measured evidence	79

Document control and reading map

Execution boundary: Archie is a single-workstation system. Inference, orchestration, retrieval indexing, CAD compilation, rendering, simulation, validation, persistence, and storage management execute on that Mac Studio. No hosted model, remote solver, streamed weight, remote vector database, or network compute fallback is permitted. The sole production network capability is an explicitly invoked, query-redacting SearXNG browser path; every engineering result remains locally computed and locally stored.

This document replaces the earlier mixture of product vision, benchmark notes, implementation claims, user-interface ideas, and experimental tactics. It is the authoritative definition of what Archie is intended to become, how the system must behave, how claims are proved, and which shortcuts are forbidden.

The word **Sacrosanct** means that an implementation may improve these requirements but may not silently weaken them. A change to a **MUST** requires an explicit design decision, evidence, and a versioned update to this document.

Normative language

Term	Meaning
MUST / MUST NOT	Release-blocking requirement or prohibition.
SHOULD / SHOULD NOT	Strong default; deviation requires a recorded reason and evidence.
MAY	Permitted option, never an implicit requirement.
TARGET	Measurable objective that has not yet been proved.
PROVED	Reproduced by the current acceptance harness with archived artifacts.

Checkmarks, leaderboard positions, throughput claims, and parity claims are prohibited in this document unless a reproducible artifact proves them against a pinned build. Aspirations are labeled as targets. A unit test is not evidence of system completion; a screenshot is not evidence of geometric correctness; and a benchmark score is not evidence of industrial readiness.

Reading map

Concern	Sections
Exact, elastic, 10x/100x feature-tree contract	0 and Appendices A-B
Mission, laws, pure-C++ boundary, hardware, and model	1-5
Intake, specification, IR, compiler, and kernel	6-10
Native agent workflow, SearXNG, and verification	11-13
Engineering scope, training, data, and benchmarks	14-17
Local deployment, product/UI, security, and release	18-21
Implementation sequence and definition of done	22-23
Sources and unresolved measured decisions	Appendices C-D

0. The Elastic Exact Feature DAG Constitution

This section is release-blocking. It governs the requirement that the feature tree be exact and as lengthy as the model request demands-potentially 10x, 100x, or orders of magnitude longer than the short examples in the supplied logs.

0.1 Definition

The **authoritative feature DAG** is the complete, typed, dependency-resolved, replayable construction and engineering history of a Forge project. The UI may present a chronological tree, hierarchical assembly tree, filtered dependency graph, or condensed overview, but those are views over the same authoritative graph.

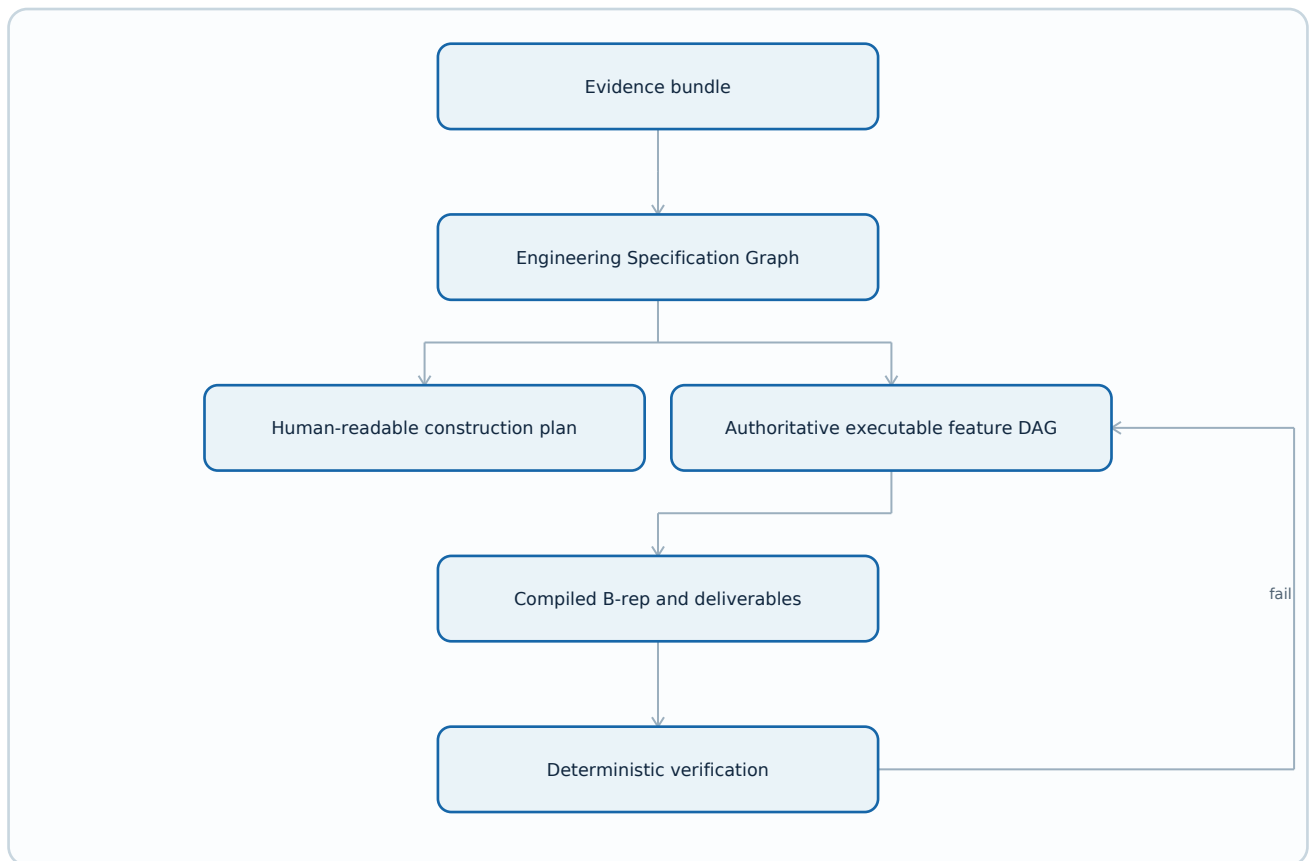


Figure 1. 0.1 Definition

Artifact	Purpose	May it be abbreviated?
Human plan	Explain intended construction	Yes, if labeled non-executable
Authoritative feature DAG	Execute, edit, replay, verify, and diff	No
Compiled B-rep/topology	Represent geometric result	Not a feature history
Render/screenshot	Provide a visual observation	Never proof by itself
Audit log	Record events and outcomes	May reference graph nodes; never replace them

A four-line summary such as "make floor; fold walls; cut six cells; add six tabs" may be a useful plan. It is not the executable feature DAG unless each operation expands into fully typed, versioned graph content and every instance is identifiable.

0.2 Exact length law

There is no global `max_features`, `max_tree_lines`, or generation-token cutoff that may redefine a complete job as finished. Let:

- **R** be the normalized requirement set;
- **D** be required design-intent and dependency nodes;
- **S** be sketch entities and constraints;
- **F** be semantic CAD features;
- **I** be pattern and assembly occurrences;
- **A** be analysis, drawing, manufacturing, and verification nodes.

The required graph is the smallest graph $G = (V, E)$ that satisfies all accepted requirements while preserving explicit intent, stable editability, deterministic replay, and applicable verification. Its size is:

$$|V| = |D| + |S| + |F| + |I| + |A| + \text{required support nodes}$$

The number is discovered from the job. It is not chosen to look impressive and must not be padded with no-op features. A simple washer should remain simple. A vehicle, machine, building system, or detailed assembly may require tens of thousands, millions, or more nodes arranged hierarchically. Operational quotas may pause and resume work; they may never silently reduce [|V|](#).

"Exact and lengthy" therefore means **semantically complete, cardinality-reconciled, and uncompressed where compression would hide editable intent**. It does not mean one feature per B-rep face, repeated no-ops, or verbose naming.

0.3 Completeness contract

The graph may enter `COMPLETE_CANDIDATE` only when all of these are true:

1. every accepted ESG requirement has at least one implementation mapping and verification mapping;
2. every node has a stable ID, type, deterministic order key, and reason for existence;
3. every dependency reference resolves to an existing or explicitly external versioned object;
4. no opaque prose placeholder remains in executable fields;
5. every pattern has an exact occurrence count and explicit occurrence identity;
6. every assembly instance has an exact occurrence identity, transform, configuration, and part-version reference;
7. all sketch entities, driving dimensions, and constraints required for regeneration are explicit;
8. every unit-bearing scalar has canonical and display units;
9. every external library feature or standard part is pinned by content hash/version and exposes its interface graph;
10. every planned body and deliverable exists or is explicitly deferred/rejected;
11. the compiler parses and type-checks every chunk;
12. a clean replay produces the accepted semantic and geometry fingerprints;
13. all applicable verification gates pass;
14. declared, parsed, compiled, replayed, and logged counts reconcile exactly.

Failure of any item keeps the status `INCOMPLETE`, `BLOCKED`, or `REJECTED`.

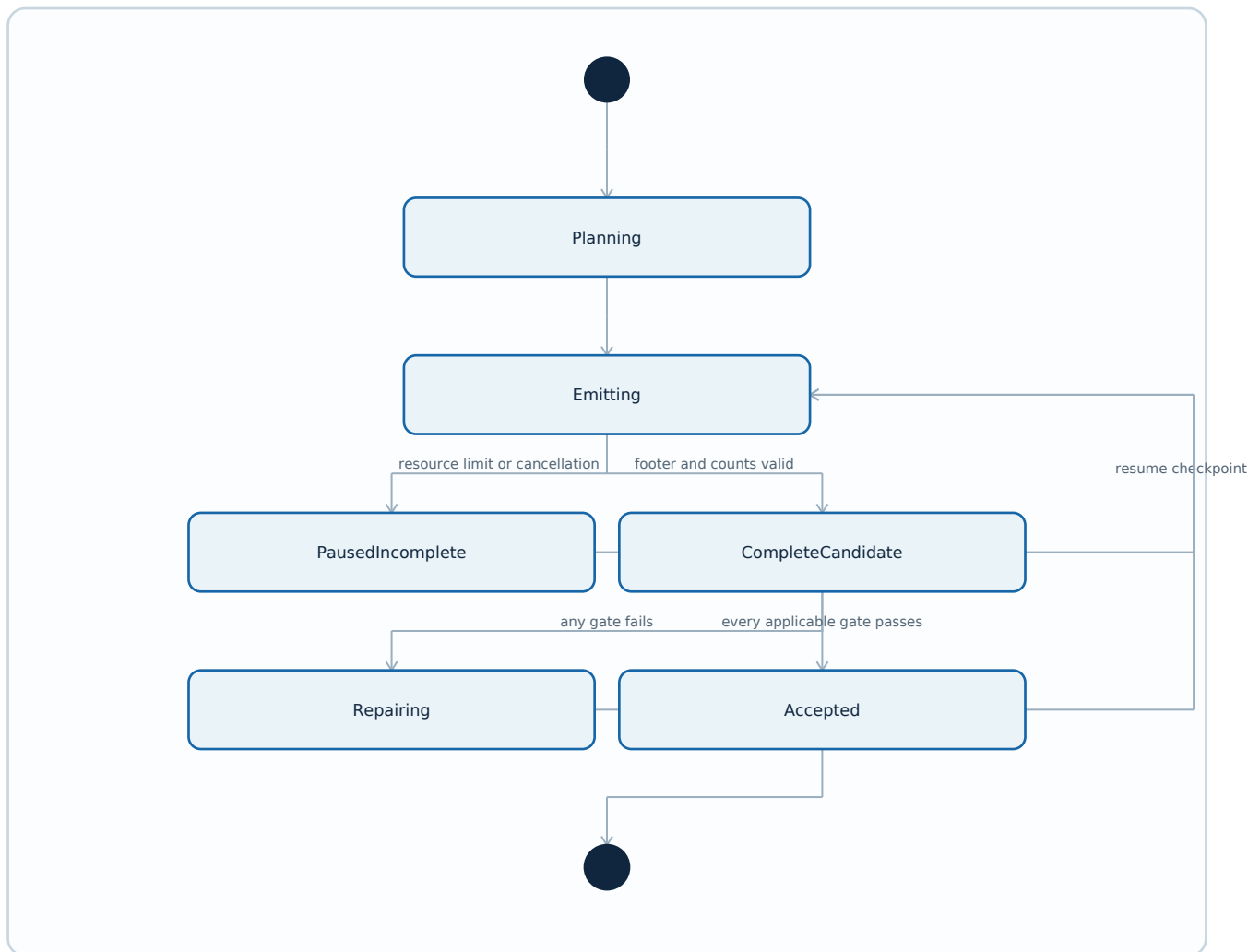


Figure 2. 0.3 Completeness contract

0.4 Cardinality reconciliation

Archie must publish count tables in the graph header/footer and verification report. At minimum:

```

N_declared_semantic_features == N_parsed_semantic_features
N_parsed_semantic_features   == N_compiled_semantic_features
N_compiled_semantic_features == N_replayed_semantic_features
N_replayed_semantic_features == N_logged_semantic_features

N_declared_pattern_instances == N_materialized_pattern_instances
N_declared_occurrences       == N_loaded_occurrences
N_declared_mates              == N_solved_or_explicitly_suppressed_mates
N_required_deliverables      == N_emitted_or_explicitly_deferred_deliverables
  
```

Separate rows are required for:

- datums and coordinate systems;
- sketches;
- sketch entities;
- geometric constraints;
- driving and reference dimensions;
- semantic solid, surface, sheet-metal, freeform, and direct-modeling features;
- pattern definitions and pattern occurrences;
- bodies and construction bodies;

- part and subassembly occurrences;
- mates, joints, contacts, and motion limits;
- materials and appearance assignments;
- manufacturing and process features;
- GD&T annotations and drawing entities;
- load cases, mesh controls, simulation runs, and result fields;
- verification predicates and results;
- unresolved, orphaned, suppressed, failed, and quarantined nodes.

The required values for unresolved references, unexplained orphans, opaque placeholders, and unapproved failed nodes are zero.

0.5 No opaque loops or macros

Executable statements such as these are forbidden:

```
for each corner: add feet
make curved web and arms
place six mounting tabs
finish the remaining holes
apply appropriate fillets
use standard fasteners
```

They omit identity, parameters, reference selection, failure behavior, or exact count. Permitted compression has only two forms:

1. **A typed pattern feature** with explicit seed, transform rule, count, suppression map, and an occurrence table whose stable child IDs can be individually referenced and edited.
2. **A versioned library feature or component** pinned by content hash, whose internal graph is retrievable and whose public parameters and interfaces are explicit.

The UI may collapse either form visually. The graph cannot erase the internal definition or child lineage.

0.6 Patterns

A pattern is one semantic feature plus exact, addressable instances. It is not duplicated model chatter and not an unexpanded sentence.

Required pattern fields include:

- `PatternId` and `SeedFeatureIds`;
- linear, circular, table, curve-driven, sketch-driven, or algorithmic pattern kind;
- axis/path/frame references using stable semantic queries;
- count, spacing/angle/table values, and canonical units;
- symmetry and direction policy;
- exact occurrence list with `OccurrenceId`, transform, suppression state, and expected result body;
- collision/merge policy;
- per-instance overrides with provenance;
- deterministic ordering and rebuild policy.

For a six-hole circular pattern, the graph must prove six intended instances, six materialized results unless suppression is explicit, and the expected angular positions. A later edit may address `hole_pattern.instance[4]` without relying on a volatile face index.

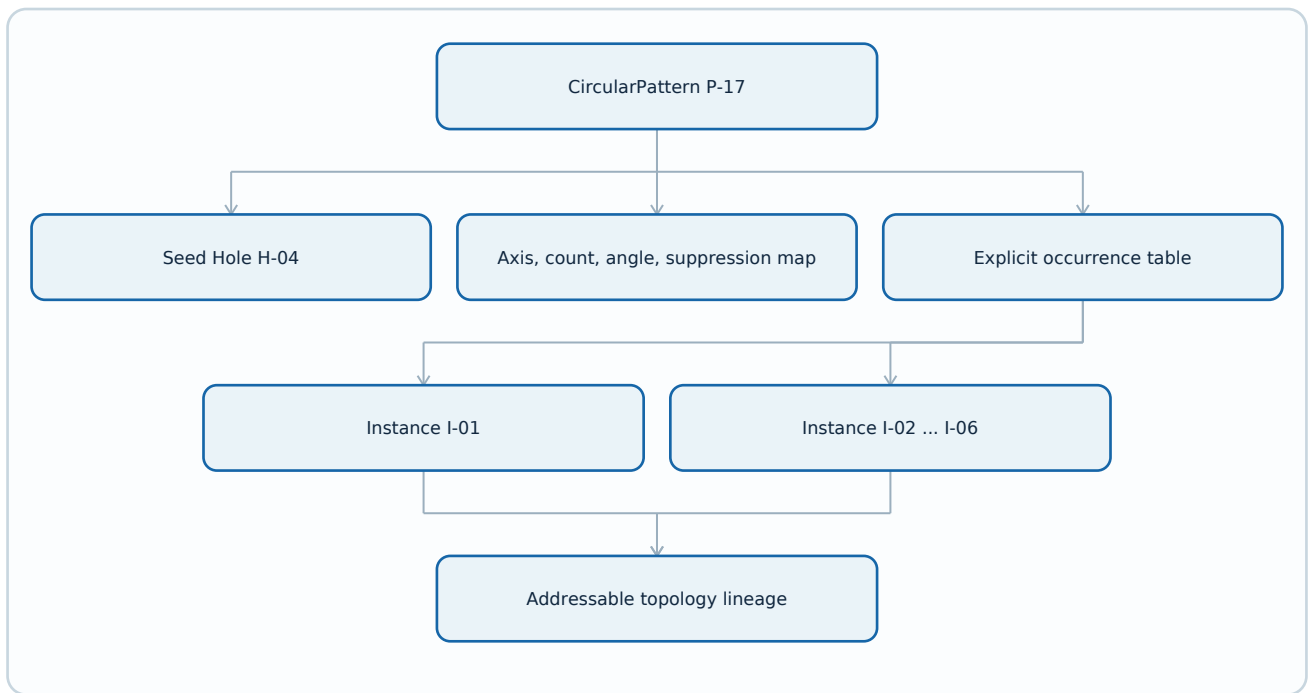


Figure 3. 0.6 Patterns

0.7 Sketch expansion

A `SketchFeature` is a container, not a hidden drawing. Its subordinate graph includes:

- every line, arc, circle, spline, point, conic, projection, and construction entity;
- every endpoint and parameterization needed for deterministic regeneration;
- coincidence, horizontal, vertical, parallel, perpendicular, tangent, concentric, equal, symmetry, midpoint, fix, curvature, and other constraints;
- dimensional constraints with driving/reference role and tolerance where applicable;
- external geometry projections using stable semantic references;
- degrees-of-freedom count before and after solve;
- solver result, redundancy/conflict diagnosis, and intended fully/under-constrained policy;
- profile and region identity for downstream features.

The default target for manufacturing geometry is a fully constrained sketch. Deliberately under-constrained conceptual geometry must be labeled and cannot silently reach a production release.

0.8 Feature expansion

Every solid, surface, sheet-metal, freeform, lattice, or direct-modeling feature carries enough data to regenerate and validate it. At minimum:

```

FeatureId
kind and schema version
semantic name
RequirementIds and rationale code
parent body/part/assembly
ordered inputs and stable reference queries
typed parameters and units
preconditions and postconditions
expected topology/geometric invariants
manufacturing intent where relevant
failure class and permitted recovery policy
suppression/configuration expression
source evidence and assumption links
deterministic ordering key
  
```

content hash

Feature families include, without treating the list as a completeness claim:

- primitives, sketches, pads/extrudes, revolves, sweeps, lofts, pipes, ribs, webs, slots, holes, pockets, shells, drafts, chamfers, fillets, offsets, thicken, split, trim, extend, sew, heal, replace-face, boolean operations, patterns, mirrors, wraps, embosses, projected curves, datum/reference construction, transformations, and direct edits;
- ruled, revolved, swept, lofted, Coons, Gordon, subdivision/freeform, NURBS, implicit, and mesh-derived surface/solid operations;
- sheet-metal walls, bends, hems, jogs, beads, louvers, corner reliefs, rip, unfold/refold, forming tools, bend allowances/deductions, and flat-pattern state;
- weldments, frames, routed systems, fasteners, holes/threads, gears/springs/cams, lattice/TPMS structures, composite layups, additive supports, mold/tooling features, and machining features;
- drawing views, dimensions, annotations, GD&T, tables, sections, details, and revision objects;
- assembly occurrences, mates, joints, contacts, envelopes, interference checks, and motion studies.

A named feature may internally use many kernel calls. Those calls are recorded in a compiler trace; they need not masquerade as user-visible design features unless they carry edit intent.

0.9 Assemblies and hierarchy

Large assemblies remain hierarchical. The system must not flatten a million fasteners into one unmanageable linear list, nor hide them behind a count with no identities.

- A part definition has one versioned feature DAG.
- An occurrence references that definition by immutable revision/hash and supplies configuration, transform, quantity role, and instance metadata.
- A subassembly owns its occurrence and mate graph.
- Repeated components use typed occurrence patterns with explicit instance IDs.
- Standard purchased components may remain referenced library parts; their internal manufacturing tree need not be copied unless the user intends to manufacture or modify them.
- Flexible components and alternate-position representations have explicit state models.
- Suppression, configuration, and effectivity expressions are deterministic and evaluable.

The count report distinguishes unique definitions from occurrences. A BOM quantity must reconcile with occurrence state and the product structure.

0.10 Stable topological naming

Downstream features must not rely only on kernel-generated indices such as `Face17`. References use a combination of:

- feature ancestry and generating operation;
- semantic role such as `mounting_face`, `bore_wall`, or `datum_A`;
- geometric class and invariant signature;
- adjacency and orientation relationships;
- expected area, length, radius, and curvature ranges;
- location in a named coordinate frame;
- persistent lineage through split and merge events;
- fallback query with uniqueness and ambiguity checks.

If a query resolves to zero or multiple candidates outside its policy, regeneration stops at that feature. Archie may propose a repair, but it must not silently bind to a convenient face.

0.11 Chunked generation and unlimited logical length

Large graphs are emitted and compiled as a cryptographically chained stream:

```

GraphHeader {
  schema_version, project_id, normalized_spec_hash,
  model_package_hash, compiler_version,
  planned_chunk_count_or_unknown, declared_count_ranges
}

FeatureChunk {
  sequence_number, previous_chunk_hash,
  nodes[], edges[], running_counts,
  unresolved_forward_declarations[], chunk_hash
}

GraphFooter {
  final_counts, unresolved_references = 0,
  requirement_coverage = 1.0,
  graph_root_hash, replay_fingerprint, completion_status
}

```

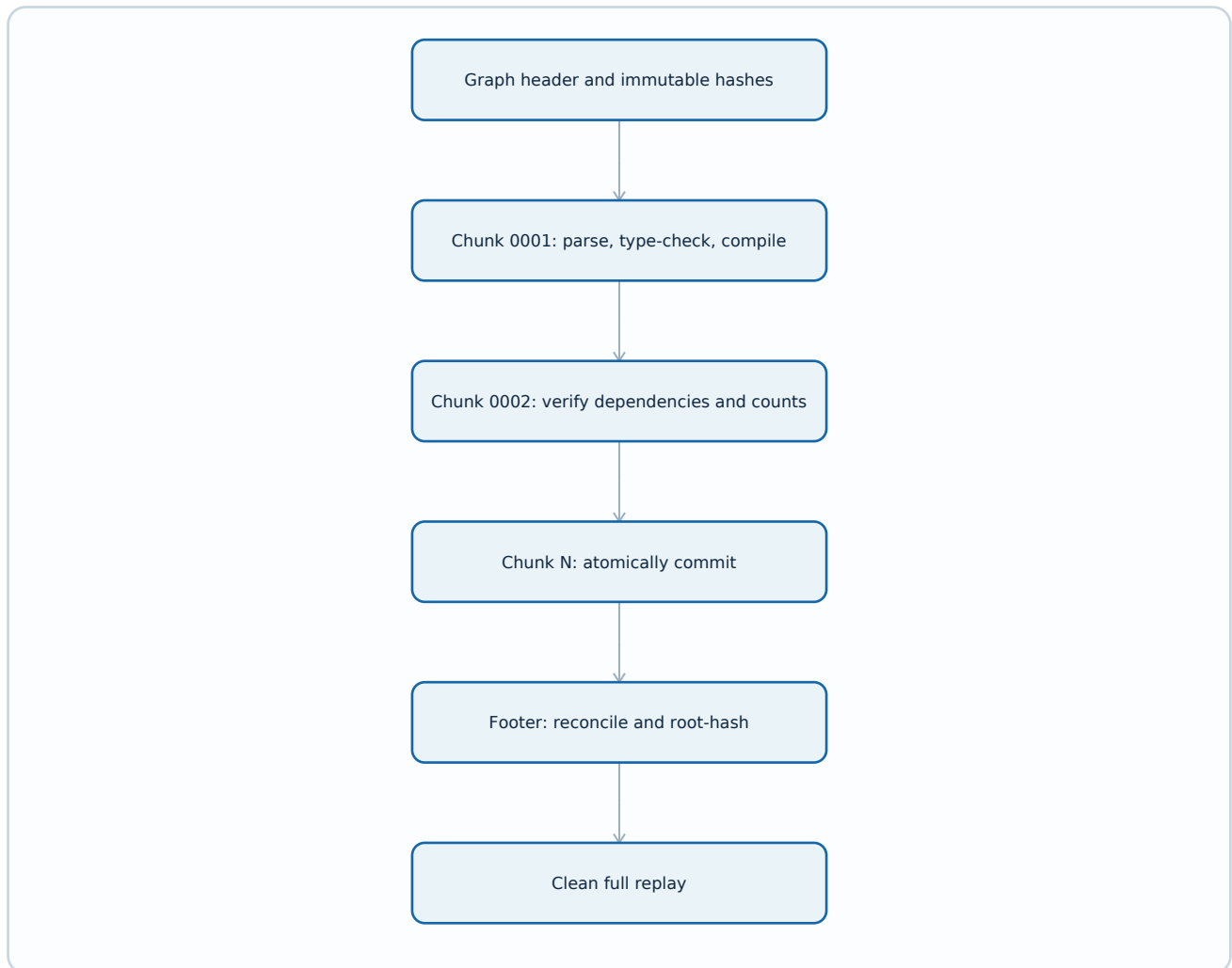


Figure 4. 0.11 Chunked generation and unlimited logical length

Rules:

1. chunks are canonical and atomically committed;
2. a chunk is parsed, type-checked, dependency-checked, and locally compiled before the next dependent chunk commits;
3. forward declarations are bounded and must resolve by the footer;

4. resumability starts at the last committed chunk, not a reconstructed conversation summary;
5. the language model receives the active dependency slice, ESG slice, symbol table, and relevant summaries-not the entire project every time;
6. graph state lives in the project store, outside the context window;
7. context compaction may summarize rationale but cannot mutate executable nodes;
8. chunk hashes and running counts detect loss, duplication, reordering, or partial writes;
9. cancellation produces a valid checkpoint with `PAUSED_INCOMPLETE`;
10. successful finalization requires a clean full replay from the immutable input bundle.

This mechanism is what permits a feature DAG to be 10×, 100×, or far longer than one model response while remaining exact.

0.12 Graph-size forecasting

Before emission, Archie provides a forecast by node family and confidence range. The forecast is for scheduling and memory planning, never a cap. After each chunk it updates remaining work from actual decomposition. A large forecast may trigger staged delivery or ask the user to select product scope, but accepted scope cannot be silently discarded.

Forecast errors are tracked. Systematic underestimation is a planning defect because it causes context and latency failures; systematic overestimation may signal needless feature inflation.

0.13 Edits

An edit begins with a graph diff budget:

- target requirements and target nodes;
- permitted dependent rebuild region;
- invariants that must not change;
- expected geometry, BOM, and drawing effects;
- forbidden changes;
- applicable revalidation gates.

Archie patches the smallest semantically correct subgraph. After replay it publishes added, removed, modified, suppressed, rebound, and regenerated nodes; changed topology; changed mass properties; changed BOM/drawings; and invariant results. An edit that produces the right screenshot while damaging unrelated graph intent is a failure.

0.14 Exactness metrics

Metric	Release requirement
Requirement implementation coverage	1.000 for accepted in-scope requirements
Requirement verification coverage	1.000 or explicit approved waiver
Unresolved references	0
Unexplained orphan nodes	0
Opaque executable placeholders	0
Declared/parsed/compiled/replayed/logged count mismatch	0
Clean replay success	1.000 on an accepted project
Canonical graph hash stability	1.000 under pinned input/package
Unapproved out-of-scope edit change	0
Pattern/occurrence/BOM cardinality mismatch	0
Silent truncation	0, always

0.15 Audit finding from supplied examples

The supplied task logs are valuable historical behavior traces but are not adequate feature DAGs. For example, `task_103.log` summarizes a 282-face result in four high-level operations, and `task_122.log` summarizes a complex body in five. `task_101.log` includes loop-like phrases such as repeated corners. These are acceptable human plans or recovered summaries only. They must be re-expressed through the schema above before they can be treated as executable training targets.

The edit logs are suitable for learning observation, diagnosis, and change localization after provenance review. They are not hidden reasoning traces and must not be labeled as such.

0.16 Feature-DAG compilation is an incremental domain compile

The phrase **compile the feature tree** has one exact meaning in Forge: validate canonical feature IR, resolve its symbols and dependencies, lower semantic nodes into a prebuilt operation plan, and execute the invalidated subgraph through trusted C++ kernel and simulation operators. It does **not** mean asking Archie to write a `.cpp` file and invoking a general-purpose compiler for every request.

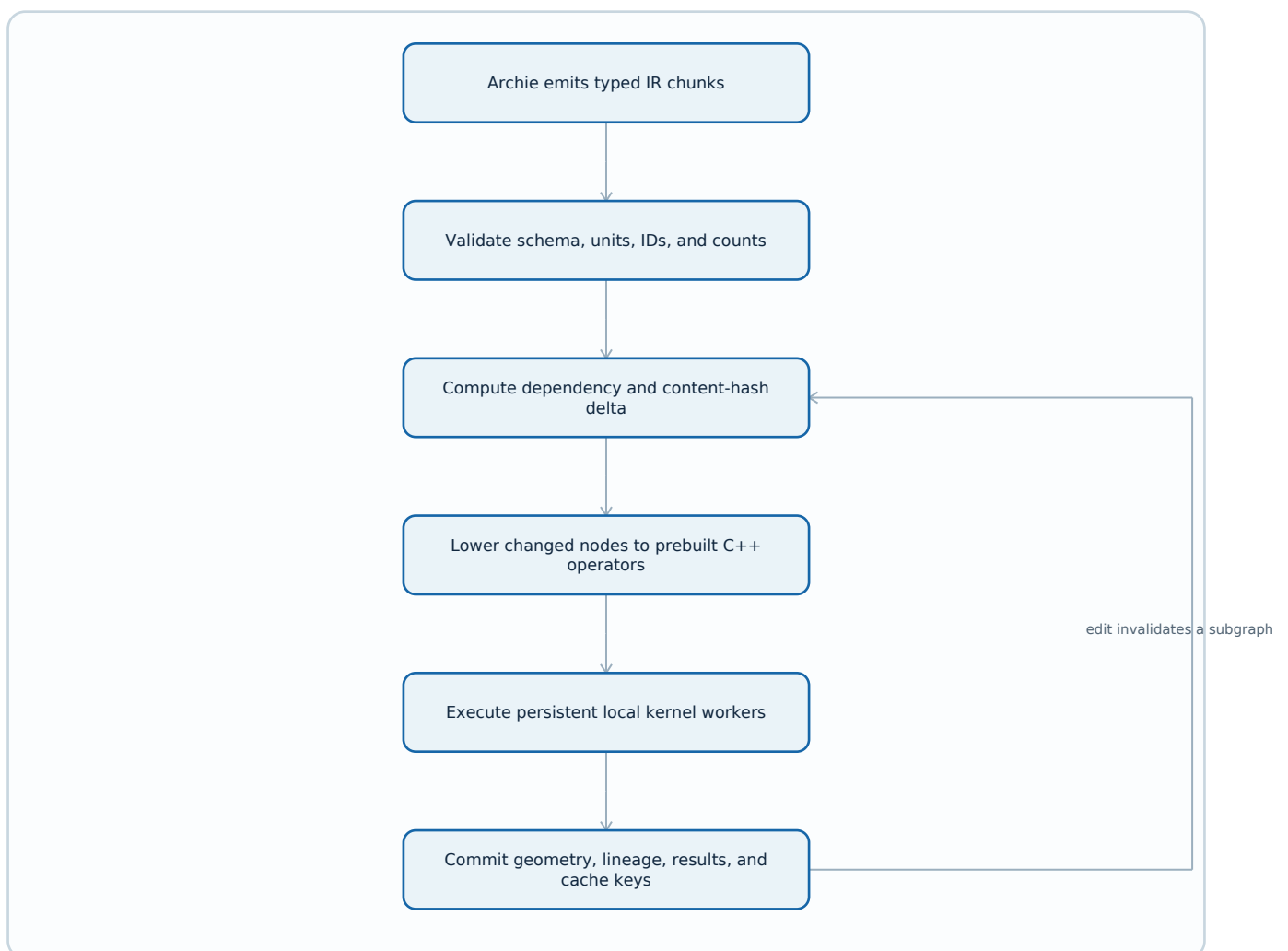


Figure 5. 0.16 Feature-DAG compilation is an incremental domain compile

This boundary gives Archie speed and safety simultaneously:

- kernel and solver code is compiled once during the Forge build;
- a persistent operator registry maps each schema version and feature kind to a typed C++ executor;
- unchanged feature results are reused only when input, dependency, tolerance, configuration, and build fingerprints match;

- independent ready nodes may execute in bounded parallel batches, but commit order remains canonical;
- persistent workers are pre-warmed with the exact geometry and solver stack required by the active project;
- a small edit recompiles and re-executes the minimal valid dependency closure rather than replaying the entire project;
- the model cannot introduce arbitrary code, symbols, dynamic modules, or side effects;
- cold-build time, warm full-graph time, incremental edit time, and cache hit correctness are measured separately.

An optional ahead-of-time library of commonly used operation plans may reduce lowering overhead, but every instantiated node remains explicit in the authoritative graph and count ledger. Cached output is an optimization, never a second source of truth.

1. The governing product axiom

A user gives Archie an engineering request in text, images, video, sketches, drawings, PDFs, existing CAD, or a combination. Archie turns that evidence into a complete, explicit, versioned engineering specification; reasons over competing interpretations; asks only questions that materially change the result; emits an exact and replayable native C++ feature DAG of whatever length the job requires; Forge compiles it into geometry; deterministic validators prove or reject the result; and the user receives editable CAD plus the evidence needed to trust, revise, manufacture, inspect, or simulate it.

In compact form:

```
Evidence -> Specification -> Candidate intent -> Exact feature DAG
          -> Deterministic compilation -> Verification/repair -> Deliverables
```

The feature DAG is the executable design record. The B-rep is a compiled artifact. A render is an observation. None may substitute for another.

2. Non-negotiable laws

1. **The shipped first-party runtime is native C++.** Inference, orchestration semantics, the feature IR, compiler, kernel integration, validation, persistence, UI, and audit logging execute in C++. Offline training and data preparation may use Python because forcing the training ecosystem into C++ would delay the product without making the shipped product more native.
2. **SearXNG is the only production network tool.** A self-hosted SearXNG sidecar runs on the same Mac and the C++ runtime calls its documented local HTTP search API. SearXNG may retrieve public web references only after explicit tool invocation and query redaction. Its implementation does not enter the Forge process, and it cannot supply compute, model inference, geometry, simulation, storage, or executable instructions.
3. **Reasoning may be probabilistic; committed output is deterministic.** Archie may explore multiple candidate interpretations, decompositions, and repair strategies. The selected specification, canonical feature DAG, compiler behavior, and acceptance result are deterministic under a pinned build and input bundle.
4. **The executable feature tree is exact and elastic.** Its length is a consequence of complete design intent, not a token budget, example length, or UI preference. It may be 10×, 100×, or orders of magnitude longer than a preliminary plan. It must never be shortened by replacing required operations with vague prose.
5. **No silent truncation.** If context, time, memory, or compute is exhausted, Archie checkpoints the graph and returns `PAUSED_INCOMPLETE`. It must not claim success, invent a compact substitute, or discard generated work.
6. **No benchmark memorization.** Public evaluation inputs, private ground truth obtained improperly, known answers, and submission artifacts are excluded from training and retrieval. Benchmark optimization means learning transferable construction and editing competence, not answer-key contamination.

7. **No unverifiable completion claims.** A capability is complete only when its stated acceptance gates pass on a clean build and archived, inspectable artifacts exist.
8. **Geometry alone is insufficient.** Design intent, constraints, units, material assumptions, tolerances, assembly relationships, manufacturing context, evidence, and change provenance remain attached to the model.
9. **Safety-critical output is reviewable engineering work, not automatic certification.** Archie can calculate, compare, flag, and prepare evidence; credentialed engineers and authorized organizations retain approval responsibility where law, safety, or policy requires it.
10. **User and company IP stay isolated.** Customer artifacts are not training data by default. Any learning from private work requires explicit, revocable authorization and a documented retention policy.
11. **Standards and datasets require source-use admission.** A public URL is not permission for model training. Every data item carries source, permitted purpose, hash, provenance, applicable terms, and removal lineage.
12. **Failure is explicit and local.** A failed feature, constraint, solver, export, or verification gate reports the precise failing node and preserves the last valid checkpoint.
13. **All engineering execution is fully local.** Model packages, schemas, kernel code, solver code, UI code, dependency sources, compiled dependency prefixes, indexes, project data, and hot runtime caches are present on the M4 workstation before a production job starts. There is no hosted-model, remote-solver, streamed-weight, remote-index, or network-compute fallback. Network loss may make live web research unavailable, but cannot stop intake, reasoning over installed evidence, CAD generation, graph compilation, rendering, simulation, validation, export, or replay.
14. **Feature-graph compilation never means compiling model-authored C++.** Archie emits schema-constrained feature IR. The native graph compiler lowers it into prebuilt, typed C++ operations and incrementally executes only the invalidated dependency region. Model text never enters Clang, a shell, a plug-in loader, or a just-in-time compiler.
15. **Physics is a first-class product path.** At the user's request Archie creates versioned analysis intent, runs an interactive physics loop where the method can meet its declared frame/latency envelope, runs a deterministic engineering-confirmation loop when fidelity or safety demands it, and attaches assumptions, convergence, sensitivity, and validity evidence to the same project graph.
16. **The required application stack is open source and locally buildable.** The core desktop shell, docking, viewport, command system, geometry path, inference host, orchestration engine, and native simulation path must be buildable from available source through the pinned local dependency plane. A proprietary SDK may support an optional interoperability adapter, but Forge's core launch, authoring, replay, simulation, and neutral interchange cannot depend on it.
17. **The Forge product repository reaches a zero-JavaScript end state.** Electron, Node, npm, JavaScript, TypeScript, browser application code, and JavaScript test harnesses are migration inputs only. After native C++ parity and rollback evidence exist, they are removed from the repository and the repository-local build/output stores. Computationally heavy product, kernel, inference, rendering, orchestration, validation, and simulation logic must be C++. Only an explicit minimal allowlist of declarative/build/documentation files, shader languages, generated schemas, and tiny bootstrap wrappers may remain outside C++.
18. **Forge continuously protects workstation storage.** A native C++ storage governor inventories only registered Forge-managed roots, classifies every artifact by owner/reference/reproducibility/retention state, removes disposable data promptly, quarantines recoverable candidates before purge, and refuses to remove any dirty, active, unique, referenced, pinned, rollback, audit, or unresolved artifact. Dead worktrees and superseded weights/datasets/builds are reclaimed through proof-based policies, never filename guesses or broad recursive deletion.

3. What "pure C++" means

The phrase applies to the distributable Forge/Archie application and its first-party execution path, not to every service or tool in the development lifecycle.

Layer	Requirement
Desktop shell, panels, feature tree, viewport, command system	C++
Archie inference host and Metal integration	C/C++ API from C++
Multimodal preprocessing and tokenization in production	C++
Agent/workflow state machine, tools, retries, checkpoints	C++
Engineering Specification Graph and feature DAG	C++ types with a versioned binary/on-disk schema
CAD compiler, OCCT adapters, geometric repair	C++
Verification, export, diffing, observability	C++
Local database and artifact store	Called from C++
SearXNG	Same-Mac sidecar accessed through local HTTP JSON; sole approved production web egress
Training, synthetic-data generation, evaluation research	Separate development workflow; may use non-runtime tools, but cannot create a hosted or vendor-compute dependency in installed Archie
Third-party CAD/CAE interoperation	Local sandboxed adapter, neutral file exchange, or explicit same-machine API boundary; never a hosted service

The existing Electron/JavaScript desktop is migration input, not proof that this requirement is met. The migration is complete only when no JavaScript runtime is required to launch, reason, generate, compile, inspect, or export a normal Archie job.

3.1 Native LangGraph-style behavior

Archie must implement the useful semantics of LangChain/LangGraph rather than embedding their Python packages:

- typed state passed between named graph nodes;
- conditional routing and bounded parallel candidate work;
- durable checkpoints after every state-changing step;
- resumability after crash, cancellation, or power loss;
- interrupts for material user decisions and approvals;
- idempotent tools with explicit side-effect classes;
- retry policies based on error type, not blind repetition;
- subgraphs for intake, research, generation, editing, simulation, and export;
- event streaming to the UI;
- per-node time, memory, token, and artifact accounting;
- deterministic replay from an immutable input bundle and pinned versions.

The reference semantics are described by [LangGraph durable execution](#), [persistence](#), and [interrupts](#). They are behavioral references, not runtime dependencies.

3.2 Zero-JavaScript repository and non-C++ allowlist

The migration target is stronger than "the main executable happens to be C++." The Forge product repository must no longer carry a shadow browser product, Node build graph, JavaScript integration suite, or JavaScript-generated source of engineering truth.

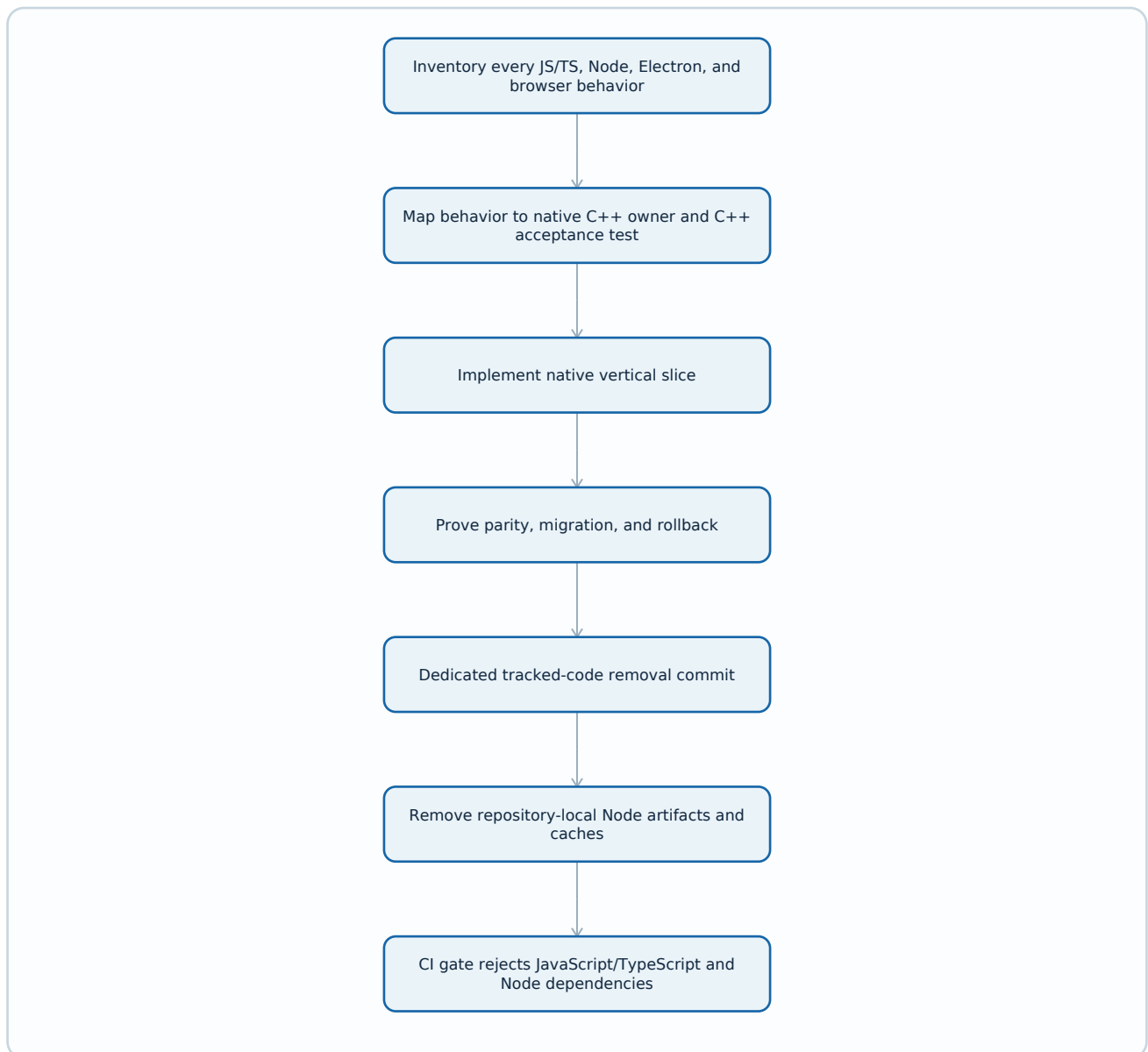


Figure 6. 3.2 Zero-JavaScript repository and non-C++ allowlist

Required removal scope after parity:

- `electron/`, `frontend/`, browser renderer/preload/main-process code, Node services, npm scripts, JavaScript/TypeScript utilities, and JavaScript E2E tests;
- root and nested npm manifests/lockfiles, Electron packaging configuration, Vite/Webpack/Babel/ESLint configuration, and Node-based release paths;
- repository-local `node_modules`, frontend distributions, browser caches, Playwright browser payloads/reports, Electron packages, and other Node-derived build output;
- CI jobs, documentation commands, and developer instructions that make Node or JavaScript a requirement.

Removal is permitted only through a reviewed migration manifest that maps every deleted behavior to its C++ symbol, C++ test, and accepted evidence. Tracked removal occurs in a dedicated Git commit so it is recoverable from history. Local cleanup resolves and prints exact repository-contained targets before deletion; it must never search broadly outside the Forge checkout or delete user assets by extension alone.

The remaining non-C++ allowlist is narrow:

Category	Permitted purpose	Constraint
CMake and toolchain files	configure/build/test the C++ system	no hidden application behavior

Category	Permitted purpose	Constraint
JSON/YAML/TOML and schemas	declarative manifests, presets, fixtures, generated contracts	parsed and validated by C++ where used at runtime
Markdown and rendered documents	architecture, evidence, user/developer documentation	non-executable
Metal/graphics shader sources	GPU rendering and compute	ahead-of-time built, pinned, validated, no general application orchestration
Small shell/bootstrap wrappers	invoke deterministic build/dependency commands	thin wrappers only; complex logic moves to C++ tools
Offline model-training research	model development outside the distributable Forge application	isolated from the product runtime and excluded from the final zero-JavaScript gate

Every non-C++ executable file receives an allowlist entry with owner, reason, size, call graph, and planned replacement where applicable. CI fails on any `.js`, `.jsx`, `.mjs`, `.cjs`, `.ts`, `.tsx`, Node/Electron/npm dependency, or unapproved executable language file added to the target repository after the cutover.

4. Hardware truth and resource envelope

The primary machine is an Apple M4 Max Mac Studio with 36 GB unified memory. Serial numbers, hardware UUIDs, provisioning identifiers, and other device secrets do not belong in project documentation.

4.1 Consequences of the 36 GB envelope

- A full-precision 30B-class model does not fit with useful runtime headroom.
- The preferred local form is a quantized mixture-of-experts vision-language model, loaded through a Metal-capable C++ runtime.
- Model weights, vision projector, KV cache, image tensors, geometry kernel, viewport, OS, and application all compete for unified memory.
- Context length must be allocated deliberately; "maximum advertised context" is not the same as a usable interactive context on this machine.
- Full parameter training is not an on-device plan. Domain adaptation uses LoRA/QLoRA or other parameter-efficient methods on suitable training hardware, followed by local quantized inference.
- Large feature DAGs live in the persistent graph store and are retrieved in dependency-aware slices; they are never kept solely in the language-model context.

4.2 Memory budget gate

Before a model/runtime combination is promoted, the benchmark harness must record at least:

Consumer	Measurement
Quantized language weights	mapped and resident bytes
Vision encoder/projector	mapped and peak resident bytes
KV cache	bytes by context length and cache precision
Input images/pages	decoded and encoded peak bytes
Forge kernel and model	steady and peak bytes by model complexity
Viewport meshes/textures	steady and peak bytes
Workflow/checkpoint buffers	steady and peak bytes
OS safety reserve	observed pressure margin under sustained work

The release target is no swap thrashing during the representative interactive suite, no operating-system memory-pressure termination, and a documented fallback for exceptionally large projects.

4.3 Throughput and latency

"50 tokens per second" is a target to measure, not a promise. Report prompt-processing and generation throughput separately, with model quantization, context size, image count, Metal offload, thermal state, and concurrent kernel load. CAD task completion time is the governing metric; a slower model that produces a correct graph in one pass can outperform a fast model that causes repeated rebuilds.

5. Model architecture

5.1 Recommended foundation

The initial 30B-class candidate is [Qwen3-VL-30B-A3B-Instruct](#), an Apache-2.0 multimodal mixture-of-experts model listed as 31B total parameters. Its spatial perception, OCR, image/video understanding, and approximate 3B active-parameter routing are a better match to the hardware and multimodal CAD intake than a dense 30B model. The official [Qwen GGUF release](#) provides a concrete local deployment path.

This is a candidate, not an irrevocable identity. A challenger replaces it when the pinned whole-system evaluation proves better CAD correctness, edit survival, ambiguity handling, and latency within the same resource envelope.

5.2 Production inference

The primary engine is a pinned, regression-tested build of [llama.cpp](#): a C/C++ inference runtime with Metal support, GGUF loading, quantization, CPU/GPU hybrid execution, and grammar-constrained generation. [MLC LLM](#) and [ONNX Runtime Core ML](#) are challengers, not simultaneous production dependencies.

Grammar-constrained decoding reduces malformed output but is not a correctness boundary. Archie must still parse, type-check, resolve, compile, and validate the IR independently. A grammar failure or runtime regression must fail closed. Runtime commits, quantization tools, templates, chat formatting, tokenizer, vision projector, and sampler settings are pinned as one model package.

5.3 Dual-mode behavior

Archie uses two logically distinct modes:

Exploration mode

- temperature and sampling may be non-zero;
- multiple interpretations or plans may be generated;
- candidates may disagree;
- retrieval may expand the evidence set;
- candidates are ranked by deterministic feasibility and verification signals;
- the system records a concise decision rationale, not hidden chain-of-thought.

Commitment mode

- fixed model package and prompt template;
- deterministic or lowest-variance supported sampler;
- fixed seed where the runtime honors it;
- schema-constrained output;
- canonical units, numeric representation, ordering, and serialization;
- no unresolved identifiers or free-form executable prose;
- compiler and verifier decide acceptance.

Repeated commitment from the same normalized specification and pinned package must yield the same canonical graph hash. Where floating-point or parallel kernel behavior prevents bitwise geometry identity, the release contract defines platform-specific tolerances and a stable semantic/topological fingerprint.

5.4 Specialist system, not one undifferentiated prompt

The 30B model is the general reasoner. Small specialist adapters, deterministic algorithms, retrieval indexes, and solvers provide depth:

- drawing/OCR and dimension interpretation;
- sketch constraint synthesis;
- feature-history planning;
- topology reference selection;
- edit localization and change-impact prediction;
- assembly and mate reasoning;
- DFM and process planning;
- GD&T and drawing production;
- geometric failure diagnosis;
- benchmark-specific I/O adapters that do not alter core policy.

Routing is based on the typed specification and observed failure class. A specialist may propose changes; it may not bypass the feature-graph compiler or verification gates.

6. End-to-end execution pipeline

Every job is an append-only, resumable workstream with immutable inputs and versioned derived artifacts.

Phase	Required output	Blocking gate
1. Ingest	hashed evidence bundle	every file readable or explicitly rejected
2. Normalize	units, views, OCR, geometry metadata, source mapping	transformations reproducible
3. Specify	Engineering Specification Graph (ESG)	no hidden assumptions
4. Disambiguate	ambiguity ledger and resolved decisions	material ambiguities resolved or user-approved
5. Plan	semantic construction plan and graph-size forecast	all requirements assigned
6. Emit	chunked exact feature DAG	schema/count/reference gates pass
7. Compile	native features, bodies, occurrences, drawings	deterministic compiler succeeds
8. Observe	canonical views, sections, topology and metrics	observation set complete
9. Verify	geometry, requirements, edits, manufacturing, optional physics	all applicable blockers pass
10. Repair	minimal scoped patch and replay	no regression outside allowed change set
11. Deliver	editable native project plus neutral exports and report	artifact manifest complete
12. Learn	permission-filtered failure/example record	no private or evaluation leakage

No phase may erase the evidence or decisions that produced the next phase. A later repair creates a new graph revision linked to its parent.

7. Multimodal intake and ambiguity handling

7.1 Accepted evidence

Archie must support mixed bundles of:

- natural language, tables, equations, and requirements documents;
- raster images and photographs;
- hand sketches, whiteboard drawings, and annotated screenshots;
- vector/PDF engineering drawings, including multi-page drawing sets;
- STEP, IGES, B-rep, mesh, point-cloud, and existing native Forge projects;
- DXF/DWG-derived drawing content through approved readers;
- BOM, material, supplier, tolerance, and process data;
- short videos or turntables where they disambiguate shape or mechanism;
- prior revisions and explicit edit requests.

Every source receives [SourceEvidenceId](#), cryptographic hash, original filename, media type, page/frame/view coordinates, ingest time, and permission label. OCR text and derived measurements retain a pointer to the exact source region.

7.2 Normalization

Normalization must preserve originals and produce:

- canonical SI values with original display units retained;
- page/view orientation and drawing scale hypotheses;
- dimension, tolerance, symbol, note, leader, and title-block candidates;
- visible/hidden/center/construction line classification;
- orthographic-view correspondence candidates;
- detected datums, section arrows, detail views, revision clouds, and BOM balloons;
- image camera hypotheses and uncertainty;
- CAD topology, units, coordinate systems, metadata, and repair status;
- contradictions between sources rather than silent precedence.

7.3 Ambiguity ledger

Ambiguity is data, not a conversational failure. Each ambiguity record includes:

```
AmbiguityId
question or disputed proposition
candidate interpretations
supporting and contradicting SourceEvidenceIds
effect on geometry, function, manufacturing, safety, cost, or schedule
reversibility and downstream blast radius
confidence/calibration bin
resolution policy
selected answer and decision owner
```

Archie asks a question when the alternatives materially change safety, interfaces, fit, function, manufacturing process, cost class, legal compliance, or a large downstream region. It may select a reversible conventional default for low-impact ambiguity, but it must show that default before final acceptance and retain it as an assumption.

Question order is based on expected information gain multiplied by downstream impact. Questions are batched only when answers are independent and a batch will not overwhelm the user.

7.4 No hallucinated precision

An unscaled photo does not establish a 25.000 mm dimension. An occluded face does not establish a hole. A generic material name does not establish a certified grade or heat treatment. Archie must distinguish observed, inferred, retrieved, user-specified, defaulted, and calculated facts.

8. Engineering Specification Graph

The ESG is the complete semantic contract before geometry. It prevents a visually plausible model from quietly violating the request.

8.1 Required entity families

- requirements, subrequirements, priorities, and acceptance criteria;
- functions, flows, loads, operating states, and failure conditions;
- dimensions, limits, fits, tolerances, datums, and surface requirements;
- materials, treatments, finishes, colors, and prohibited substances;
- manufacturing processes, tooling directions, stock constraints, and inspection plans;
- interfaces, keep-outs, envelopes, connectors, fasteners, and service clearances;
- parts, bodies, assemblies, configurations, variants, and quantities;
- kinematic relationships, degrees of freedom, and motion envelopes;
- simulation assumptions, load cases, mesh controls, and allowable responses;
- standards clauses or authorized references with edition and applicability;
- evidence, assumptions, ambiguities, decisions, risks, and confidence;
- delivery formats, drawing sheets, metadata, and revision policy.

8.2 Traceability

Every MUST-level requirement maps to one or more planned feature nodes and one or more verification predicates. Every feature node maps back to a requirement, construction necessity, or verified reusable library component. Untraceable decorative or speculative geometry is rejected unless the user authorizes creative latitude.

The minimum traceability chain is:

```
SourceEvidence -> ESG requirement/decision -> FeatureId(s)
               -> compiled topology/body -> verification result -> deliverable
```

The release gate is 100% mapping coverage for in-scope requirements. "Not applicable," "deferred," and "requires human approval" are explicit states, never missing rows.

8.3 Requirement conflict policy

When requirements conflict, Archie must not optimize silently. It produces the smallest conflict set, explains the physical or logical incompatibility, and offers ranked resolutions. User priority wins unless it creates an unapproved safety, legal, or impossibility condition.

9. Native C++ feature IR and compiler

9.1 IR design principles

The feature IR is a typed engineering language, not arbitrary C++ source pasted into the application. It must be easy for a model to emit, strict for a compiler to reject, safe to migrate, and stable enough for decades of project history.

- **Closed executable vocabulary:** feature kinds are registered, versioned types. Unknown kinds are rejected.
- **Canonical representation:** normalized UTF-8 identifiers, canonical SI values, fixed ordering, finite numeric values, and deterministic floating serialization.
- **Explicit schemas:** no executable `std::any`, untyped property bags, arbitrary shell commands, or embedded scripts.
- **Separate definition and evaluation:** the serialized graph describes intent; trusted C++ code performs the operation.
- **Schema evolution:** migrations are explicit, reversible when possible, and validated against golden projects.
- **Content addressability:** immutable node payloads, chunks, library components, and evidence use hashes.
- **No raw topology indices:** stable queries and lineage tokens select geometry.
- **No hidden global state:** tolerances, units, coordinate frames, materials, kernel settings, and configurations are project or node inputs.
- **Fail-closed parsing:** duplicate IDs, unknown fields where prohibited, overflow, NaN, infinity, cyclic dependencies, and unresolved references stop compilation.

9.2 Logical C++ type model

The exact implementation may evolve, but it must preserve these semantics:

```
using FeatureId      = StrongId<struct FeatureTag>;
using RequirementId  = StrongId<struct RequirementTag>;
using EvidenceId     = StrongId<struct EvidenceTag>;
using BodyId        = StrongId<struct BodyTag>;
using OccurrenceId  = StrongId<struct OccurrenceTag>;

struct Quantity {
    long double si_value;
    UnitDimension dimension;
    std::string display_unit;
};

struct StableQuery {
    FeatureId ancestry_root;
    SemanticRole role;
    GeometrySignature expected;
    AdjacencyPredicate adjacency;
    CardinalityPolicy cardinality;
};

struct FeatureCommon {
    FeatureId id;
    SchemaVersion schema;
    FeatureKind kind;
    std::string semantic_name;
    std::vector<RequirementId> requirements;
    std::vector<EvidenceId> evidence;
    std::vector<FeatureId> dependencies;
    Preconditions preconditions;
    Postconditions postconditions;
    ConfigurationExpression activation;
    FailurePolicy failure_policy;
    OrderingKey order;
```

```
};

using FeaturePayload = std::variant<
    DatumPayload, SketchPayload, ExtrudePayload, RevolvePayload,
    SweepPayload, LoftPayload, HolePayload, FilletPayload,
    PatternPayload, SheetMetalPayload, SurfacePayload,
    BooleanPayload, DirectEditPayload, OccurrencePayload,
    MatePayload, DrawingPayload, ManufacturingPayload,
    SimulationPayload, VerificationPayload>;

struct FeatureNode {
    FeatureCommon common;
    FeaturePayload payload;
    ContentHash hash;
};
```

The model does not need to emit raw C++ syntax. It emits a canonical schema-constrained representation that the C++ parser converts to these types. Internally generated native C++ builders use the same validation path.

9.3 Compiler stages

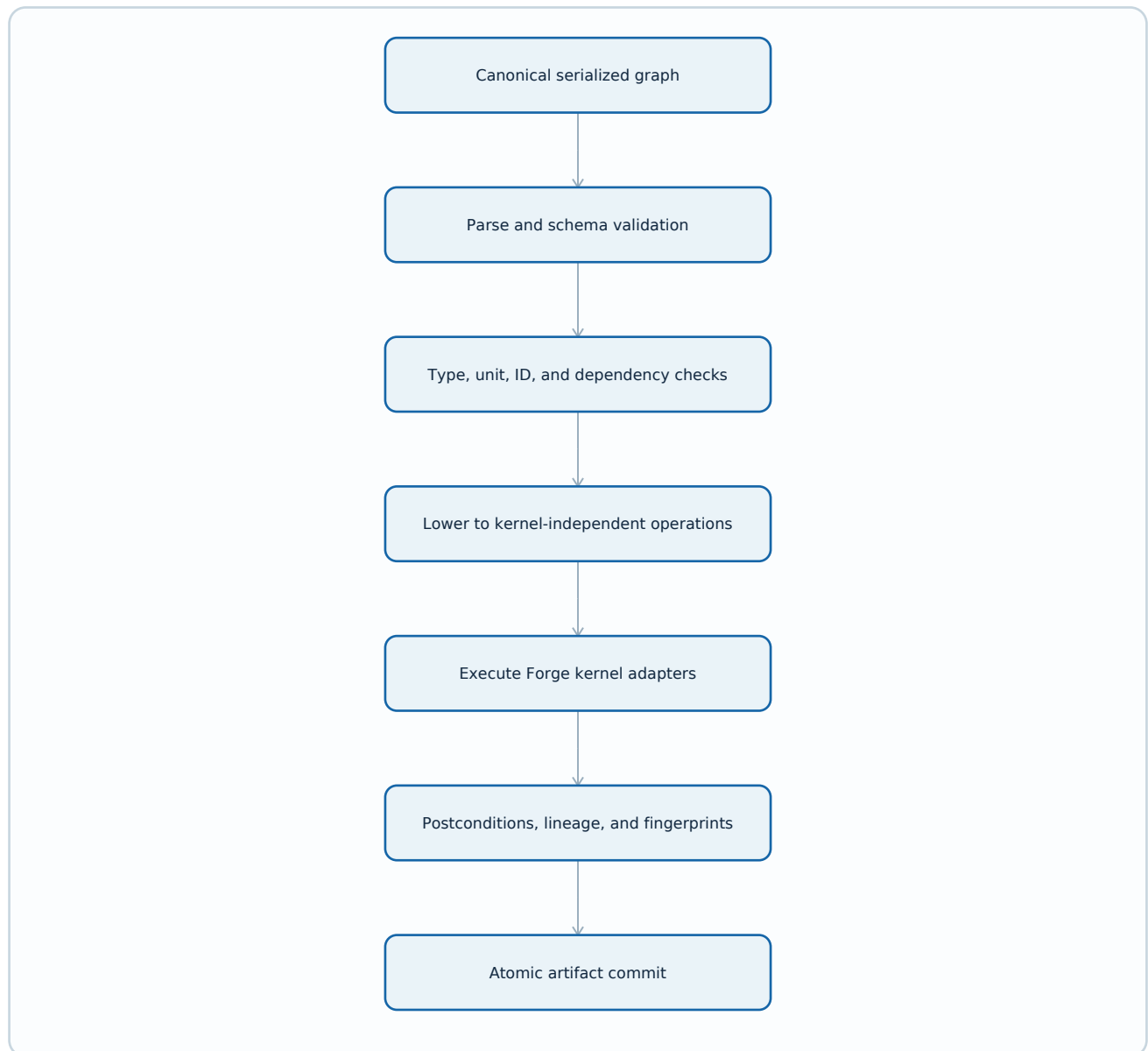


Figure 7. 9.3 Compiler stages

1. **Lex/parse:** reject malformed or noncanonical input.

2. **Schema validation:** enforce kind-specific required fields, ranges, and forbidden combinations.
3. **Symbol resolution:** resolve IDs, configurations, library versions, units, and evidence pointers.
4. **Graph validation:** enforce acyclicity within the construction dependency graph; feedback belongs in versioned workflow iterations, not cyclic feature dependencies.
5. **Static engineering checks:** impossible dimensions, negative thickness, invalid tolerance order, impossible pattern count, and obvious self-reference.
6. **Lowering:** translate semantic features into a kernel-neutral operation plan.
7. **Execution:** call trusted adapters with transaction boundaries.
8. **Healing policy:** apply only declared, bounded healing; record all tolerance changes and topology effects.
9. **Postconditions:** validate body count, volume/surface class, topology expectations, dimensions, and lineage.
10. **Commit:** atomically persist feature result, dependency fingerprint, kernel trace, and observation artifacts.

9.4 Determinism envelope

The graph compiler pins:

- IR schema and migration version;
- C++ compiler and build flags;
- dependency versions and patches;
- kernel versions and modeling tolerance policy;
- CPU/GPU execution policy where it affects results;
- locale, unit registry, rounding mode, and deterministic seeds;
- model package for regenerated model-authored graphs.

Bitwise equality is required for graph serialization and metadata. Geometry equality is defined by a tiered fingerprint: exact kernel serialization where stable; entity lineage and topological invariants; analytic surface/curve parameters; mass properties and bounding boxes within stated tolerance; and sampled Hausdorff/volumetric checks as secondary evidence.

9.5 Sandboxing and resource guards

Model output never becomes arbitrary native code. Each feature executor enforces maximum local allocations, time budgets, recursion limits, geometry-complexity forecasts, cancellation checks, and bounded diagnostic output. A budget overrun pauses or fails the node without corrupting the project.

Imported files, compressed archives, PDFs, and meshes are treated as untrusted. Parsing occurs in restricted worker processes where practical, with size, page, entity, nesting, and decompression limits.

9.6 Prebuilt operator ABI and incremental execution cache

Every executable feature kind has a versioned C++ operator contract:

```
struct OperatorKey {
    FeatureKind kind;
    SchemaVersion schema;
    KernelProfile kernel_profile;
};

struct ExecuteRequest {
    const FeatureNode& node;
    std::span<const DependencyArtifact> dependencies;
    const ProjectExecutionContext& context;
    CancellationToken cancellation;
};

struct ExecuteResult {
    NodeStatus status;
```

```
ArtifactSet artifacts;  
TopologyLineage lineage;  
VerificationEvidence evidence;  
ResourceAccounting resources;  
};
```

The operator registry is assembled and validated at application startup. Operators are ordinary prebuilt C++ functions or isolated persistent C++ workers selected by explicit policy. Runtime discovery may read signed manifests, but it may not execute a library that was absent from the activated local build manifest.

The node cache key contains the canonical node hash, ordered dependency artifact hashes, active configuration, unit/tolerance policy, operator ABI, kernel/solver build fingerprint, deterministic execution profile, and required external reference hashes. A cache hit must still pass cheap structural checks. Random spot re-execution detects poisoned or incorrectly keyed cache entries.

Parallel execution follows the graph frontier. The scheduler may run independent nodes concurrently, reserve unified-memory budgets before launch, and reduce concurrency under thermal or memory pressure. It may never reorder canonical commits, expose half-committed artifacts, or accept a descendant whose dependency failed.

10. Forge geometry kernel architecture

Forge is a kernel federation with a single semantic contract, not a bag of geometry libraries called opportunistically.

10.1 Kernel roles

Component	Primary role	Boundary and caution
Open CASCADE Technology	B-rep modeling, NURBS/analytic geometry, STEP/IGES, topology, meshing, visualization primitives	Primary exact CAD kernel; pin source revision, patches, build flags, and tolerance policy.
CGAL	Robust computational geometry, arrangements, meshes, polygon operations, spatial algorithms	Admit only the exact packages recorded in the activated dependency manifest and regression suite.
libfive	Implicit/F-rep modeling and procedural fields	Use behind an explicit conversion/validation boundary; audit numerical behavior and provenance.
PicoGK	Voxel/implicit computational engineering and lattice workflows	Useful for algorithmic geometry; not a substitute for editable B-rep history.
Manifold	Robust manifold triangle-mesh booleans and mesh operations	Valid manifold input, precision, and representation-conversion contracts must be enforced.
Custom Forge code	Feature semantics, stable naming, constraints, history, healing policy, interchange, validation	This is where the unified behavior and long-term design intent live.

No library is treated as universally superior. Dispatch depends on representation and required result:

- exact prismatic/freeform engineering solids default to OCCT B-rep;
- planar/arrangement or specialized robust geometry may use audited CGAL packages;
- field-driven lattices and graded structures use an implicit/voxel path;
- mesh repair and mesh Boolean operations may use Manifold;
- conversions are explicit feature nodes with accuracy, watertightness, and lineage reports.

10.2 Capability matrix, not fictional parity

"1:1 parity with Parasolid and ACIS" is not a truthful completion claim without access to proprietary specifications and exhaustive certification. The practical target is a public, versioned capability matrix against the operations and interoperability engineers need.

Each capability row records:

- semantic operation and supported parameter space;

- representation and backend;
- exactness/tolerance contract;
- known degeneracies and fallback behavior;
- performance envelope;
- regression corpus count and pass rate;
- import/export preservation results;
- comparison evidence against permitted reference systems;
- production status: experimental, bounded, or released.

The matrix covers sketching, parametric solids, surfacing, direct modeling, assemblies, sheet metal, weldments, frames, routing, drawings, GD&T/PMI, configurations, freeform/subdivision, reverse engineering, additive/lattice, CAM preparation, visualization, and neutral/native exchange.

10.3 Geometric robustness doctrine

- Maintain one documented modeling tolerance hierarchy; do not scatter epsilon literals.
- Scale-sensitive algorithms normalize or use adaptive/exact predicates where suitable.
- Booleans preflight operands, intersections, sliver risk, and nonmanifold conditions.
- Healing is never an invisible "make valid" button. Each modification is bounded and audited.
- Fillet/chamfer operations support variable radius/setback where the backend can prove continuity; failure reports the exact edge set and limiting condition.
- Surface operations report continuity class and deviation at joins.
- Tessellation is a derived view with chordal/angular tolerances, not the source of truth for B-rep parts.
- Conversions report volume, area, boundary, Hausdorff, and topology deltas.
- Import keeps original bytes, parser diagnostics, unit evidence, and a quarantine state for suspect geometry.

10.4 Sketch constraint system

The constraint solver is an independent subsystem with symbolic/structural preprocessing, numerical solve, conflict isolation, redundancy detection, degree-of-freedom reporting, and deterministic tie-breaking. It needs a benchmark corpus of under-, fully-, over-, and inconsistently constrained sketches, not only successful rectangles.

Third-party solvers may inform implementation, but the shipped implementation and every imported source fragment must be admitted by the separate source-use policy. The engineering architecture in this document remains independent of that review.

10.5 Import and export

At minimum, the roadmap covers STEP AP203/AP214/AP242 as applicable, IGES, STL, OBJ, 3MF, glTF/GLB, DXF, SVG, PDF drawings, and native Forge packages. Export does not mean "a file was written." Gates verify:

- parser round-trip and unit preservation;
- body/shell/face/edge counts where the format supports them;
- assembly hierarchy, names, colors, layers, and metadata;
- PMI/GD&T preservation where supported;
- geometry deviation and mass-property change;
- watertight/manifold state for mesh manufacturing formats;
- no missing external references;
- deterministic package manifest and hashes.

Proprietary native formats require approved vendor SDKs or controlled host-application adapters. Reverse engineering must remain lawful and isolated from the core format contract.

10.6 Fully local kernel and solver dependency plane

Forge MUST be buildable and operable from a complete local dependency plane. The repository records immutable dependency identities and patches; the workstation stores verified source archives, expanded source trees, compiled prefixes, debug symbols, and binary caches. Production execution must never fetch kernel code, solver code, shaders, schemas, plug-ins, or model shards on demand.

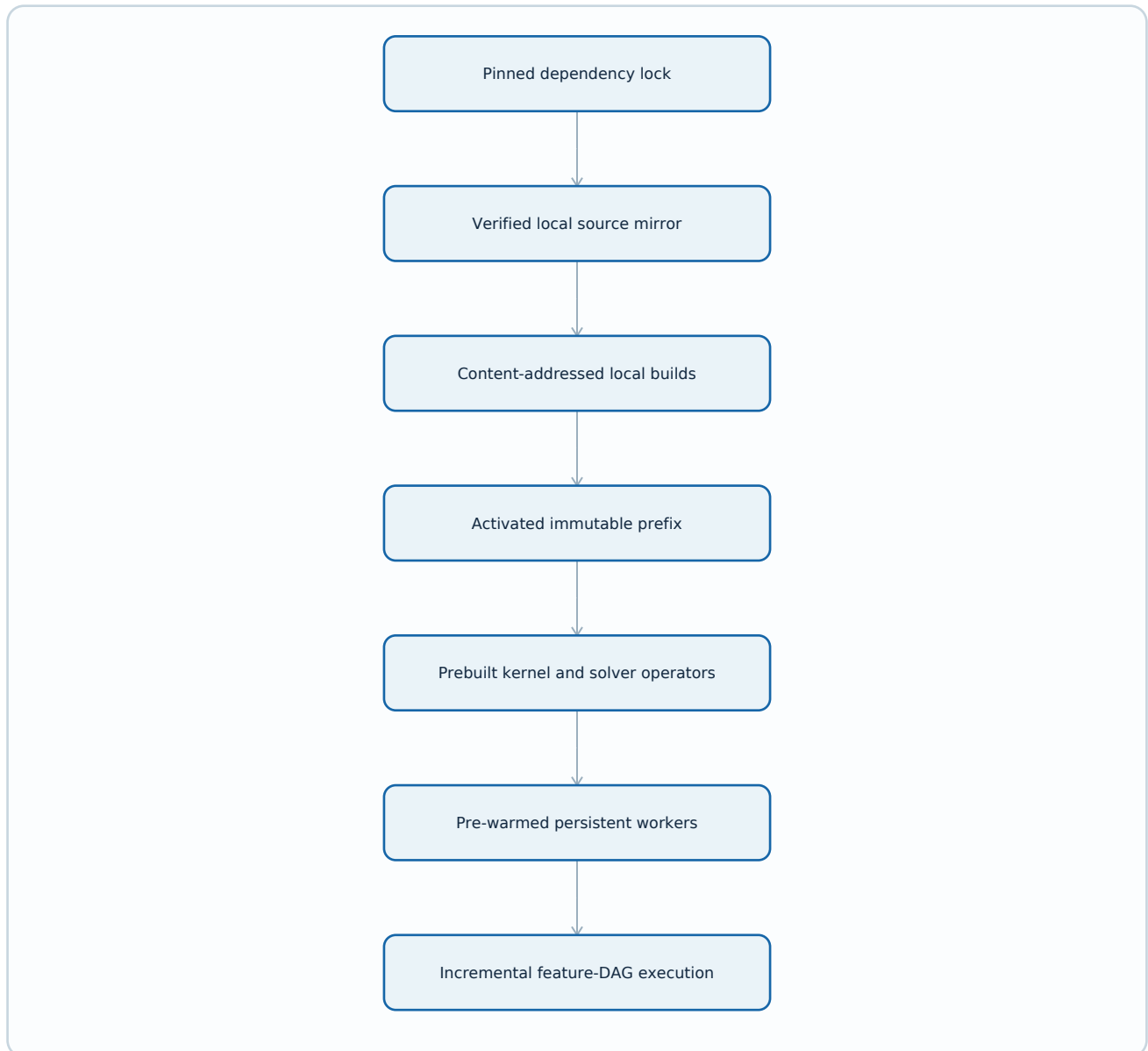


Figure 8. 10.6 Fully local kernel and solver dependency plane

Required layout semantics:

```

third_party/
  manifest/deps.lock.json    # name, exact revision, archive hash, patches, features
  ports/                    # Forge-controlled build recipes
  patches/                  # reviewable deterministic patches
  notices/                  # generated source-use and attribution records outside Sacrosanct
.forge-local/               # ignored workstation state
  sources/<name>/<revision>/
  builds/<triplet>/<build-hash>/
  prefixes/<triplet>/<build-hash>/
  binary-cache/<artifact-hash>/
  model-cache/<package-hash>/
  
```

Only manifests, recipes, patches, and reproducibility metadata belong in Git. Expanded third-party sources, build trees, compiled objects, model weights, and caches stay local or in an approved artifact store and are excluded from ordinary commits. The bootstrap process has two explicit modes:

1. **ONLINE_SEED**: obtain exactly the missing pinned artifacts, verify hashes, and populate the local mirror;
2. **OFFLINE_BUILD**: block network access, build only from the mirror, run dependency smoke tests, and activate an immutable prefix atomically.

CMake Presets define target triplets and clean/warm workflows. Ninja performs builds. A compiler cache may accelerate identical compilation results. A local package binary cache and asset cache avoid redundant source builds. The build must support **FORGE_NETWORK=OFF**, and CI must prove a clean offline build using only the exported dependency bundle.

The local dependency plane is not copied wholesale into the application process. In-process components are used where latency, memory safety, and ABI stability are proved; complex or fault-prone parsers and solvers may run in persistent local C++ workers over versioned IPC. Either boundary is local, pinned, pre-warmed, and observable.

11. Durable native agent architecture

Archie's orchestration is a versioned C++ state graph. It is not a single unbounded model response and not a chain of ad hoc prompts.

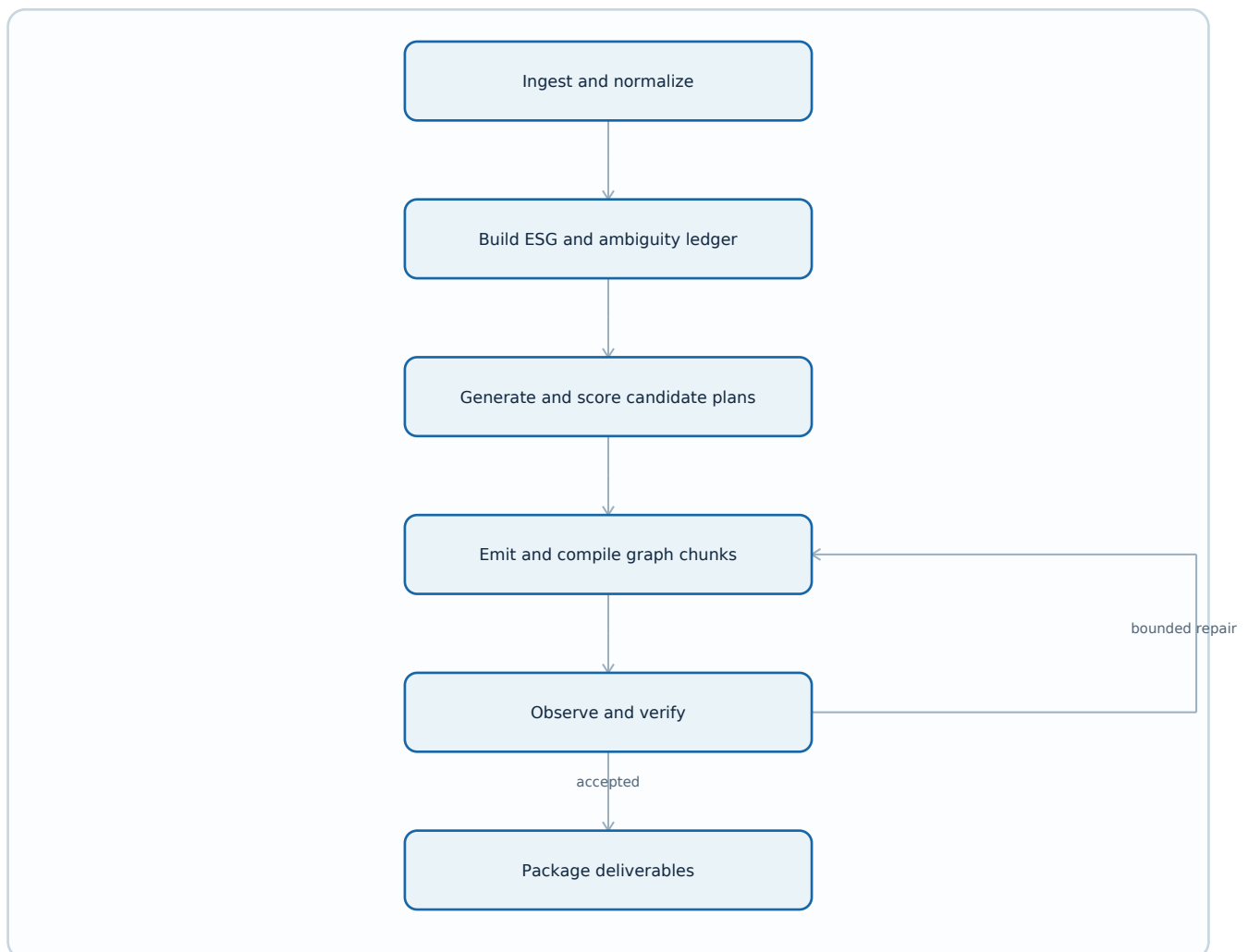


Figure 9. 11. Durable native agent architecture

11.1 Typed workflow state

Every workstream state includes:

- immutable input-bundle hash;
- current ESG revision and ambiguity ledger;
- active candidate set and deterministic scores;
- graph header, committed chunk tip, and count ledger;
- compiled artifact manifest and observation set;
- gate results, failure taxonomy, and repair budget;
- user approvals and pending interrupts;
- model/runtime/tool/kernel package hashes;
- cost, token, time, memory, and retry accounting;
- privacy, retention, and network policy labels.

11.2 Node contract

Each workflow node declares inputs, outputs, read effects, write effects, idempotency key, timeout, retryable error classes, checkpoint policy, and human-approval class. The scheduler must not replay a non-idempotent project mutation, local export, or locally attached physical action merely because a process response was lost.

Tool calls belong to one of these side-effect classes:

Class	Examples	Default behavior
Pure/read-only	inspect graph, calculate property, search local index	retry safely under policy
Reproducible derived write	render view, compile chunk, create local report	write content-addressed artifact
Project mutation	commit graph revision, update drawing	transactional and versioned
Reviewed local package	prepare RFQ/ECO/release files for manual transfer	preview, local export, and approval record; Archie does not transmit it
Locally attached high-impact action	write an approved controller package to explicit removable/local media	separate authorization, domain interlock, and no direct machine motion

11.3 Candidate reasoning

For ambiguous or difficult jobs Archie may produce several candidate ESG resolutions or construction plans. A deterministic scorer uses:

- requirement coverage;
- predicted compile feasibility;
- constraint solvability;
- design-intent/editability score;
- manufacturing/process plausibility;
- interface preservation;
- complexity and performance cost;
- precedent similarity from admitted training/reference data;
- uncertainty and assumption impact.

The selected candidate is then emitted through commitment mode. Candidate diversity must not leak into nondeterministic final serialization.

11.4 Error taxonomy and repair

Errors are classified before retry:

- malformed IR;
- schema/type/unit/reference error;
- impossible or contradictory specification;
- sketch conflict or underconstraint;
- kernel numeric/degeneracy failure;
- topological naming ambiguity;
- verification mismatch;
- manufacturing or physics violation;
- data/tool/network failure;
- resource exhaustion;
- policy, permission, or source-use block.

The system chooses a repair operator specific to the class. Blindly resampling the entire answer is prohibited after a localized failure is known.

11.5 Memory architecture

Archie uses four isolated memory classes:

1. **Project truth:** versioned ESG, graph, artifacts, decisions, and approvals. This is authoritative.
2. **Engineering reference memory:** admitted standards excerpts, manuals, material/process data, and reusable patterns with citations.
3. **Learning memory:** permission-filtered examples and failure records eligible for training/evaluation.
4. **Ephemeral reasoning state:** temporary candidate context deleted according to policy.

Retrieval is scope- and permission-aware. A textual similarity match cannot override project truth or expose another customer's data.

12. SearXNG retrieval and evidence architecture

[SearXNG](#) is Archie's real-time metasearch gateway. The C++ client uses the documented [Search API](#) through GET or POST requests to [/search](#), preferably against a self-hosted instance with JSON enabled.

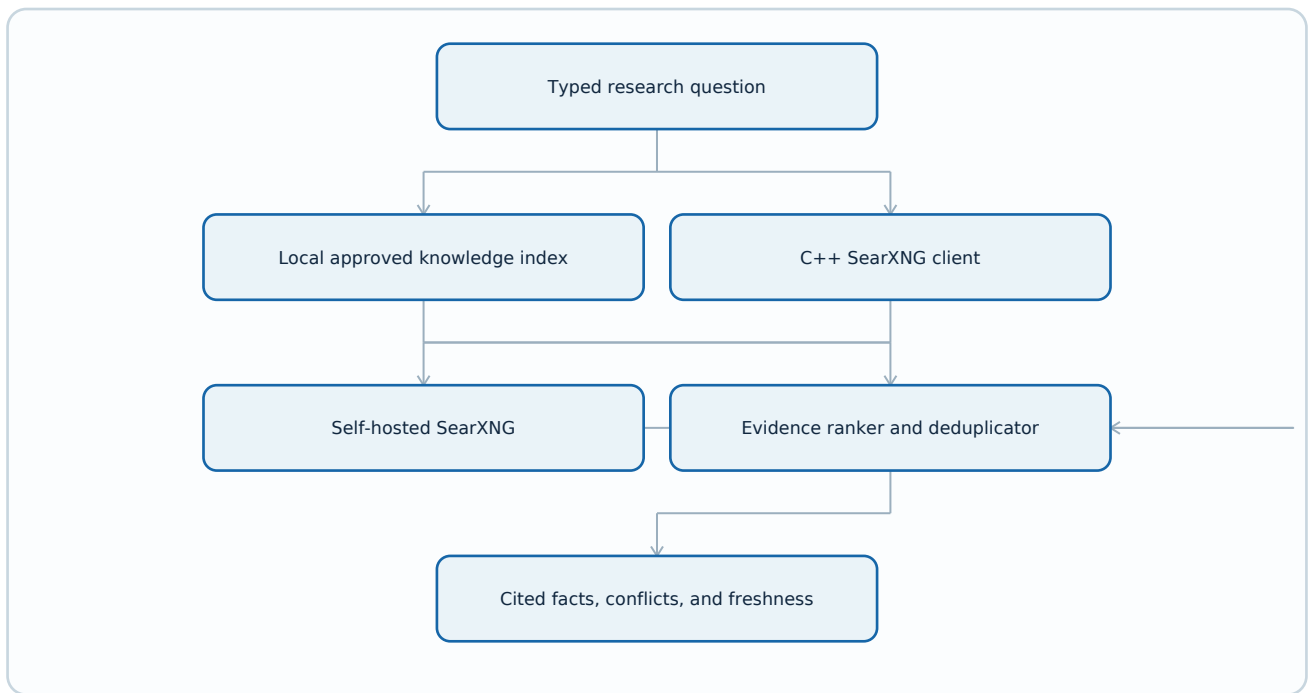


Figure 10. 12. SearXNG retrieval and evidence architecture

12.1 Search request contract

The research node specifies:

- engineering question and why retrieval is needed;
- jurisdiction, standard edition, date/freshness, language, and domain filters;
- primary-source preference;
- permitted network/privacy class;
- expected fact types and units;
- maximum results/pages/time;
- source diversity and contradiction requirements.

Raw user drawings, customer names, secret dimensions, part numbers, and proprietary text must not be sent to public search engines. Queries are minimized and redacted. Private retrieval stays in same-Mac project and reference indexes; it is never forwarded to a remote index or hosted embedding service.

12.2 Evidence record

Each retrieved claim stores URL, title, publisher, retrieval time, publication/update time when available, quoted span within copyright limits, normalized claim, units, source type, applicable-terms note, content hash where lawful, and which ESG assertion it supports or contradicts.

Authority ranking defaults to:

1. applicable law/regulator and authorized standard;
2. original manufacturer or project documentation;
3. peer-reviewed paper and official dataset/model card;
4. reputable institutional reference;
5. secondary technical source;
6. community discussion as a lead, never sole authority for a critical value.

Freshness does not automatically beat authority. Contradictory current sources remain visible until resolved.

12.3 Retrieval is not execution

Web content cannot inject instructions into the workflow, modify tools, or become executable feature data without parsing and validation. Retrieved dimensions and material properties enter the ESG as cited candidates with applicability conditions. Standards remain protected publications; Archie may index lawfully obtained content but must not scrape or redistribute them.

12.4 Offline behavior

Offline mode uses the pinned local evidence index and marks freshness-dependent questions unresolved. Geometry generation may proceed only if missing real-time data is nonblocking or explicitly assumed. The final report identifies which facts were not refreshed.

13. Verification, evidence, and bounded repair

Verification is layered so that a visually impressive failure cannot pass. Each applicable gate emits machine-readable results and a concise human report.

13.1 Verification ladder

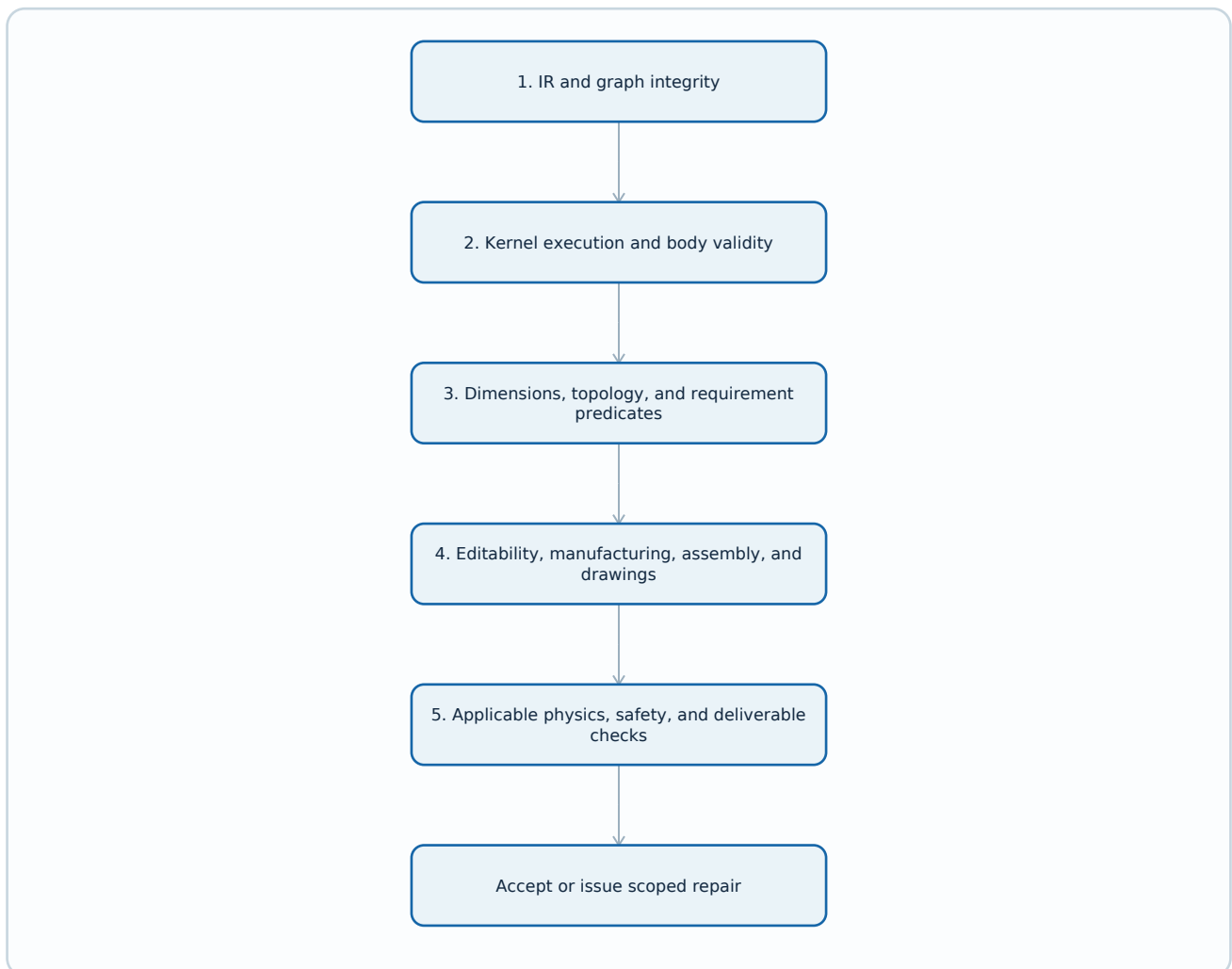


Figure 11. 13.1 Verification ladder

Gate 1: graph integrity

- schema, IDs, units, types, acyclicity, references, chunks, hashes, counts, and requirement mapping;
- no executable prose placeholders;

- no forbidden or unapproved external dependency;
- deterministic canonical serialization.

Gate 2: geometric execution

- feature-by-feature compile success;
- expected body/shell state;
- valid topology, finite geometry, and tolerance bounds;
- watertight/manifold state where required;
- no unintended self-intersections, zero-thickness regions, duplicate bodies, or sliver defects beyond policy;
- stable lineage and clean replay.

Gate 3: specification fidelity

- driving dimensions, tolerances, hole/thread definitions, mass, volume, center of mass, envelopes, clearances, and interfaces;
- comparison against source drawing views and annotations;
- exact predicates for requested counts and relationships;
- uncertainty-aware comparison where inputs are incomplete.

Gate 4: engineering intent

- sketch constraint state and downstream regeneration;
- controlled parameter perturbation and edit survival;
- unaffected-region invariants after edits;
- DFM/DFA rules for the selected process;
- assembly mate solvability, interference, motion envelope, and BOM reconciliation;
- drawing completeness, view consistency, dimension nonduplication, and PMI/GD&T integrity.

Gate 5: physical and delivery evidence

- applicable load cases, solver convergence, mesh independence/sensitivity, units, boundary conditions, and acceptance quantities;
- export round-trip and deviation;
- complete artifact manifest, revision metadata, and approval state;
- explicit limitations and human signoff requirements.

13.2 Visual verification set

Archie renders deterministic observations after meaningful graph milestones:

- six canonical orthographic/isometric views;
- user/source-matched camera views;
- silhouette and edge overlays;
- section cuts through critical internal features;
- exploded and motion-extreme views for assemblies;
- dimension/constraint overlays;
- change heatmaps and before/after overlays;
- manufacturing views such as flat patterns, tool direction, stock, supports, and inspection datums.

The vision model may diagnose discrepancies from these views. Deterministic geometry checks remain the acceptance authority.

13.3 Repair budget and convergence

A repair iteration contains one failure hypothesis, a minimal graph patch, and targeted revalidation followed by affected downstream gates. The budget is bounded by attempts, wall time, changed-node count, and regression radius. If convergence stalls, Archie stops with the best valid checkpoint and a precise blocker instead of degrading the design until a visual check passes.

13.4 Evidence bundle

Every accepted delivery includes:

- input and normalized-spec hashes;
- model/runtime/compiler/kernel package versions;
- graph root hash and count reconciliation;
- feature compile trace and failed/repaired node history;
- requirement verification matrix;
- canonical observations;
- manufacturing/assembly/physics results as applicable;
- export round-trip results;
- assumptions, waivers, warnings, and approval records;
- reproducible invocation manifest.

14. Full engineering scope and closure

Archie's ultimate scope is the physical-product lifecycle, but scope must be implemented as auditable modules with shared representations and maturity gates. Listing a field does not make the system competent in it. Every module progresses through `schema -> deterministic tools -> reference corpus -> validation -> bounded release`.

14.1 Domain map

Domain family	Included capability	Required connection to the graph
CAD and geometric modeling	parametric/direct solid, surface, freeform, sheet metal, assemblies, drawings, reverse engineering, lattice/additive	feature DAG, bodies, topology lineage, configurations
CAM and manufacturing	machining, turning, sheet metal, molding, casting, forging, welding, additive, composite, inspection planning	manufacturing feature/process graph, stock, setup, tool/access predicates
CAE	FEA, CFD, thermal, multibody, vibration, fatigue, fracture, electromagnetics, acoustics, optimization	analysis model, material law, boundary/load cases, mesh, solver evidence
Systems/mechatronics	requirements, architecture, interfaces, electrical/fluid routing, controls, robotics, digital twin	ESG functions/flows/interfaces and synchronized domain artifacts
Lifecycle operations	PLM, PDM, configuration/change management, BOM, ECO, release, supplier/Rfq, quality, field feedback	immutable revisions, effectivity, approvals, provenance, product structure
Production operations	MES, ERP, SCADA, industrial automation, virtual commissioning, IIoT	controlled integration adapters and state/event schemas; no unapproved plant action
Built environment	BIM, GIS, plant/facility layout, CIM	coordinate/reference systems, spatial hierarchy, interfaces, requirements
Quality and compliance	GD&T, tolerance stack, CMM, NDT, FMEA, RAMS, LCA, regulatory evidence	verification predicates, risk records, inspection and approval evidence
Industrial design and visualization	ergonomics, human factors, CMF, rendering, animation/motion communication	versioned appearance/scene data linked to engineering geometry

These domains are not "lite" in the final vision. They are also not released simultaneously under a false universal-ready label. Maturity is explicit per operation, material model, process, solver, format, and jurisdiction.

14.2 Manufacturing intelligence

Manufacturability is evaluated against a declared process, machine envelope, material state, quantity, cost/time objective, and quality requirement. Generic "manufacturable" is not a valid predicate.

Machining

- stock selection and datum/setup strategy;
- tool accessibility, reach, holder collision, minimum internal radius, and aspect ratio;
- feature recognition for pockets, slots, holes, threads, bosses, planar/contoured regions;
- tolerance and surface-finish capability by process;
- thin-wall, chatter, fixturing, deformation, and deburring risk;
- operation sequence and in-process inspection;
- conservative cost/time drivers, never invented supplier quotes.

Injection molding and polymer processes

- parting line and draw direction;
- draft by material/texture/process assumption;
- wall-thickness distribution, coring, ribs, bosses, gussets, and sink/warp risk;
- undercuts, lifters/slides, shutoffs, vents, gates, weld lines, and ejection concepts;
- moldability review tied to simulation/experienced-review status.

Sheet metal

- gauge/material table, bend radius, K-factor/bend allowance/deduction source;
- bend order, tooling access, minimum flange/hole-to-bend distance;
- relief, hems, seams, corners, formed features, grain direction;
- flat-pattern reconciliation and DXF export validation.

Additive manufacturing

- process/material/machine envelope;
- orientation, support, overhang, trapped-powder/resin, minimum wall/feature;
- lattice/TPMS printability and resolution;
- allowance, post-processing, inspection, and distortion strategy;
- watertight/manifold export and build-volume nesting evidence.

Casting, forging, welding, composites, and forming

Each receives process-specific feature schemas, allowance and tooling direction, joining/layup definitions, defect/risk rules, heat-treatment state, inspection, and simulation interfaces. Heuristics must name their source and applicability.

14.3 GD&T, tolerance, and inspection

Archie must understand ASME- and ISO-style semantics only through lawfully obtained references and edition-aware rule sets. It must not mix symbol conventions silently.

- select functional datum reference frames from interfaces and assembly function;
- distinguish size, location, orientation, profile, and runout controls;
- retain modifiers, material condition, projected tolerance zones, datum mobility, and pattern semantics;
- construct one-dimensional, vector-loop, or statistical tolerance models as appropriate;
- detect overdimensioning, contradictory controls, missing basic dimensions, and inspection ambiguity;

- generate inspection characteristics and CMM-friendly datum/setup proposals;
- link every annotation to controlled topology and its governing requirement;
- preserve semantic PMI through supported STEP AP242 workflows and verify round-trip.

A drawing is not "complete" because it looks dense. Completeness follows an auditable checklist for function, manufacturing, inspection, title/revision data, and the selected standard.

14.4 Assembly intelligence

Assembly reasoning includes:

- product structure, alternatives, configurations, effectivity, make/buy, and BOM reconciliation;
- explicit interfaces and connection types;
- mate/joint synthesis with remaining-degree-of-freedom analysis;
- fastener selection only from verified catalogs and applicable design rules;
- stack-up, preload, torque assumptions, locking, access, service, and tool clearance;
- static/dynamic interference and collision along motion paths;
- mechanism kinematics, limits, singularities, and sequence;
- cable, hose, pipe, harness, and human-access envelopes;
- DFA, assembly order, error proofing, and serviceability;
- exploded views, instructions, callouts, and revision-aware handoff.

14.5 Physics in the loop

Simulation is a typed, local, revision-coupled product path - never model-generated colorful contours. A simulation request becomes explicit graph content: the physical question, geometry scope, material state, idealizations, contacts, loads, boundary conditions, discretization, solver profile, observables, acceptance criteria, and requested fidelity.

14.5.1 Two-loop truth contract

Archie provides two connected loops and labels them so the user cannot mistake responsiveness for validated accuracy.

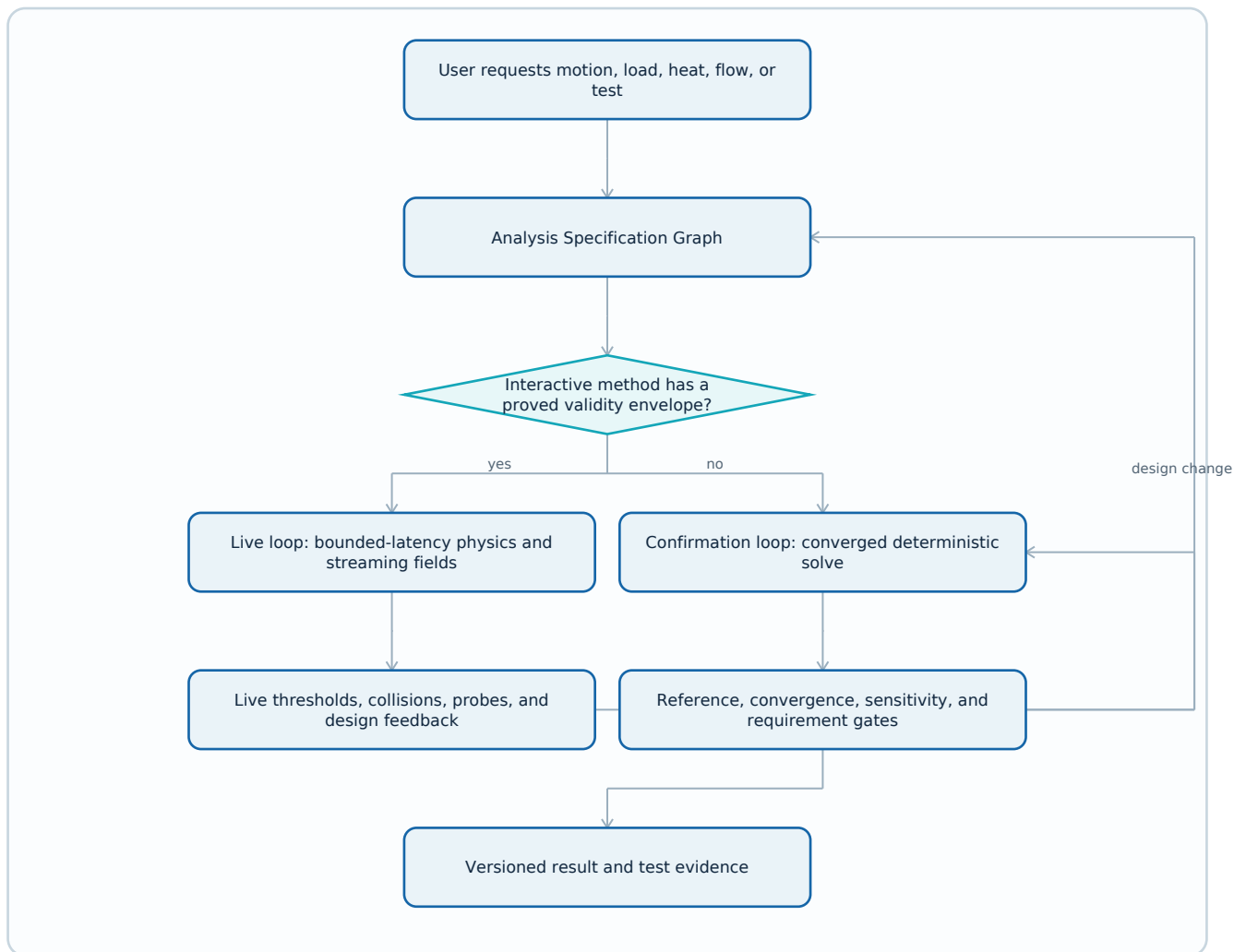


Figure 12. 14.5.1 Two-loop truth contract

Loop	Purpose	Required behavior	Forbidden claim
Live interactive physics	immediate mechanism, contact, deformation proxy, heat/flow proxy, collision, and sensor feedback during design	explicit target rate/latency; deterministic initial state; bounded step; streamed probes and fields; stale-result invalidation after geometry edits; displayed approximation/error envelope	may not be called converged, certified, or high-fidelity merely because it runs at 30-60 Hz
Engineering confirmation	produce the decision-grade result for the declared model and assumptions	mesh/discretization study; residual/conservation checks; time-step or load-step sensitivity; solver convergence; reference-problem correlation; repeatable result hash; complete evidence bundle	may not inherit validity from the live loop without comparison evidence

Not every high-fidelity structural, thermal, fluid, electromagnetic, or multiphysics solve can run in real time on a 36 GB workstation. The product promise is therefore exact: Archie starts a responsive live loop whenever an admitted method fits the problem, clearly marks its fidelity, and escalates to the confirmation loop automatically when the requested decision exceeds that envelope.

14.5.2 Native C++ simulation stack

Layer	Selected role	Minimum credibility gates
Numeric core	Eigen plus admitted sparse/direct/iterative backends	conditioning, residual, deterministic profile, units, reference systems
Multibody, mechanism, vehicle, contact, flexible bodies	Project Chrono embedded behind Forge schemas	constraint residual, timestep sensitivity, energy/momentum behavior, contact convergence, canonical initial state

Layer	Selected role	Minimum credibility gates
Structural, thermal, and PDE discretization	MFEM as the primary C++ finite-element foundation	element/material tests, patch tests, manufactured solutions, mesh refinement, benchmark correlation
Meshing	OCCT tessellation for viewport/quick paths; evaluated Gmsh C++ API or Forge mesher behind an isolated adapter for analysis meshes	geometry conformity, Jacobian/quality distributions, boundary tags, refinement and remesh stability
CFD and conjugate heat transfer	Forge-native reduced-order/live path; controlled local C++ confirmation adapter such as OpenFOAM where admitted	mass/momentum/energy conservation, bounded fields, mesh independence, turbulence/model applicability, reference correlation
Optimization and uncertainty	C++ design-of-experiments, adjoint/gradient where available, derivative-free search, Monte Carlo and interval/tolerance propagation	frozen objectives/constraints, reproducible seeds, convergence history, Pareto integrity, sensitivity
Field rendering and probes	Diligent/Metal viewport compute and rendering, compressed result tiles, stable entity/mesh IDs	unit/legend integrity, no data-altering interpolation, probe reproducibility, revision match

These components are compiled and installed in the local dependency plane. Solver processes remain persistent between runs when isolation is useful; they reuse symbolic factorizations, contact structures, meshes, basis operators, reduced models, and result buffers only when their full cache keys remain valid.

14.5.3 Analysis graph and invalidation

Simulation state is not a sidecar folder. It is a typed subgraph connected to feature and requirement IDs:

```

AnalysisCase
  geometry_revision and selected stable entities
  physics domains and coupling graph
  material/property records with condition and source
  idealizations and dimensional reductions
  contacts, joints, interfaces, loads, constraints, initial conditions
  mesh/discretization controls and region tags
  solver, integrator, tolerances, deterministic profile
  requested observables, probes, thresholds, events, and stop rules
  live-model identity and validity envelope
  confirmation-model identity and correlation evidence
  verification predicates and approval state

```

An edit triggers dependency-aware invalidation:

- a visual color change invalidates no physics;
- a fillet outside the selected load path may invalidate tessellation but not a validated reduced model only when a geometric-influence test proves that exemption;
- a material, thickness, contact, boundary, load, or topology change invalidates every dependent live and confirmation result;
- stale results remain viewable only with a persistent watermark and may not satisfy a current requirement gate;
- mesh morphing and reduced-basis reuse are permitted only with explicit quality and applicability checks.

14.5.4 Real-time testing and closed-loop events

On request, Archie can run virtual tests while the user edits or drives the model: range-of-motion, collision and clearance, drop/impact proxy, load sweep, thermal ramp, flow operating point, vibration response, tolerance Monte Carlo, actuator/sensor loop, fatigue-cycle estimate, and threshold/event detection. Each virtual test declares input trajectories, sample rate, deterministic initial state, measured signals, pass/fail predicates, abort conditions, and a replay artifact.

The live loop owns a frame budget. Physics stepping, meshing, result transfer, rendering, and UI work have separate time and unified-memory budgets. If the loop cannot maintain its stated target, it reports **LIVE_DEGRADED**, reduces visualization fidelity before physics correctness, or transitions to a paced simulation; it does not silently enlarge the timestep or skip required events.

14.5.5 Evidence and bounded conclusions

Every simulation record includes geometry revision, idealizations, material model/source, contacts, loads, constraints, mesh and quality, solver/build/settings, convergence history, result units, sensitivity, validation reference, limitations, live-versus-confirmation correlation, and approval state. Surrogates and reduced models accelerate exploration only inside an accepted error envelope. A decision outside that envelope requires deterministic confirmation or an explicit blocked result.

14.6 Optimization

Parametric, topology, shape, multidisciplinary, and robust optimization operate over explicit objectives, constraints, uncertainty distributions, and evaluation budgets. Archie must show Pareto tradeoffs when objectives conflict. It cannot bury infeasible candidates, change constraints mid-run, or report a local optimum as universally best.

14.7 PLM/PDM and engineering work products

Archie maintains:

- part/document numbers and immutable revisions;
- product, manufacturing, and service BOM views;
- requirements and verification baselines;
- branches, merges, release states, effectivity, and configuration rules;
- change request/order/notice packages with affected-item analysis;
- supplier/RFQ packages prepared for review;
- drawings, models, meshes, reports, renders, instructions, and inspection artifacts;
- signatures/approvals and a tamper-evident audit trail.

Archie does not call external CAD, PLM, ERP, email, supplier, ordering, or release services. It prepares and proves versioned local exchange packages for an authorized human or separately governed system to transfer. Local host-application adapters may operate through explicit same-machine process or file boundaries, but they cannot create a second network client.

14.8 Physical feedback loop

Field and manufacturing feedback may include inspection results, nonconformance, scrap/rework, test measurements, sensor histories, service reports, warranty returns, supplier deviations, and failure analysis. It is linked to the exact as-designed/as-built/as-tested configuration.

Learning is never automatic from a single anecdote. The loop separates measurement error, process drift, design defect, usage outside specification, and unknown cause; proposes an engineering change; validates the change; and requires the appropriate approval.

14.9 High-end visualization, DCC, and digital-scene scope

The supplied image/video references establish a second quality bar: very dense modeled vehicles, environments, architecture, procedural scenes, PBR rendering, animation, rigs, effects, and cinematic presentation. This is adjacent to-but not the same as-parametric mechanical CAD.

- Engineering truth remains the exact feature DAG/B-rep/assembly graph.
- Presentation truth is a derived, versioned scene graph with cameras, lights, materials, textures, animation, effects, and render settings.
- A visually dense mesh cannot replace editable manufacturing geometry.
- CAD-to-DCC conversion records tessellation/deviation, UV/material assignment, instances, levels of detail, and revision links.
- Simulation visualization reads immutable result fields and unit metadata; it cannot repaint values to make a preferred story.

- Character rigs, vegetation, destruction/effects, and large environment generation are optional scene modules, not features silently embedded in a mechanical part history.

Candidate C++-compatible ecosystem components include [OpenUSD](#) for scene composition, [MaterialX](#) for materials, [OpenColorIO](#) for color management, [OpenImageIO](#) and [OpenEXR](#) for imagery, [OpenVDB](#) for sparse volumes, and [Embree](#) for CPU ray-query/rendering research. Each remains subject to platform, performance, source-use, and dependency review; the production macOS viewport stays Metal-native.

The supplied social-media screenshots and clips are design references, not automatically admitted training data. Use them to derive stress-test categories and UX expectations, not to copy protected assets or creator work into a dataset without permission.

15. Training architecture and curriculum

Training creates a model that understands engineering intent and uses deterministic tools. It does not try to memorize the whole kernel, all standards, or every current catalog value in weights.

15.1 Competence decomposition

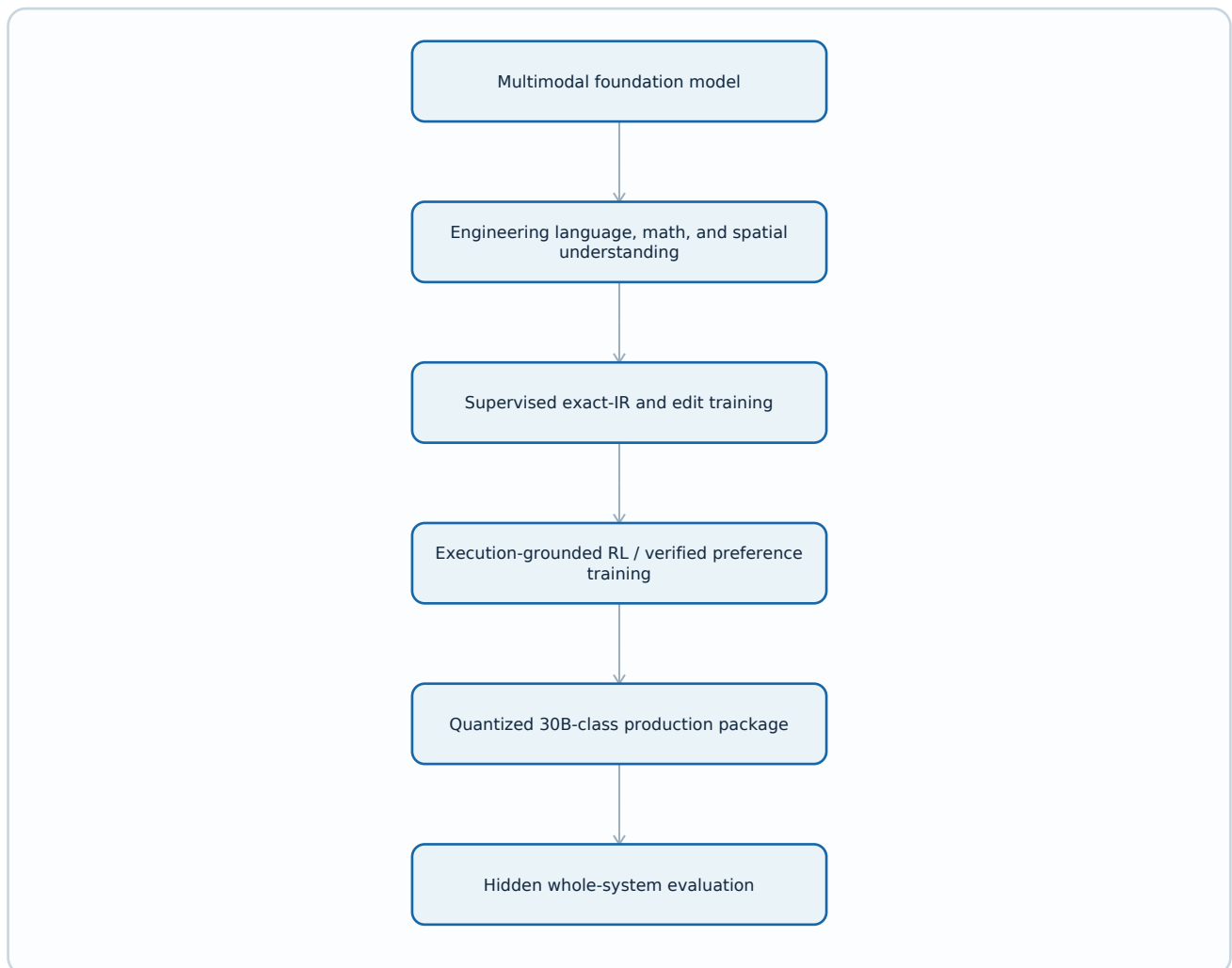


Figure 13. 15.1 Competence decomposition

Weights should internalize:

- engineering language and conceptual relationships;
- mathematics, logic, units, estimation, uncertainty, and physical intuition;
- visual/drawing/spatial understanding;

- decomposition into requirements and design intent;
- construction strategy, edit strategy, and tool selection;
- feature IR grammar and typical failure recovery;
- calibrated uncertainty and question asking.

Retrieval and deterministic systems should provide:

- current standards, law, catalogs, prices, availability, and material records;
- exact numerical solve and geometry execution;
- private project context;
- large feature-graph state;
- current tool and kernel capability;
- citations and provenance.

15.2 Training stages

Stage A - foundation selection and baseline

Freeze a reproducible baseline of the candidate model, tokenizer, vision encoder, prompts, quantization, and inference runtime. Measure all target benchmarks plus hidden tasks before adaptation.

Stage B - continued pretraining, only if evidence justifies it

Use source-use-approved technical text, equations, drawings, code, and structured engineering data. Maintain general reasoning replay to prevent catastrophic forgetting. Continued pretraining is accepted only if it improves downstream whole-system tasks enough to justify cost and does not degrade calibration, instruction following, or visual performance.

Stage C - supervised fine-tuning

Train on high-quality, versioned examples of:

- evidence-to-ESG extraction and contradiction detection;
- ambiguity classification and concise question asking;
- ESG-to-feature-DAG planning and exact chunk emission;
- sketch constraints, parametric construction, assemblies, and drawings;
- feature-level diagnostics and minimal edits;
- tool calls, observation interpretation, and cited retrieval;
- manufacturing/inspection reasoning and simulation setup;
- refusal/deferral when evidence or authority is missing.

Targets use canonical graph serialization. Human plans and recovered logs are labeled separately so the model never learns that a summary is executable.

Stage D - execution-grounded preference learning and RLVR

Rewards come from deterministic or calibrated evaluators:

- parse/type/compile/replay success;
- requirement and count coverage;
- geometric validity and similarity where appropriate;
- dimension/interface/topology predicates;
- edit locality and invariant preservation;
- sketch constraint quality and design intent;

- manufacturing, assembly, drawing, and solver gates;
- correct uncertainty/clarification behavior;
- graph efficiency without semantic compression;
- resource and latency cost after correctness.

Compiler failure diagnostics are structured feedback, not a reason to reward superficial shape matching. Reward hacking tests include no-op padding, hidden bodies, invalid suppression, tolerance inflation, screenshot gaming, and benchmark-specific branching.

Stage E - quantization-aware validation

Evaluate Q4, Q5, Q6, and any mixed quantization candidates on the same hidden suite. The smallest package that passes accuracy, calibration, long-graph, and latency gates wins. Do not assume a quantization is acceptable from perplexity alone.

Stage F - champion/challenger promotion

The incumbent remains production until the challenger passes statistical and engineering significance thresholds, regression limits, safety tests, memory/latency budget, and reproducibility. Promotion produces a signed model card and rollback package.

15.3 Dataset record and sampling

Every training sample carries:

- source dataset and original artifact hash;
- source-use and permitted-purpose decision;
- generator/human/tool provenance;
- part family, operation families, domain, difficulty, and representation;
- quality and verification results;
- contamination split/group identifiers;
- private/public status and deletion lineage;
- transformations, augmentations, and synthetic ancestry;
- sample weight and curriculum stage.

Split by design family, source project, geometry lineage, and near-duplicate cluster-not random rows. Near duplicates across train/test are contamination even when filenames differ.

15.4 Synthetic data factory

The most scalable safe source is Forge-generated, execution-verified data built from source-use-approved primitives, procedural families, and authored templates.

1. sample an ESG with dimensions, constraints, materials, process, and interfaces;
2. generate the exact feature DAG from trusted templates or verified generators;
3. compile and reject invalid geometry;
4. render multi-view drawings/images and export neutral CAD;
5. create diverse but faithful natural-language requests and edit sequences;
6. inject controlled ambiguity, contradictions, feature failures, and repair tasks;
7. verify target counts, predicates, and edit invariants;
8. deduplicate and assign family-level splits;
9. store full provenance and generator version.

Synthetic diversity spans scale, units, coordinate frames, feature order, design families, rare edge cases, and manufacturing processes. It must not simply teach one generator's style; mix templates, algorithmic generators, human-authored graphs, imported/open models with reconstructed intent, and adversarial mutations.

15.5 Long feature-DAG curriculum

Training length grows by semantic complexity rather than token padding:

Band	Typical scope	Required behavior
S	single sketch/simple part	exact constraints, short replay
M	multi-feature manufactured part	stable references, DFM, edit sequence
L	complex part or small assembly	chunking, configurations, drawings, invariants
XL	subsystem/large assembly	hierarchy, occurrence patterns, partial rebuild, resume
XXL	project beyond one context window	persistent symbol table, dependency-slice retrieval, multi-session completion

Examples must include jobs whose exact DAG is 10× and 100× longer than the short supplied summaries. Training explicitly penalizes premature footer emission, omitted occurrences, summary substitution, count drift, and success after truncation.

15.6 Reasoning supervision policy

Do not depend on collecting private hidden chain-of-thought. Train on concise engineering rationales, assumptions, evidence links, intermediate formal artifacts, tool observations, failure labels, and verifiable subgoals. This yields inspectability without requiring unverifiable narrative reasoning traces.

15.7 Adapter strategy

- one stable base model package;
- a small number of composable, versioned domain adapters only when evaluation proves benefit;
- no adapter stack with unknown interaction effects;
- routing policy included in the signed package;
- adapter merge/quantization tested as a unique model;
- full rollback to base and previous champion;
- on-device inference only;
- any separately invoked training or adapter preparation is a development process, never a runtime service or fallback, and installed Archie remains complete when every network interface except the SearXNG browser path is denied.

16. Data and source registry

No finite document can truthfully list "all possible data." This is the curated initial registry of high-value sources; the live registry must remain machine-readable, versioned, and continuously reviewed. A dataset is not admitted merely because it appears below.

16.1 Admission states

State	Meaning
TRAIN-APPROVED	Source-use, provenance, privacy, and contamination review permit the stated training use.
CONDITIONAL	Technically relevant, but applicable terms, provenance, quality, or scope must be resolved before training.
EVAL-ONLY	Held out for evaluation; no training, retrieval, answer-key use, or tuning on the evaluated split.
REFERENCE-ONLY	May be retrieved/read under its terms; not copied into weights by default.

State	Meaning
PROHIBITED	No use for the proposed purpose.

Source-use review is performed on the actual data release, not inferred from a code repository's general terms.

16.2 Parametric CAD, sketches, parts, and assemblies

Source	Value	Initial state and caution
DeepCAD	about 178k CAD construction sequences/models	CONDITIONAL : repository terms do not automatically settle source-model rights. Audit Onshape/ABC lineage and dataset terms.
Fusion 360 Gallery Reconstruction	thousands of parametric reconstruction sequences	CONDITIONAL : strong history data; review the dataset terms and permitted product use.
Fusion 360 Gallery segmentation/assembly/joint subsets	parts, assemblies, joints, segmentation	CONDITIONAL : preserve subset-specific terms and family splits.
SketchGraphs	approximately 15 million real-world CAD sketches and constraint graphs	CONDITIONAL : excellent constraint curriculum; verify release terms and source restrictions.
ABC Dataset	about one million CAD models with B-rep curves/surfaces	CONDITIONAL : verify dataset terms; do not infer permission from related code.
JoinABLE	assembly joint prediction and geometry	CONDITIONAL : useful for connection/interface learning; audit data terms.
AutoMate	large assembly/mate corpus	CONDITIONAL : strong mate learning; verify Dryad and original Onshape terms.
MFCAD	machining-feature CAD models	CONDITIONAL : verify data and repository terms per artifact.
MFCAD++	roughly 59k labeled STEP models	CONDITIONAL : confirm repository-record terms and commercial permissions.
MFinstSeg / AAGNet	tens of thousands of B-rep models with machining feature instances	CONDITIONAL : excellent recognition/evaluation source; audit dataset ancestry.
FeatureNet machining-feature data	procedural labeled machining features	CONDITIONAL : repository and artifact terms require source-use and architectural review.
NIST CAD/PMI validation models	STEP/PMI test cases, drawings, validation reports	candidate TRAIN-APPROVED /validation after record-specific review; especially valuable for interoperability.
BRepNet	B-rep segmentation framework and datasets	CONDITIONAL : useful representation baseline; review each dataset.
UV-Net	B-rep learning plus SolidLetters/MFCAD/Fusion subsets	CONDITIONAL : audit each subset separately.
PartNet-Mobility and GPartNet	articulated objects and actionable parts	CONDITIONAL : useful for part/function/motion; commonly research-oriented terms.
Assembly101	procedural assembly videos/actions	CONDITIONAL : weak supervision for sequence and tool use; verify source-use and privacy.
User-authored Forge synthetic corpus	exact ESG, DAG, CAD, drawings, edits, failures	candidate TRAIN-APPROVED when generator inputs and templates are cleared. This should become the core controllable corpus.

16.3 Text/drawing-to-CAD and CAD editing sources

Source	Value	Initial state and caution
Text2CAD	approximately 170k models and hundreds of thousands of annotations	CONDITIONAL /research-only by default: published terms do not make it a commercial training default.
Drawing2CAD	vector engineering drawing to parametric CAD	hold official evaluation split EVAL-ONLY ; audit training subset and generated drawing rights.
ParaCAD / PHT-CAD	very large drawing annotations and real-world test drawings	hold test split EVAL-ONLY ; audit training-release terms and document provenance.

Source	Value	Initial state and caution
HistCAD	parametric histories with explicit constraints/editability	hold benchmark split EVAL - ONLY ; review permitted use of any training split.
neuralCAD-Edit	expert-authored multimodal CAD edits	entire scored set EVAL - ONLY ; use only official permitted training resources outside it.
Source-use-approved CadQuery/OpenSCAD/FeatureScript-like examples	code-to-geometry pairs	CONDITIONAL : ingest only repository files whose applicable terms and provenance are recorded; generated screenshots and CAD must be verified.
Patent drawings and public-domain technical archives	diverse mechanisms/drawings	CONDITIONAL : public-domain status, jurisdiction, OCR quality, and modern relevance must be proven per source.

16.4 Supplemental 3D vision, articulation, and material-learning candidates

These sources can improve visual grounding, part hierarchy, material appearance, and reconstruction. Mesh/scene data does not teach parametric feature history by itself and must not dominate the CAD curriculum.

Source	Value	Initial state and caution
ShapeNet	large categorized 3D shape collection	CONDITIONAL : research terms and original-model rights restrict use; not a commercial default.
PartNet	fine-grained hierarchical parts and instances	CONDITIONAL : registered, noncommercial research terms; useful for taxonomy research, not product training without new rights.
Objaverse / Objaverse-XL	hundreds of thousands to millions of diverse annotated 3D objects	CONDITIONAL : collection terms and each object's source-use status both matter; filter object-by-object and deduplicate aggressively.
Amazon Berkeley Objects	product images, turntables, metadata, and thousands of textured glTF models	CONDITIONAL : valuable image/material/multi-view grounding; confirm release terms and keep catalog provenance.
Scan2CAD	alignment of CAD models to real scans	CONDITIONAL : useful for reverse engineering/registration; audit ScanNet/ShapeNet source terms.
OmniObject3D	real scanned objects and multi-view imagery	CONDITIONAL : visual reconstruction only after source-use review.
Matbench and matminer datasets	curated material-property prediction tasks and datasets	preserve official benchmark splits; audit each constituent dataset and distinguish scientific properties from certified engineering allowables.

16.5 Engineering physics and surrogate-learning candidates

Source	Focus	Initial state and caution
PDEBench	diverse PDE simulation data and benchmarks	CONDITIONAL ; review per-dataset terms and preserve benchmark splits.
AirFRANS	RANS airfoil simulations	CONDITIONAL : library and data use different terms; review both records.
DrivAerNet++	thousands of automotive geometries and large CFD corpus	CONDITIONAL : very large storage footprint and source-use review; ingest only selected, locally stored shards that fit the workstation budget.
SimJEB	jet-engine bracket CAD, meshes, and FEA cases	CONDITIONAL : high-value geometry-to-analysis link; verify terms.
The Well	multi-terabyte, multi-physics datasets	CONDITIONAL : audit each constituent dataset; use only curated local subsets or derived fixtures that fit explicit storage and compute budgets.
RealPDEBench	real-world PDE problems	CONDITIONAL ; hold official benchmark cases.
MultiPhysicsBench	coupled multiphysics evaluation	EVAL - ONLY for official tasks unless a separate training split is explicitly admitted.

These datasets train intuition, surrogate proposals, and tool selection. They never replace deterministic solver validation.

16.6 Materials, manufacturing, standards, and public technical knowledge

Source	Permitted role
Materials Project API	cited materials research data under published terms; verify applicability to engineering grades and conditions
NIST Materials Data Repository	authoritative research datasets; use record-specific terms and units
NIST JARVIS	materials/design data and model resources; verify individual artifacts
NIST MBE/PMI validation	interoperability and PMI validation
NASA Technical Standards System	public authorized standards and handbooks as reference; edition/applicability tracking required
Government technical reports and handbooks	REFERENCE-ONLY or admitted record-by-record; public access is not always public domain
Manufacturer datasheets and supplier catalogs	retrieval/reference with current citations and terms; do not memorize availability/prices
ISO, ASME, DIN, SAE, ASTM, IEC and similar standards	REFERENCE-ONLY unless training rights are explicitly obtained; do not scrape or redistribute
MatWeb and other proprietary aggregators	no scraping; use only through lawful subscription/API and permitted use

16.7 Mathematics, code, and technical language

Use source-use-approved math/science educational resources, official C++ and library documentation, admitted engineering repositories, and verified synthetic problems. Repository-level code ingest requires file-level terms/provenance filtering and secret scanning. Source with reciprocal distribution obligations may be used only under an approved policy compatible with the intended model and distribution.

Books, papers, standards, forums, videos, and course content are not assumed open. ArXiv availability is not blanket training permission. Community sources-including mechanical-engineering forums and subreddits-may be used for trend discovery and problem taxonomy through permitted APIs/terms; mass scraping into training data is prohibited without a documented lawful basis and privacy review.

16.8 Data quality gates

Before training, each sample must pass relevant gates:

- source-use/provenance/consent;
- malware, secret, PII, and customer-IP scan;
- parse and unit sanity;
- geometry execution and validity;
- duplicate and lineage clustering;
- drawing/CAD correspondence;
- annotation consistency and language quality;
- feature-count and requirement traceability;
- split/benchmark contamination;
- versioned quarantine and appeal/removal process.

16.9 High-yield source program for industrial geometry and physics

No source, model, repository, vendor page, or dataset "boosts Archie astronomically" merely by being named or downloaded. A source has value only after it produces eligible local artifacts, executable tasks, verified ground truth, and a statistically credible improvement on hidden system tests without regressions. This section routes the requested AMD, NVIDIA, FreeCAD, and CATIA material into that proof process while preserving the M4-only execution boundary.

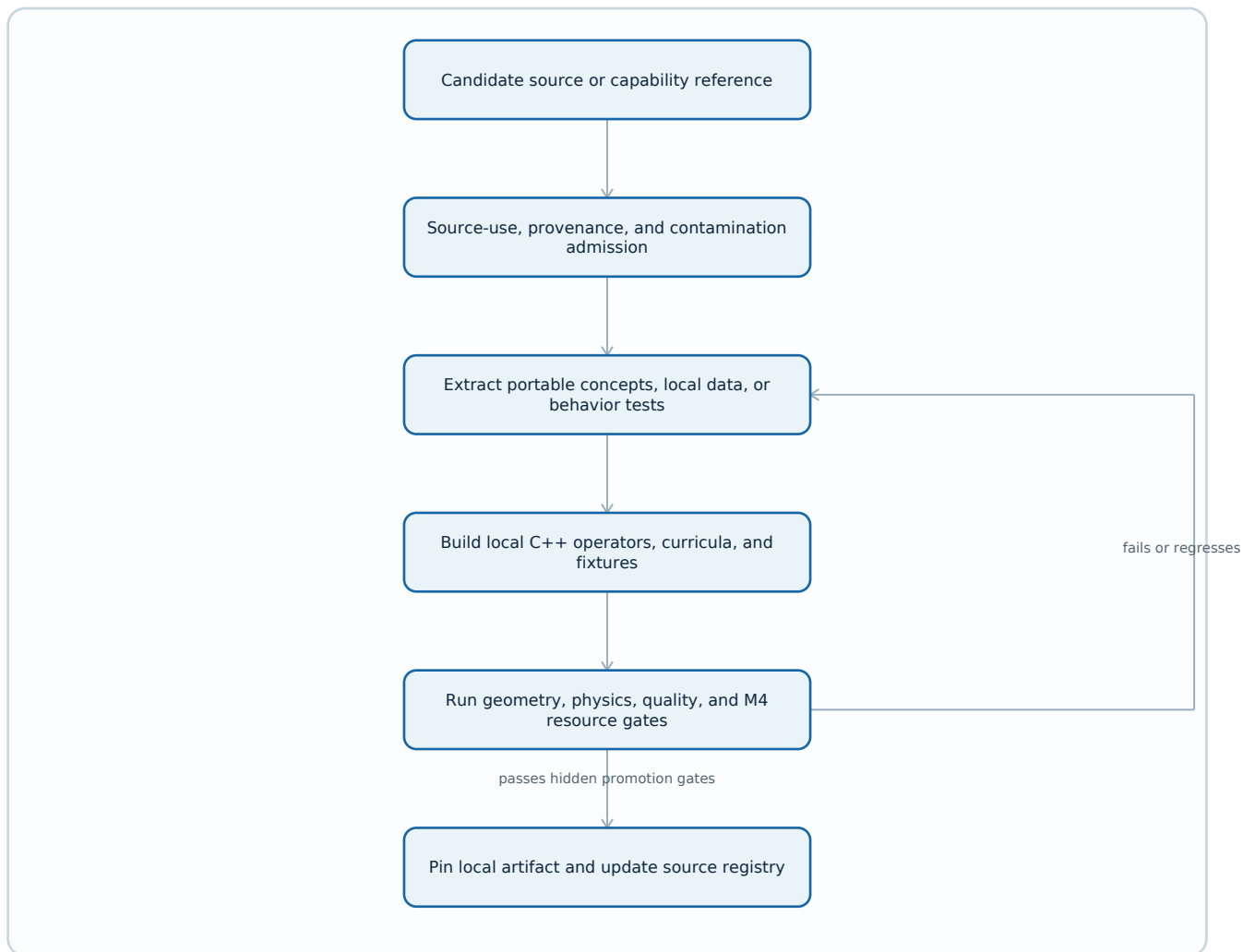


Figure 14. 16.9 High-yield source program for industrial geometry and physics

Every admitted record declares `source_id`, immutable origin, retrieval date, content hash, permitted role, extracted files, transformation lineage, benchmark-family cluster, target capability, local storage class, retention state, test owner, and removal lineage. A public project page is never silently converted into training data. Web access discovers and cites sources; it does not provide production compute.

16.9.1 Requested-source routing

Source	High-value material to investigate	M4-local route	Boundary that must not be crossed
AMD Research and Advanced Development open-source projects	Chakra workload traces/schema concepts, Astra-Sim scheduling/system ideas, Brevitas/QONNX quantization concepts, and Omnistat/Omnitrace measurement taxonomy	REFERENCE-ONLY first; extract hardware-neutral scheduling, quantization, trace, and profiling test ideas into documented local C++ experiments	ROCm, AMD GPU services, vendor-hosted execution, and non-Apple binaries cannot enter the M4 production path; an AMD project name is not evidence of benefit
NVIDIA open-source projects	OpenUSD large-scene design, Kaolin 3D representation/evaluation ideas, PhysicsNeMo physics-ML task design, Isaac Sim synthetic-scene taxonomies, and public PhysX test concepts	admit portable specifications, mathematical references, neutral datasets, and locally re-executable tests; OpenUSD C++ may be built locally after its own gates	CUDA, Omniverse, NVIDIA services, hosted inference, remote simulation, and vendor-only binaries are excluded; surrogate predictions never become confirmation truth
FreeCAD, features, source repository, and source architecture	OCCT-backed document/feature architecture, App/Gui separation, parametric recompute behavior, import/export cases, workbench taxonomy, regression tests, and eligible example models	mine only admitted files and behavior specifications; re-execute accepted fixtures through Forge; use C++ core patterns and neutral CAD cases as oracles/challengers	FreeCAD is not a drop-in pure-C++ Forge runtime: its high-level product contains extensive Python behavior; no Python module or interpreter may become required by Archie

Source	High-value material to investigate	M4-local route	Boundary that must not be crossed
CATIA, CATIA V5, mechanical engineering and Class-A surfacing, and current surface-refinement capabilities	vocabulary and public capability taxonomy for assemblies, free-form surfaces, surface refinement, engineering workflows, design/simulation integration, and neutral-format expectations	REFERENCE-ONLY; convert public capability statements into independent Forge requirements and authorized black-box interoperability/evaluation cases	CATIA is proprietary: do not ingest hidden behavior, code, protected manuals, customer files, or generated answers as training data; Forge must not require CATIA to run or claim parity from screenshots

The AMD and NVIDIA catalogs are idea and evidence sources, not runtime stack recommendations. Every candidate that assumes a discrete AMD/NVIDIA GPU is rejected for production unless a separate portable C++/Metal implementation passes the same local acceptance tests on the M4. FreeCAD provides unusually rich observable CAD behavior, but its mixed C++/Python architecture is filtered rather than copied wholesale. CATIA defines valuable industrial expectations, but only authorized interoperability and independent capability evaluation are admissible.

16.9.2 Massive-assembly competence track

The assembly path combines exact product structure with a streamed visual/geometry working set. "One million occurrences" does not mean keeping one million unique detailed B-reps resident. Definitions are content-addressed once; occurrences store stable identity, transform, configuration, suppression, metadata, and links; payloads and representations load only when required.

Primary source	Competence extracted	Required local implementation or test
OpenUSD composition, performance guidance, and glossary	references, payloads, instancing, variants, purposes, composition arcs, working sets, and large-scene diagnostics	C++ scene/assembly bridge with lazy payload activation, prototype instancing, configuration mapping, deterministic flatten/export checks, and no Python runtime
OCCT XDE and STEP translation	assembly definitions/instances, names, colors, layers, validation properties, external references, and STEP assembly exchange	stable definition-occurrence model, metadata preservation, AP242-oriented test fixtures, incremental import, and round-trip difference reports
FreeCAD C++ core and admitted regression/example files	real parametric document/recompute and import/export failure modes	differential oracle runs that compare structure, validity, metadata, and edit survival rather than pixels alone
NIST CAD/PMI validation models, verification results, and STEP examples	neutral STEP, assembly, semantic/graphic PMI, and validation-property cases with documented error categories	held-out interoperability suite; semantic PMI and occurrence errors remain individually visible instead of collapsing into one score
Forge deterministic assembly factory	exact scale and edge cases unavailable in public corpora	seeded families with repeated/unique parts, nested subassemblies, flexible configurations, suppression, external references, mates, PMI, lightweight representations, corruptions, and known answers

The scale ladder is mandatory and cumulative:

Band	Minimum structure	Representation discipline	Proof obligations
ASM-1K	at least 1,000 occurrences with nested definitions	detailed B-rep permitted for the active working set	exact occurrence/BOM/mate counts; load-save-replay; deterministic selection; edit and undo
ASM-10K	at least 10,000 occurrences with repeated prototypes and configurations	instancing, culling, background tessellation, and bounded detail activation	cold/open and warm/reopen timing; p50/p95 frame time; selection latency; memory; partial rebuild; interference subset
ASM-100K	at least 100,000 occurrences and deep hierarchy	payload/lazy geometry, lightweight envelopes, spatial index, out-of-core artifact store	time to first useful view; bounded resident working set; exact query/BOM; cancellation/restart; corrupted-payload isolation
ASM-1M	at least 1,000,000 addressable occurrences, mostly instanced	prototypes plus compact occurrence records; detail loaded by task/visibility/proximity	no flattening or count drift; addressable stable IDs; deterministic transforms/configurations; responsive hierarchy/query under the declared target envelope; no memory-pressure crash

Every band records total and unique definitions, occurrences, mates, active detailed bodies, triangles, payload bytes, resident memory, swap activity, time to first useful view, p50/p95/p99 frame and selection latency, open/save/replay time, incremental rebuild closure, and hash stability. The resource governor may reduce visual detail or unload inactive geometry; it may never change engineering counts, transforms, mates, metadata, or selected configuration.

Large-assembly curriculum tasks include:

- identify and repair one bad occurrence transform without touching its siblings;
- replace a part definition while preserving eligible occurrence identities and mates;
- switch configurations and reconcile suppressed occurrences with the BOM;
- resolve external references locally and report missing payloads without flattening;
- run clearance/interference only over spatially relevant candidates, then prove completeness of the candidate filter;
- apply an upstream dimension edit and rebuild only affected definition and occurrence closures;
- export/import a neutral assembly and report every structural, metadata, PMI, unit, and geometry difference;
- resume a million-occurrence graph after interruption with identical count and root fingerprints.

16.9.3 Precise-simulation competence track

Simulation training is execution-grounded. Archie learns to formulate and diagnose analyses, but deterministic local solvers and verification cases decide engineering acceptance. Learned surrogates may propose meshes, regimes, preconditioners, reduced models, or live approximations; they cannot certify a result or replace a confirmation solve.

Source	Role in the local stack or curriculum	Required evidence before promotion
Project Chrono	local C++ multibody, mechanism, contact, vehicle, and flexible-body foundation	analytic mechanisms, energy/momentum checks, timestep studies, contact sensitivity, deterministic replay, and M4 timing
MFEM	primary local C++ finite-element foundation for structural, thermal, and general PDE confirmation	element patch tests, manufactured solutions, h/p refinement, residual norms, energy balance, conditioning, and reference comparisons
PETSc user guide	evaluated local solver/preconditioner and unstructured-mesh challenger for problems that exceed the initial sparse path	same-Mac build, deterministic configuration, solver equivalence, convergence, peak memory, and net benefit over the simpler accepted backend
Gmsh	local meshing adapter and mesh-quality/reference suite	geometry-revision binding, sizing-field determinism, Jacobian/aspect/skewness gates, refinement study, and entity-map preservation
OpenFOAM Foundation	controlled local C++ CFD confirmation adapter for admitted cases	network-denied execution, versioned case package, mass/momentum/energy conservation, mesh/time-step independence, and exact result provenance
NASA Turbulence Modeling Resource	authoritative turbulence-model definitions plus verification, validation, experimental, DNS, and LES reference cases	held-out CFD verification/validation cases with configuration fingerprints, discretization error, uncertainty, and no tuning on answer cases
NIST STEP/AP209 examples	neutral finite-element model/results interchange and product-definition context	local import/export semantic checks, units/frames/materials/mesh/result preservation, and explicit unsupported-field report
G+Smo	C++ isogeometric geometry/simulation challenger using a unified spline basis	locally built prototype, spline/geometry exactness, convergence against MFEM/reference solutions, conditioning, memory, and demonstrated value before adoption
NVIDIA PhysicsNeMo, Kaolin, Isaac Sim, and public PhysX material	task taxonomies, geometry/physics representations, synthetic-case ideas, and surrogate challenger definitions	all admitted artifacts converted to local immutable fixtures; no CUDA/vendor service; confirmation remains a local deterministic solver
AMD quantization, workload, and profiling projects	experiment design for model compression, scheduling, traces, and performance attribution	locally reproduced Metal/CPU measurements; no ROCm/runtime dependency and no result accepted by analogy

Each physics family has a verification ladder:

1. **Dimensional and setup validity:** canonical units, frames, sign conventions, materials, section properties, loads, contacts, constraints, initial state, and solver regime are explicit.
2. **Analytic or manufactured truth:** constant-strain patches, beams/plates with known limits, heat kernels, modal systems, conservation cases, and manufactured PDE fields expose implementation errors.
3. **Discretization evidence:** mesh, basis order, timestep, contact tolerance, and nonlinear tolerance are refined; observed order and quantities of interest are reported.
4. **Numerical evidence:** residuals, conditioning warnings, iterations, conservation, energy behavior, and solver termination reason are stored, not summarized away.
5. **Independent reference:** compare with a second formulation, admitted implementation, experiment, or published verification case when consequence requires it.
6. **Sensitivity and uncertainty:** vary material, boundary, contact, load, and numerical assumptions over justified ranges; distinguish input uncertainty from discretization error.
7. **Live/confirmation correlation:** the interactive approximation declares its domain and error envelope, and comparison is repeated after design edits that change that domain.
8. **M4 resource proof:** record wall time, peak/resident memory, thread count, Metal/CPU utilization, thermal behavior, cancellation latency, checkpoint size, and storage growth.

The confirmation path defaults to double-precision quantities where the solver supports them. Reduced precision, GPU approximation, relaxed contact, coarser mesh, skipped events, or larger timesteps are never silent performance modes. They are distinct analysis methods with explicit validity envelopes and cannot inherit acceptance from a higher-fidelity method.

16.9.4 Complex Class-A surface competence track

Class-A capability is a numeric geometry discipline, not a glossy render. A surface is accepted only when parameterization, topology, continuity, fairness, trim/offset behavior, downstream manufacturability, edit stability, and highlight behavior all pass the declared purpose-specific tolerances.

Source	Competence extracted	Forge use
OCCT modeling algorithms	B-spline/Bezier curves and surfaces, interpolation/approximation, ruled/pipe/filling/plate construction, extension, deformation, and fair/minimal-variation curves	primary exact B-rep operator and adversarial geometry-test foundation; every wrapper gets validity, tolerance, continuity, and topology tests
G+Smo	spline bases, multi-patch geometry, hierarchical splines, and isogeometric analysis	challenger for local refinement, multi-patch continuity, fairing, and geometry-to-analysis preservation
Pixar OpenSubdiv	high-performance subdivision evaluation for interactive smooth-shape control	locally built C++ control-cage/display and concept-form challenger; never substitutes subdivision display geometry for exact manufacturing B-rep
FreeCAD Part/Surface behavior and admitted tests	observable OCCT construction, recompute, import/export, and failure cases	differential behavior cases and corpus seeds after deduplication/source-use admission; exclude Python runtime paths
CATIA public Class-A and surface-refinement capability pages	industrial vocabulary for free-form surfaces, refinement, joins, fillets, analysis, and downstream engineering	independent requirement taxonomy and authorized black-box/neutral-file evaluation only; no hidden implementation inference
Forge analytic and synthetic surface factory	exact curves, derivatives, seams, singularities, pathological trims, slivers, offsets, blends, and patch networks with known answers	unlimited local curriculum/evaluation families with held-out parameter seeds, construction families, and corruption operators

The authoritative surface-quality record includes:

- part scale, units, intended process, visible/hidden regions, evaluation tolerance, and coordinate frame;
- degree, knot vectors, weights, parameter domain, control net, trim loops, periodicity, orientation, and singularities;
- **G0** positional gap and maximum/percentile deviation along every declared join;
- **G1** tangent-direction/normal-angle discontinuity with orientation handling;
- **G2** normal-curvature/curvature-vector discontinuity and radius behavior;

- [G3](#) curvature-derivative continuity where explicitly required;
- curvature combs, Gaussian/mean/principal curvature, inflection and oscillation diagnostics;
- fairness energies and localized variation maps, with the functional and sampling density stated;
- zebra, reflection-line, environment-highlight, silhouette, and draft observations tied to the exact surface revision;
- trim, sew, offset, thicken, shell, split, join, fillet, export, and downstream operation survival;
- self-intersection, inverted normal, non-manifold, sliver, tiny-edge, tolerance-stack, and invalid-region findings;
- upstream parameter edits and topology-lineage behavior.

No universal numeric threshold defines every Class-A part. Forge therefore stores a scale- and process-aware quality profile with hard limits, sampling rules, and named evaluation regions. Visual zebra/reflection checks are mandatory observations for visible surfaces but cannot override failed derivative or deviation gates.

The surface curriculum must cover, at minimum:

- constrained curve networks and multi-section lofts with deliberate poor parameterization;
- bridge, fill, and n-sided patches with boundary-position/tangent/curvature targets;
- variable-radius, chordal, rolling-ball, setback, and corner blends;
- sweep/pipe with frame singularities, twist control, guide curves, and curvature spikes;
- trimmed and periodic surfaces, seam placement, offset failure, and repair without shape drift;
- global/local fairing with protected boundaries, sections, feature lines, and deviation budgets;
- symmetric and asymmetric automotive-style exterior panels with highlight-critical regions;
- Class-A exterior joined to Class-B/Class-C engineering surfaces, thickness, ribs, bosses, and tooling constraints;
- reverse-engineered point/section data fitted to an explicit tolerance with noise/outlier handling;
- export/import/rebuild comparisons that preserve analytic/spline intent where the target format permits it.

16.9.5 Local artifact conversion and promotion

Source work is complete only when the following local chain exists:

```
SOURCE_CANDIDATE
-> ADMITTED_SNAPSHOT
-> EXTRACTED_BEHAVIOR_SPEC or ELIGIBLE_LOCAL_SAMPLE
-> NORMALIZED_EXECUTABLE_TASK
-> LOCAL_C++_GROUND_TRUTH
-> FAMILY_DEDUPLICATION_AND_SPLIT
-> TRAIN / CHALLENGE / HIDDEN_EVAL assignment
-> TARGET_M4_SYSTEM_RESULT
-> PROMOTED or REJECTED_WITH_EVIDENCE
```

Promotion requires all of these:

1. the candidate improves at least one predeclared massive-assembly, simulation, or surface metric on untouched hidden families with confidence intervals or repeated-seed evidence;
2. exact graph counts, CAD validity, deterministic commitment, replay, source-use, contamination, privacy, and safety gates remain green;
3. no public benchmark answer, vendor output, test case, or near-duplicate crosses into its training family;
4. the improvement survives target-M4 memory, thermal, latency, storage, offline, cancellation, and restart tests;
5. simpler ablations prove which source-derived component caused the gain;
6. rollback removes the component and every derived artifact through recorded lineage;
7. the report states failures and unchanged metrics as prominently as gains.

The default answer to "did this source boost Archie?" is **UNPROVED** until this chain passes. Citation count, downloaded bytes, parameter count, attractive surfaces, solver completion, or one benchmark increase is not a promotion signal by itself.

17. Benchmark and evaluation constitution

The target is leadership through transferable engineering competence. The user's desired threshold is **0.80 or greater on every normalized primary score where 0-1 semantics make that target meaningful**, while also reporting every raw official metric. No single fabricated aggregate may hide weak dimensions.

17.1 Public benchmark matrix

Benchmark	What it tests	Official evidence and treatment	Target policy
BenchCAD	industrial parametric generation/editing, vision/code/QA across many families and standards	execution-grounded official harness; all scored tasks held out	≥ 0.80 on each normalized primary task metric; publish validity and family breakdown
neuralCAD-Edit	expert real-world multimodal CAD editing	official 192 requests/384 edits and human/automatic evaluations held out	≥ 0.80 where metric is normalized; no degradation of non-target geometry
Drawing2CAD	engineering drawing to editable parametric CAD	official test split held out; report geometry and parameter/history metrics	≥ 0.80 normalized primary score plus validity and editability gates
ParaCAD / PHT-CAD	technical drawing geometry and annotation understanding	real-world official test drawings held out	≥ 0.80 normalized primary score and per-annotation/geometry breakdown
Text2CAD	natural-language to CAD sequence/shape	official split held out; applicable terms respected	≥ 0.80 on meaningful normalized metrics; validity and requirement coverage mandatory
HistCAD	constraints, parametric history, and editability	benchmark cases held out	≥ 0.80 normalized composite only with raw constraint/history/edit metrics
MUSE	functional, manufacturable, assemblable engineering design	official design cases held out; execution and rubric evidence retained	map no metric artificially; target ≥ 0.80 only for official normalized metrics and publish rubric subscores
CADGenBench	drawing-based generation and STEP editing	official inputs/hidden ground truth; strict submission isolation	≥ 0.85 stretch target, ≥ 0.80 release target across reported normalized dimensions

Leaderboard screenshots are historical leads, not authoritative score records. Rankings are recorded only with official dated results, version, submission hash, and validation status.

The repository material reviewed for this rewrite describes a historical CADGenBench v18 result of approximately 0.507, but does not by itself establish the current official rank or a clean reproducible baseline. Before "reclaiming" a position, rerun the pinned submission through the official process, archive the validation evidence, and then measure improvement from that verified point.

17.2 System-level evaluation

Benchmarks evaluate the whole Archie + Forge system unless a benchmark explicitly isolates the model. Report:

- model package, quantization, prompt/workflow version, kernel/compiler version;
- allowed tools, retrieval, attempts, wall time, and hardware;
- valid-output rate before similarity scoring;
- pass@1 as primary; additional attempts separately;
- score distribution, confidence intervals, and task-family breakdown;
- cost, latency, token use, graph size, and repair count;
- failure taxonomy and artifact links;
- any benchmark-specific adapter code.

17.3 Contamination firewall

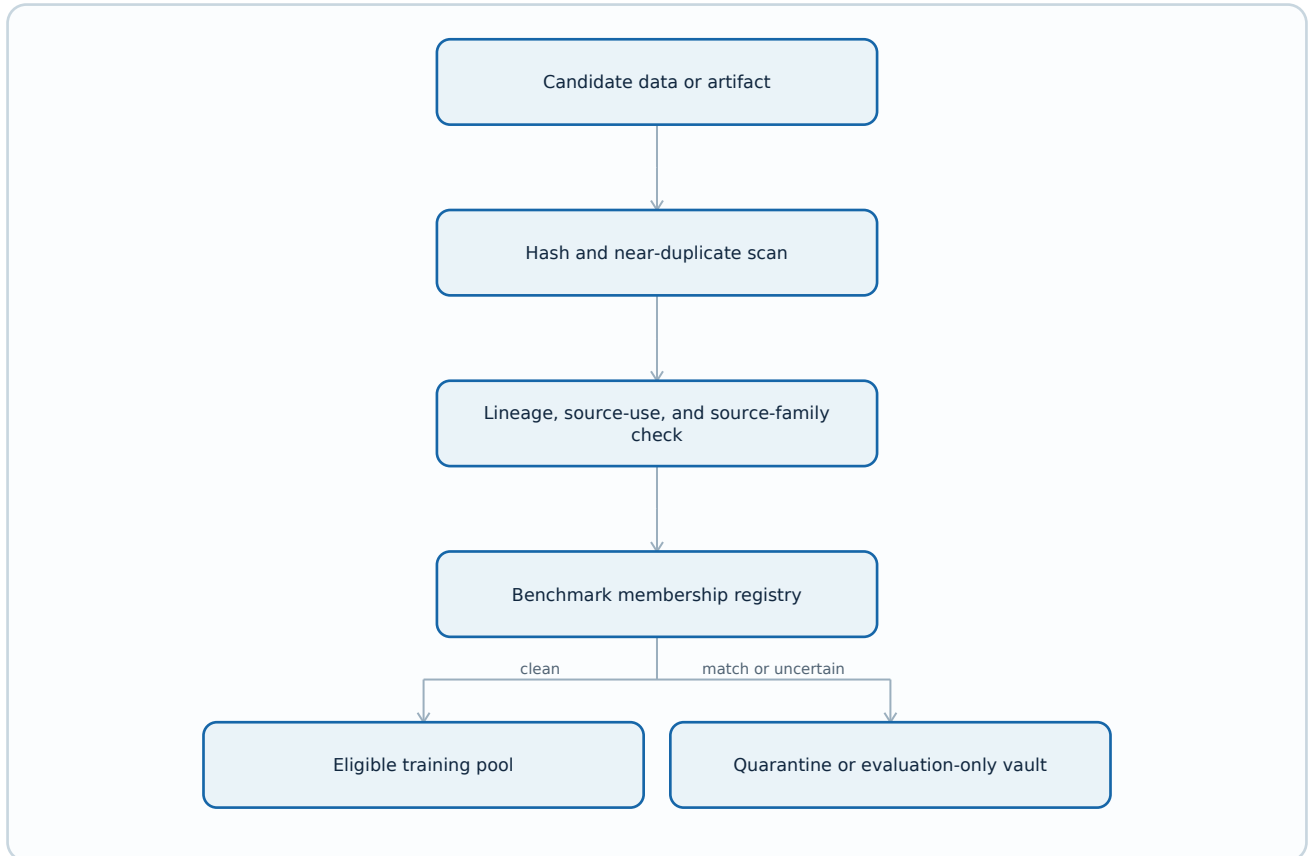


Figure 15. 17.3 Contamination firewall

- benchmark inputs, ground truth, public answer logs, and submission outputs are registered by exact/perceptual/geometry hashes;
- near duplicates and descendants are excluded by family/lineage, not only bytes;
- evaluation credentials and data are inaccessible to training jobs;
- no retrieval from benchmark repositories during scored runs unless rules expressly allow it;
- prompts, adapters, and hyperparameters are selected on development sets, never the final test set;
- every release includes a signed contamination report;
- suspected leakage invalidates the score until a clean rerun.

The old strategy of learning CADGenBench answers "by heart" is permanently prohibited. Historical submission logs may be retained in a quarantined audit vault to study infrastructure failures, but not used as target answers or adjacent-task retrieval.

17.4 Hidden Forge evaluation suite

Public leaderboards are not enough. The hidden suite must cover:

- novel design families and parameter distributions;
- adversarial ambiguous or contradictory multimodal requests;
- exact long-DAG tasks across S through XXL bands;
- edit chains with topology-changing upstream modifications;
- imports with unit, tolerance, orientation, and corruption traps;
- assemblies, configurations, BOM, mates, and drawings;
- DFM, GD&T, inspection, and basic solver setup;

- prompt injection and malicious files;
- crash/resume, checkpoint corruption, and resource exhaustion;
- held-out human industrial reviews with blinded scoring.

No model release sees the hidden answer artifacts. A rotating portion is retired only after potential leakage, then replaced before it may be used for research.

17.5 Engineering quality scorecard

Geometry similarity is only one axis.

Axis	Example measures
Validity	parse, compile, replay, body validity, export round-trip
Fidelity	dimensions, interfaces, topology, silhouette/volume, drawing correspondence
Parametric quality	fully constrained intent, stable references, configurations, edit survival
Exactness	requirement coverage, count reconciliation, no truncation/placeholders
Manufacturing	process-specific rule satisfaction and documentation
Assembly	occurrence/BOM consistency, mate solve, interference, motion
Physics	setup validity, convergence/sensitivity, acceptance quantities
Reliability	pass@1, calibration, deterministic commitment, crash recovery
Efficiency	wall time, peak memory, graph efficiency after correctness
Trust	citations, provenance, limitations, approval behavior, data isolation

Hard validity gates precede weighted scores. An invalid or truncated result cannot compensate through attractive rendering.

18. Local inference, model storage, and weight delivery

18.1 Preferred package

- official or reproducibly converted GGUF;
- Q4_K_M as the initial memory/performance candidate, with Q5/Q6 challengers;
- matching vision projector/encoder package;
- pinned tokenizer, chat template, grammar/schema bundle, and runtime commit;
- signed manifest with upstream source record, hashes, quantization provenance, evaluation results, and minimum Forge version.

18.2 Local mapped weights; network inference and weight streaming are forbidden

The production path stores the complete selected package locally and memory-maps its shards. Operating-system paging and llama.cpp's mapped loading avoid an unnecessary full copy while preserving predictable access. Every shard, tokenizer, projector, adapter, grammar, and manifest required by an activated package is verified before atomic activation.

Fetching experts, layers, embeddings, KV state, prompts, or completions over the network during generation is forbidden. Archie exposes no remote-inference mode, hosted-model fallback, vendor endpoint, or managed network cache. Model installation is a separate administrator action from a verified local bundle or explicit import; a user job never downloads or activates a partial package.

For the 36 GB workstation, the practical strategy is a measured quantized local MoE package, local `mmap`, bounded KV cache, graph state outside context, one loaded champion plus an on-disk rollback, and aggressive state checkpointing. If the selected 30B candidate cannot meet memory, thermal, latency, or quality gates locally, it remains unpromoted; the system may not conceal the failure by borrowing network compute.

18.3 Context management

The prompt assembler gives the model:

- current user turn and active decision;
- normalized ESG slice and unresolved ambiguities;
- feature-DAG symbol table and dependency neighborhood;
- exact count ledger and chunk contract;
- relevant kernel/tool errors and observations;
- selected cited references;
- concise immutable summaries of older work.

It never re-serializes an entire enormous project by default. Retrieval uses graph reachability, semantic roles, requirements, edit target, and recency. Summaries are non-authoritative and cannot overwrite graph state.

18.4 Performance harness

Test cold and warm startup, prompt processing, image encoding, first-token latency, generation throughput, graph parse/compile per node/chunk, viewport update, validation, save/resume, and full task time. Workloads cover small parts, drawing reconstruction, long feature graphs, assemblies, and concurrent geometry/rendering.

Thermal and memory tests run long enough to reveal throttling and swap. Results include median, p95, p99, failures, and resource peaks-not one favorable instantaneous number.

19. Product, workflow, and user-interface architecture

19.1 Product lessons from the market

Public product claims are useful requirements leads, not evidence that a competitor's implementation has been reproduced.

Reference	Publicly visible pattern Archie should absorb	Archie's stronger contract
P-1 AI	custom agentic harness, structured design representation, continual skills learning, SFT/RLVR, requirements synthesis, component selection, trade studies, and professional engineering-tool use	exact native feature DAG, deterministic Forge execution, local deployment, evidence gates, and transparent maturity by capability
P-1 AI demo	end-to-end engineering workflow across requirements, tools, analysis, and artifacts	reproduce each workflow class through typed tools and archived verification, never by marketing imitation
Prometheus, reported public scope	AI for designing and automating manufacture of complex physical systems; public reporting also describes an "artificial general engineer" ambition	keep sophisticated physical-product design, manufacturing, robotics, and multidisciplinary engineering inside the end-state scope; invent no proprietary architecture that has not been published
Kyrall	conversational generation of functional parametric CAD intended for manufacture	explicit ambiguity ledger, exact feature history, manufacturing predicates, editable delivery, and local privacy
Adam	work across CAD, PLM, BOM, sourcing, drawings, revisions, communication, and human approvals	one project truth graph, scoped edits, source citations, side-effect controls, and Forge-native geometry

P-1 AI publicly calls its engineering agent **Archie**. Before this project uses Archie as an external product name, counsel must perform trademark/name-confusion review. The technical codename can remain internal while branding is decided. No proprietary competitor implementation, data, prompt, or trade secret should be inferred or copied; only publicly stated product requirements and independently developed methods belong here.

Prometheus has not publicly disclosed a complete product or technical architecture. "Include all of Project Prometheus" can therefore mean include its publicly reported physical-engineering outcome space, not claim access to confidential systems. As credible details appear, they enter the source registry and capability matrix through normal evidence review.

19.2 Desktop technology

The selected application stack is open-source modern C++20/23 with CMake, Ninja, and the following single-responsibility layers. Every required component is source-buildable through the local dependency plane; closed binaries are not accepted in the core application path.

Layer	Selected open-source framework	Forge contract
Industrial desktop shell	Qt 6 Widgets , C++ only	native menus, actions, text, input, accessibility, model/view, dialogs, printing, and platform integration; no QML or JavaScript runtime in the shipped path
Docking/workspaces	KDDockWidgets	saveable NX/CATIA-style workspace layouts, tabbed/floating tools, multi-monitor recovery, central viewport, deterministic default layouts
Real-time viewport and compute	Diligent Engine	Metal backend on Apple Silicon; Vulkan and Direct3D backends later; one Forge render graph for CAD, overlays, picking, result fields, and compute
Exact CAD visualization source	OCCT tessellation/topology plus Forge stable IDs	B-rep remains truth; tessellation packets carry stable entity IDs, revision, chord/angle policy, and selection lineage
Optional presentation rendering	Filament in an isolated render worker only if evaluation proves value	high-quality PBR images/turntables; never owns CAD selection, measurements, engineering overlays, or model truth
Scene/material interchange	OpenUSD and MaterialX adapters	DCC exchange, large-scene composition, appearances, camera/light metadata, and non-destructive round trips

The product has one authoritative interactive renderer. Competing render abstractions are challengers in benchmarks, not simultaneous core paths. On the M4 Max the primary backend is Metal, shader binaries are produced ahead of time, and pipeline caches are warmed before an interactive job. OCCT's interactive viewer may remain a diagnostic reference during migration but does not become a second production interaction model.

19.2.1 CAD command and selection architecture

The UI is not wired directly to widgets. Every command is a versioned C++ descriptor with stable ID, label, icon key, category, shortcut, required selection signature, enabled predicate, parameter schema, preview policy, side-effect class, undo contract, help/evidence link, and equivalent feature-IR operation. Menus, ribbons/toolbars, command search, context menus, radial/marketing menus, macros, keyboard maps, and Archie tool calls all invoke the same command registry.

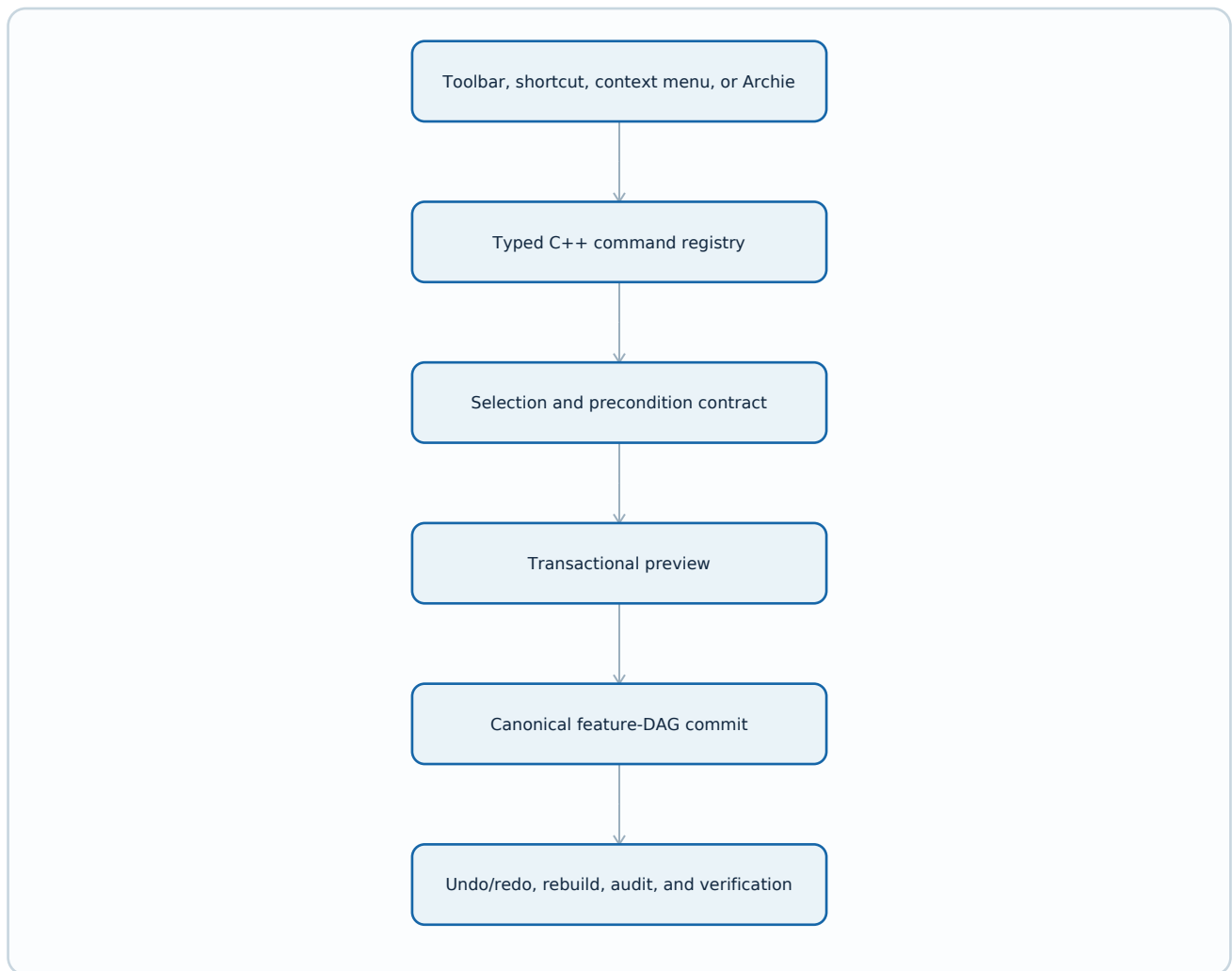


Figure 16. 19.2.1 CAD command and selection architecture

Selection is a typed service, not ad hoc viewport state. Filters distinguish bodies, faces, edges, vertices, curves, sketch entities, datums, occurrences, mesh regions, result probes, drawing objects, and semantic roles. Preselection, selection, focus, and committed references are separate states. Every committed geometric reference passes the stable-topology resolver.

19.2.2 Professional workspaces and tool mapping

Forge ships deterministic workspace profiles inspired by the interaction depth of NX, CATIA, and Blender without copying their protected assets or hidden behavior:

Workspace	Primary panels and interactions
Part Design	feature history, dependencies, parameters, constraints, section/measure, design checks
Sketch	constraint palette, DOF diagnostics, dimensional expressions, construction geometry, profile inspector
Assembly	product tree, mates/joints, configurations, interference, motion timeline, BOM
Surface/Class-A	continuity, curvature combs, zebra/reflection lines, deviation and patch diagnostics
Sheet Metal/Manufacturing	process features, flat state, tools/setups, DFM, stock and operation tree
Drawing/PMI	sheets/views, annotations, GD&T, tables, sections/details, print/export checks
Simulation	analysis graph, materials, loads/contacts, mesh, live controls, probes, plots, convergence, results
Archie	request/evidence, ambiguity ledger, candidate plan, graph emission, gates, repair, citations, resume
DCC/Presentation	scene hierarchy, appearances, lights/cameras, animation, OpenUSD/MaterialX exchange

Key maps are profile-based and user-remappable. **Forge Native**, **NX-like**, **CATIA-like**, and **Blender-like** profiles map familiar navigation and selection conventions onto Forge command IDs; they do not create separate

implementations. A command-map validator detects collisions, unreachable commands, missing context predicates, and platform-reserved shortcuts. Import/export adapters preserve names, layers, colors, hierarchy, coordinate systems, metadata, and stable cross-reference records where the source format permits them.

19.2.3 Viewport budgets and engineering truth

The viewport targets smooth interaction under concurrent Archie and simulation work. Frame timing is measured as input-to-photon, not only GPU time. Geometry upload, selection acceleration structures, labels, plot overlays, simulation fields, and image capture use revisioned buffers and explicit memory budgets. Large assemblies use hierarchical culling, level of detail, instancing, deferred loading, and origin rebasing without changing the B-rep or occurrence graph.

Engineering overlays - dimensions, constraints, section caps, contact states, boundary conditions, mesh quality, contours, vectors, streamlines, probes, uncertainty, and pass/fail regions - carry units, coordinate frame, geometry revision, analysis revision, time/load step, and result hash. If any identity mismatches, the viewport refuses to present the overlay as current.

19.3 Workspace anatomy

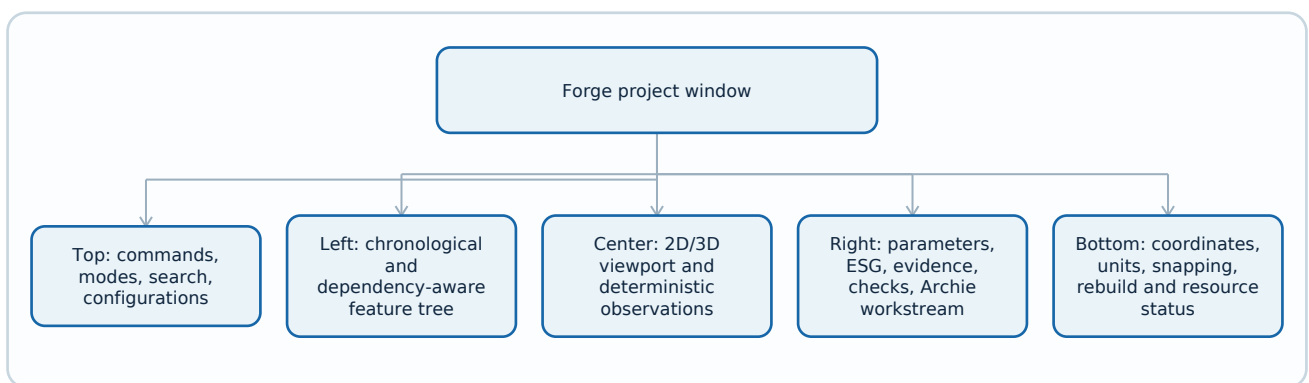


Figure 17. 19.3 Workspace anatomy

The interface supports expert density without hiding system state:

- command search and keyboard-first workflows;
- dockable but recoverable layouts;
- selection synchronization across ESG, feature graph, viewport, drawing, BOM, and evidence;
- explicit preview versus committed geometry;
- rollback/rebuild position and dependency impact;
- configuration and revision context always visible;
- unit-aware input with expression support and dimensional validation;
- accessible names, keyboard navigation, scalable text, high contrast, and non-color status cues;
- deterministic capture modes for training/evaluation observations.

19.4 Feature-tree UI for enormous graphs

The UI must show the entire logical project without trying to render every row simultaneously.

- virtualized, lazy tree rows with bounded memory;
- hierarchical part/subassembly/configuration navigation;
- chronological, dependency, requirement, manufacturing, and failure views;
- expandable typed patterns and library components with child counts;
- visible subtree node count, unresolved count, status, and hash;
- filters by feature kind, requirement, body, source, author, revision, configuration, and gate status;

- graph search by stable ID or semantic role;
- diff mode for revisions and scoped edits;
- breadcrumb and minimap for deep trees;
- no collapse operation may alter or discard the underlying graph;
- export of a machine-readable full graph and a human-readable summarized report.

For a million-node project, the user can see 1,248,552 total nodes; 0 unresolved; 34,120 suppressed by current configuration without loading a million visual rows. Expanding a subtree streams its indexed children from the project store.

19.5 Archie workstream UI

The workstream exposes phase, current target, committed chunk, running count ledger, last valid geometry, applicable gates, cited evidence, assumptions, pending questions, and cancellation/resume controls. It does not expose private chain-of-thought or simulate confidence with vague animation.

Interrupts present:

- the decision needed;
- two or more materially distinct options;
- evidence and consequences;
- Archie's recommendation with uncertainty;
- whether the choice is reversible;
- the exact downstream graph region affected.

19.6 Visualization rules

- Charts are used for numerical trends, sensitivity, convergence, tolerance distributions, Pareto fronts, and benchmark breakdowns.
- Mermaid-style topology diagrams are used in documentation for architecture, workflows, states, and dependencies.
- Exact mappings use tables and matrices.
- CAD geometry is viewed in a real interactive viewport with measurements and sections, not replaced by a diagram.
- Every plot carries quantity, units, coordinate frame, data revision, and source/result hash.
- Color maps are perceptually suitable, accessible, and paired with numeric legends; default rainbow stress plots are prohibited.
- Animation represents time, configuration, motion, or a result sequence and includes pause/scrub controls. Decorative motion cannot imply progress or certainty.

20. Security, privacy, safety, and governance

20.1 Threat model

Threat	Required control
Malicious prompt or retrieved-page injection	treat content as data; typed tools; instruction hierarchy; allowlisted operations; no web-to-executable path
Malicious CAD/PDF/image/archive	sandboxed parsers, fuzzing, resource/decompression limits, quarantine
Model emits arbitrary code	closed IR vocabulary; no compilation or shell execution from model text
Customer IP leakage	local-first processing, workspace isolation, query redaction, encryption, permission-scoped retrieval

Threat	Required control
Cross-project memory leakage	tenant/project ACLs enforced before retrieval; isolated indexes and caches
Poisoned dataset or dependency	provenance, signatures, hashes, reproducible builds, SBOM, quarantine, rollback
Unsafe engineering recommendation	applicability checks, deterministic verification, uncertainty, approval and certification boundaries
Unintended export or locally attached physical action	side-effect classes, least privilege, preview, exact local target verification, explicit approval, no direct machine motion
Benchmark or training contamination	isolated vaults, hash/lineage registry, signed reports
Model/package theft or tampering	signed manifests, secure local permissions, optional encryption, integrity verification

20.2 Local-only execution and single-tool network policy

Archie runs locally with outbound network denied by default. The only production exception is an explicit SearXNG browser-tool invocation. The C++ client sends a minimized, redacted research query to the same-Mac SearXNG sidecar; the UI previews the query, destination class, and fields before transmission. Search failure returns `RETRIEVAL_UNAVAILABLE` and never triggers another web client, hosted embedding, remote index, remote model, remote solver, telemetry upload, or compute fallback.

Model/dependency/update acquisition is an administrator-controlled installation operation outside a user engineering job and must support verified offline bundles. Normal intake, reasoning over installed evidence, CAD generation, editing, compilation, rendering, simulation, validation, persistence, export, and replay are proven with the network physically or programmatically denied.

Credentials live in the operating-system credential store, never prompts, logs, project files, or model context beyond a scoped tool handle. Tool responses are filtered for secrets before persistence.

20.3 Data governance

- default customer retention is project-controlled and local;
- telemetry is off or privacy-preserving by explicit policy, never an unannounced training channel;
- user deletion propagates through indexes, caches, derived datasets, and future training manifests where legally/technically required;
- project exports include provenance without credentials or unrelated private data;
- role-based access separates view, edit, approve, release, train, and administer;
- audit records are append-only and tamper-evident;
- legal hold and regulated retention are configuration, not ad hoc copies.

20.4 Engineering safety classes

Each job declares a consequence class based on domain and intended use. Higher classes require stronger data, solver validation, review, and approvals. Consumer visualization, a shop aid, a lifting fixture, a pressure boundary, medical hardware, and flight hardware cannot share identical acceptance policy.

Archie must refuse to mark a design "certified," "safe," or "code compliant" without the required evidence and authorized approval. It may report calculated margin and applicable checks with limitations.

20.5 Supply-chain governance

Every release generates SPDX/CycloneDX-compatible SBOM data, source-use inventory, source hashes, vulnerability status, patch list, and build provenance. Dependencies are pinned and mirrored where permitted. Update packages are signed, atomic, reversible, and tested against saved-project migration.

21. Build, test, observability, and release engineering

21.1 Repository architecture target

```

forge/
  app/          # native C++ application shell
  ui/           # feature tree, panels, commands, accessibility
  inference/    # llama.cpp integration, multimodal preprocessing
  orchestration/ # durable workflow graph and typed tools
  spec/         # Engineering Specification Graph
  feature_ir/   # schema, C++ types, parser, canonical serializer
  compiler/     # lowering and transactions
  kernel/      # OCCT and other audited adapters
  constraints/  # sketch and assembly constraint systems
  verification/ # geometry, requirements, edit, DFM, export gates
  manufacturing/ # process graphs and rule engines
  simulation/   # solver schemas/adapters and result store
  interchange/  # CAD/drawing/project formats
  project_store/ # revisions, chunks, hashes, indexes, migrations
  storage/     # managed-root inventory, leases, quarantine and GC receipts
  retrieval/    # local evidence and SearXNG C++ client
  evaluation/   # hidden/public harness adapters
  tests/       # unit, property, fuzz, integration, E2E, golden
  third_party/  # lock, build recipes, patches, source-use records
  cmake/       # presets, toolchains, superbuid and offline gates
  shaders/     # ahead-of-time viewport/compute shader sources
  tools/deps/  # seed, verify, export, import and activate local bundles

```

The legacy Electron/JavaScript application is migrated capability by capability behind compatibility tests. Deletion occurs only after the native path reproduces required project behavior and migration/rollback is proved. The final cutover is mandatory, not optional: a dedicated migration-manifest commit removes tracked Electron/Node/JavaScript/TypeScript code and JavaScript tests, a separate repository-local cleanup removes resolved Node-derived artifacts, and CI installs a zero-JavaScript plus non-C++-allowlist gate.

21.2 Local build, activation, and fast execution contract

The Forge repository is the reproducibility control plane; `.forge-local/` is the workstation execution plane. A checked-in `deps.lock.json` records every source revision, archive hash, enabled feature, patch hash, toolchain constraint, target triplet, dependency edge, and expected artifact fingerprint. A generated `deps.resolved.json` records the exact realized build and host facts without silently changing the lock.

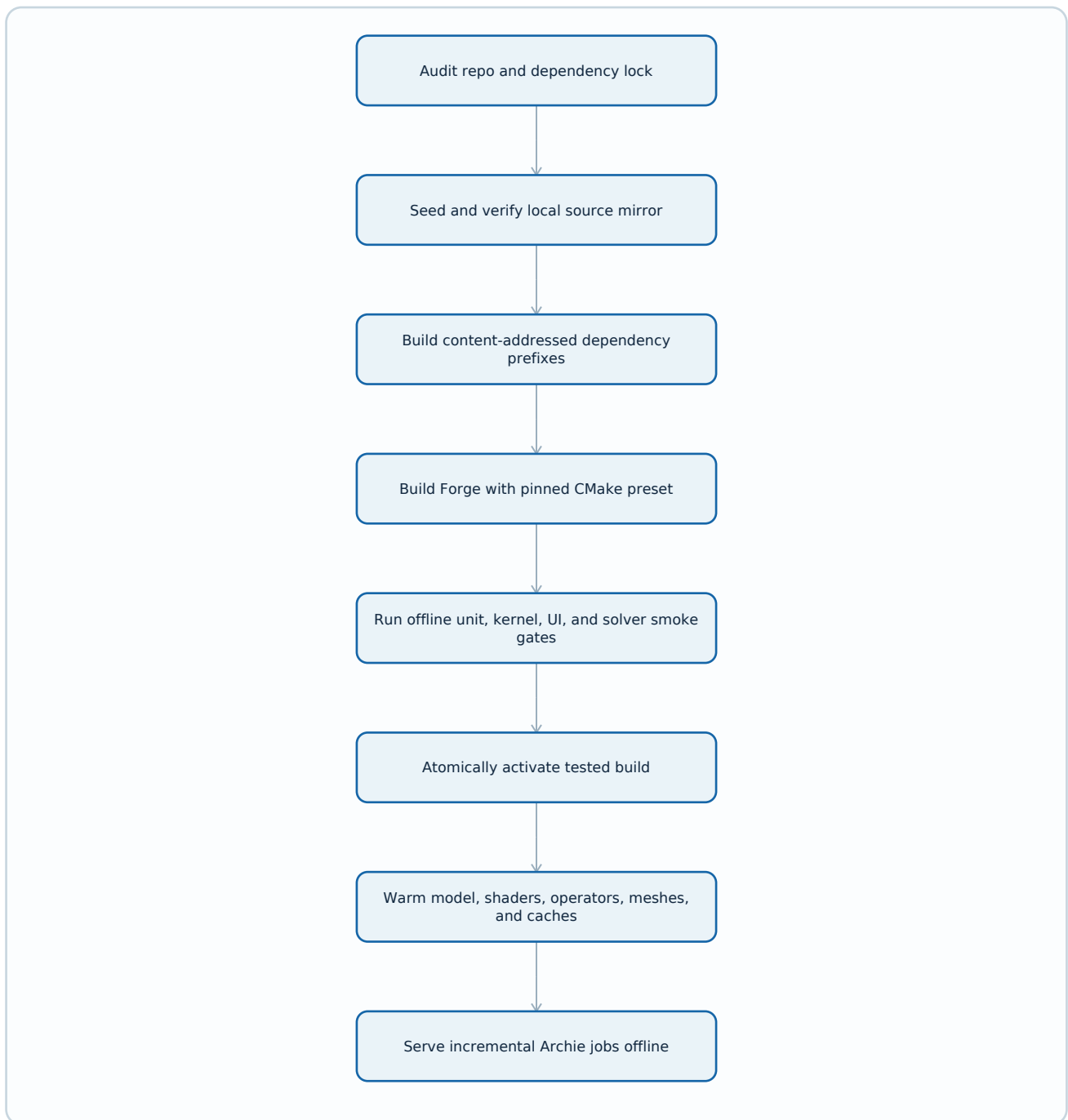


Figure 18. 21.2 Local build, activation, and fast execution contract

Required commands are stable wrappers, not tribal instructions:

```

tools/deps/seed --lock third_party/manifest/deps.lock.json
tools/deps/build --preset macos-arm64-release
tools/deps/export-bundle --output <explicit-path>
tools/deps/import-bundle --input <explicit-path> --verify-only|--activate
cmake --preset macos-arm64-release-offline
cmake --build --preset macos-arm64-release --target forge_all
ctest --preset macos-arm64-release
forge-selftest --offline --kernel --ui --inference --simulation

```

The implementation may use a manifest-mode C++ package manager for ordinary dependencies and a Forge superbld for specialized kernels/solvers, but the lock and activated prefix remain authoritative. Configure/build/test must fail when a missing artifact would require network access in offline mode.

Performance rules:

- use Ninja, a compiler cache, dependency binary cache, precompiled headers where measured, and unity builds only where diagnostics and memory remain acceptable;
- compile shaders and operator registries ahead of time; warm render pipelines and kernel workers after activation;
- isolate build output by target triplet, configuration, toolchain, dependency lock hash, and relevant feature set;
- never put workstation-specific absolute paths into serialized projects or release artifacts;
- never commit expanded dependencies, compiled prefixes, model weights, secrets, caches, or transient result databases;
- measure cold clean build, warm no-op build, one-file rebuild, kernel relink, application relink, startup, first feature, incremental graph edit, and first simulation step;
- make correctness-independent caches disposable and reproducible from the lock and source mirror;
- run at least one network-disabled clean build and network-disabled full smoke test before release.

21.3 Standing storage-health and proof-based garbage collection

Forge owns a native C++ `forge-storage` governor. It protects capacity without risking source, user work, reproducibility, benchmark integrity, or rollback. Its authority is limited to canonicalized roots explicitly registered in the Forge configuration; it must never treat a home directory, volume root, workspace root, unresolved environment variable, symlink escape, or arbitrary extension search as a deletion scope.

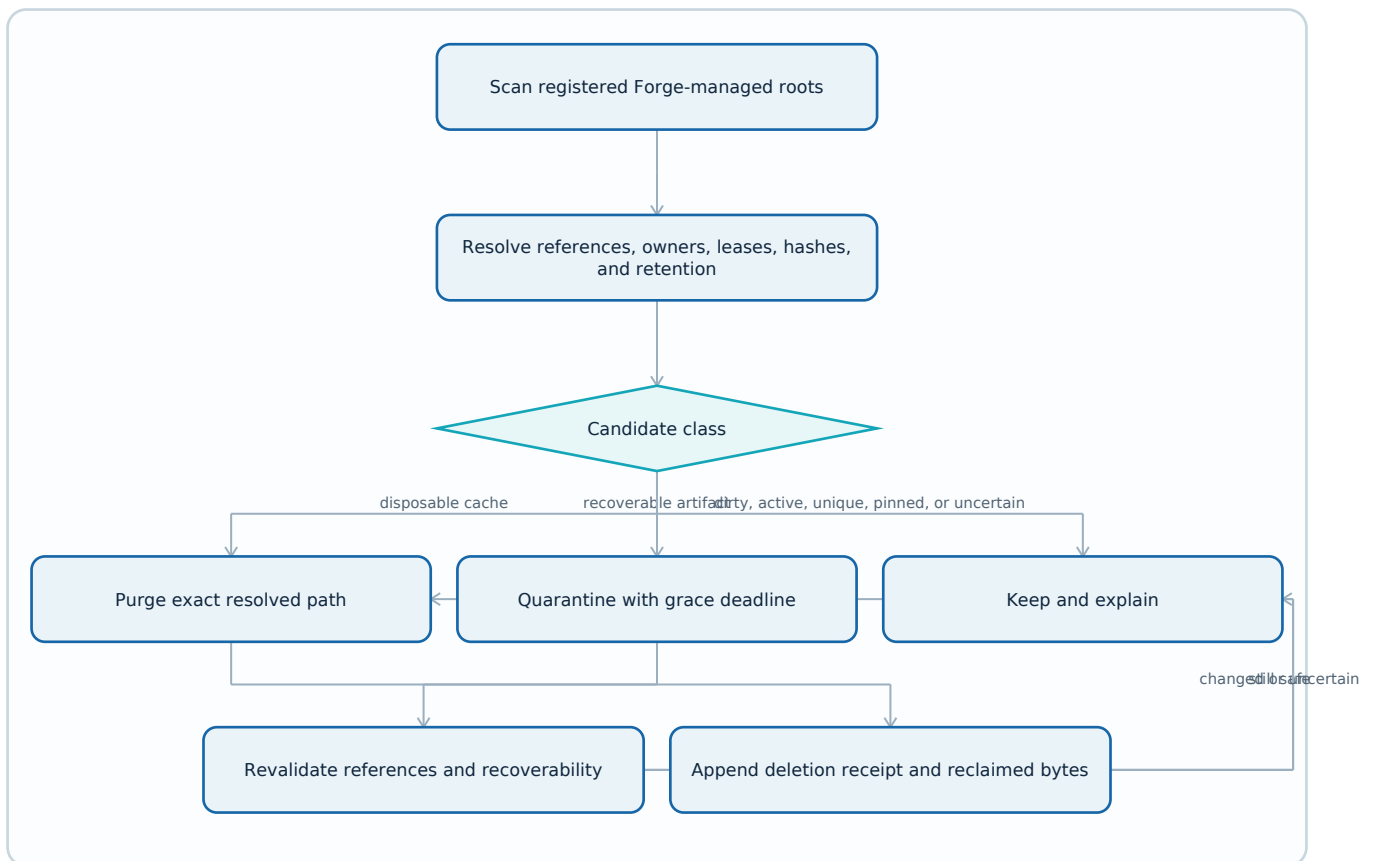


Figure 19. 21.3 Standing storage-health and proof-based garbage collection

21.3.1 Artifact record and states

Every managed artifact carries or inherits:

- canonical path below a registered root and artifact class;
- content hash or directory manifest hash;
- creating task, project, branch/worktree, tool/build/model/dataset version, and owner;

- upstream inputs and downstream references;
- creation, last verification, last access, lease expiry, and retention deadline;
- byte size, recomputation/reacquisition cost, and exact recovery method;
- remote/persistent copy status where applicable;
- state: **HOT**, **WARM**, **PINNED**, **ROLLBACK**, **ARCHIVED**, **GC_CANDIDATE**, **QUARANTINED**, or **PURGED**;
- reason code and approver/policy for each state change.

No artifact becomes **GC_CANDIDATE** merely because a task says "processed." It must be fully consumed, its accepted derivative and provenance must exist, no live reference may remain, and deletion must not destroy the ability to reproduce an accepted result when that reproducibility is required.

21.3.2 Class-specific retention law

Artifact class	Eligible only when	Must be retained
Agent Git worktree	not current; no active agent/session/lease; clean index and working tree; branch commits integrated or deliberately preserved; no unique ignored artifacts; task terminal	any dirty/untracked uncertainty, unpushed unique commit, active task, unresolved review, or rollback need
Build tree/object/cache	activated artifact and lock identify the build; no process holds it; cache entry is correctness-independent	current build, last known-good rollback, active debug reproducer, or artifact needed by a running job
Dependency source/archive/prefix	superseded and no activated build/lock/bundle references it; reacquisition or archived bundle is verified	active lock, rollback lock, offline bundle, unique patch basis, or source required for exact reproduction
Model weights/adapters/KV caches	not active/rollback/challenger-under-test; manifest, evaluation, and reacquisition/conversion recipe are archived; no live session maps the file	production champion, at least one verified rollback where capacity permits, active challenger, unique trained checkpoint awaiting durable archival
Raw/intermediate datasets	accepted derivative has exact lineage; holdout/benchmark membership is preserved; deletion policy permits loss; durable source or reacquisition exists when required	unique raw data, evaluation holdouts, contamination records, deletion/consent lineage, unresolved samples, or data required to reproduce a promoted model
CAD/project/simulation results	superseded and unreferenced; user/export/audit policy permits it; required summary/hash/provenance exists	user projects, released revisions, approval evidence, active analyses, failure reproducers, golden tests, or any unique engineering result
Documents/images/videos/logs	generated intermediate is superseded and exact regeneration exists, or durable accepted copy is verified	normative specs, accepted PDFs, audit/deletion receipts, source evidence, user attachments still referenced, and release evidence
Temporary extraction/download/transcode	transaction complete or failed and no resume lease remains	active transaction, resumable partial explicitly retained, quarantine evidence

21.3.3 Cleanup protocol

1. **forge-storage scan** computes capacity, free-space headroom, class totals, age, reference graph, active process handles, Git/worktree state, leases, and candidates.
2. **forge-storage plan** emits a deterministic dry-run manifest with exact canonical paths, bytes, reason, proof checks, recovery method, and exclusions.
3. Cheap disposable caches may be deleted immediately after path, reference, and active-handle validation.
4. Valuable or expensive candidates move atomically to a Forge quarantine on the same volume, preserving their manifest and original path. The default grace period is configurable and never silently shortened by disk pressure.
5. Before purge, Forge repeats every eligibility check. A new reference, dirty state, lease, failed archive verification, or ambiguity returns the artifact to **KEEP**.
6. Purge uses exact manifest entries, never recursive wildcards. Each action appends a tamper-evident receipt containing path hash, content/manifest hash, bytes, reason, policy, timestamps, and recovery status.
7. Cleanup is crash-safe and idempotent. A crash cannot leave the registry claiming data is present when it is absent.

Worktree cleanup additionally runs repository-aware checks: enumerate with Git, enter the exact worktree, verify clean status and no untracked/ignored unique artifacts, verify the branch tip is reachable from the integration/remote preservation point, verify no Claude task/session owns it, remove the worktree through Git, then prune only stale administrative records. A directory named "worktree" is never enough evidence.

21.3.4 Storage pressure and scheduling

The user configures a minimum free-space target as both a percentage and absolute byte floor. Defaults are derived from measured volume size and workload, then shown for approval. Status bands are **HEALTHY**, **WATCH**, **PRESSURE**, and **CRITICAL**.

- **WATCH**: stop speculative cache growth and present the dry-run reclaim plan;
- **PRESSURE**: pause new large downloads/builds/training imports, purge safe disposable caches, quarantine proved candidates, and resume only when projected peak fits with headroom;
- **CRITICAL**: stop state-expanding jobs at safe checkpoints; preserve projects, manifests, databases, current/rollback packages, and audit records; perform only pre-authorized exact cleanup;
- no pressure state permits deleting uncertain, dirty, unique, user-authored, or pinned data.

The storage governor runs at application startup in scan-only mode, before any operation with a declared large footprint, after dependency/model/dataset activation, after agent-team integration waves, after completed import/training/evaluation pipelines, and on a configurable low-priority schedule. It reports current use, projected peak, reclaimable bytes by confidence, quarantined bytes and deadlines, and the largest blockers.

21.4 Test pyramid

- unit tests for units, schemas, IDs, canonicalization, topology queries, and rules;
- property/metamorphic tests for transforms, unit conversion, ordering, round-trip, and invariants;
- geometry regression tests for every feature family and known degeneracy;
- fuzz tests for parsers, IR, CAD imports, graph migrations, and query resolution;
- integration tests from ESG through compile and verification;
- golden project replay across versions;
- long-DAG tests far beyond one context window;
- crash/power-loss injection at every transaction boundary;
- performance/memory/thermal tests on the target Mac;
- security tests for prompt injection, malicious files, path traversal, secret leakage, and tool authorization;
- blinded expert acceptance and benchmark runs.

Use sanitizers and static analysis in supported configurations: AddressSanitizer, UndefinedBehaviorSanitizer, ThreadSanitizer, clang-tidy, compiler warnings as errors for first-party code, dependency scanning, and coverage focused on safety/transaction paths.

21.5 Observability

Each workstream produces structured events with timestamps, correlation IDs, feature/workflow node IDs, versions, resource use, state transition, result class, and artifact hashes. Logs never contain hidden chain-of-thought, credentials, raw private evidence beyond policy, or enormous geometry dumps by default.

Metrics include:

- intake and question resolution latency;
- candidate/commit token and time cost;
- graph nodes planned/emitted/compiled/replayed;
- chunk size, failure, resume, and count mismatch;
- kernel operation latency/failure by feature kind;

- repair cause, patch size, attempts, and success;
- verification failures by gate;
- peak memory, swap, thermal state, and UI frame latency;
- benchmark and hidden-suite distributions;
- user override, rejection, and rollback rates.

21.6 Release gates

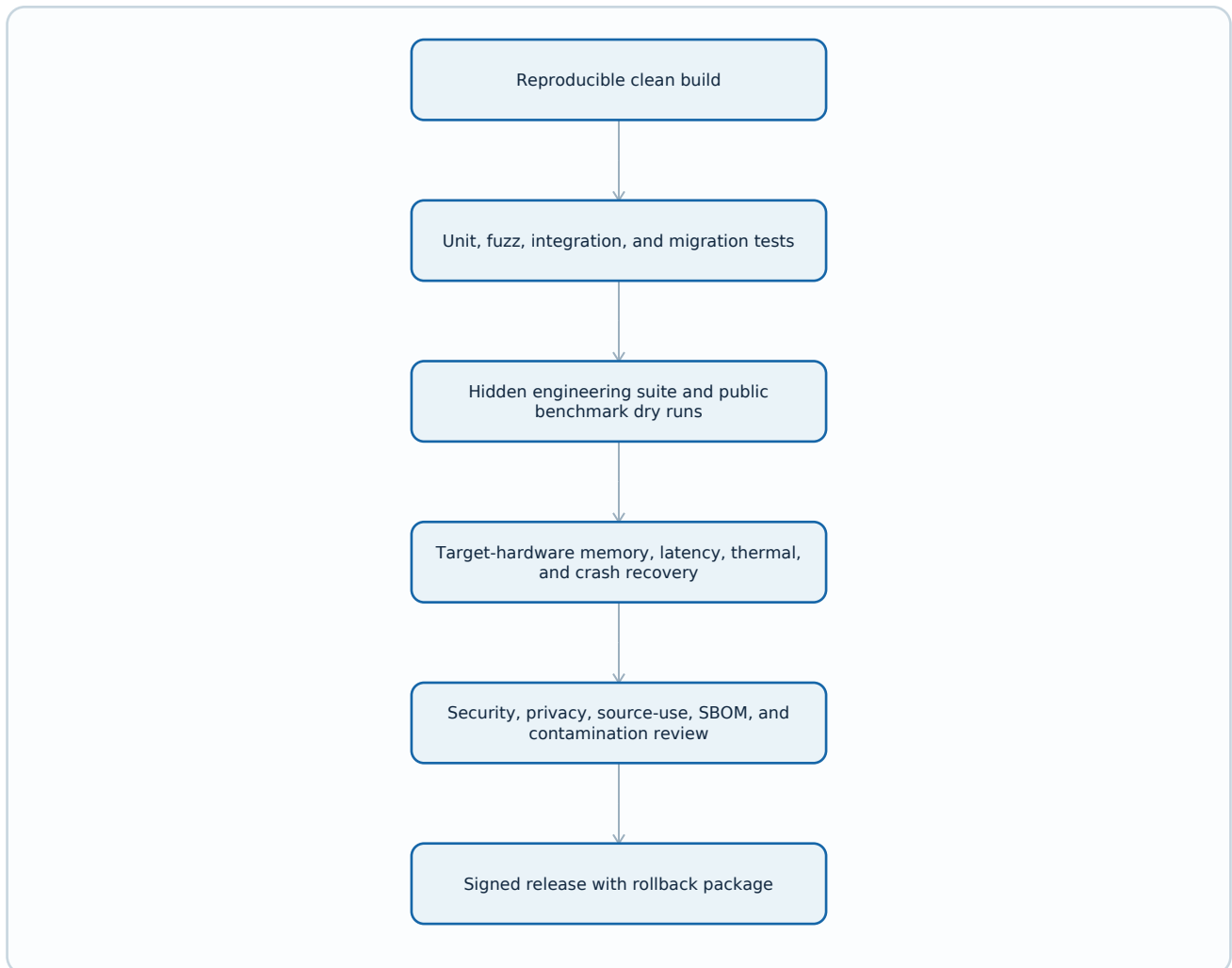


Figure 20. 21.6 Release gates

A release is blocked by any regression in graph exactness, silent truncation, project corruption, unresolved source-use issue in the actual mixture, benchmark leakage, critical security defect, or inability to reproduce the model/runtime package.

22. Implementation sequence

All end-state domains remain in scope; sequencing prevents broad claims from outrunning dependable foundations.

Phase 0 - truth, governance, and reproducibility

- replace legacy checkmarks with the status/evidence system in this document;
- inventory current C++, Electron/JavaScript, model, adapters, datasets, sources, build inputs, and runtime dependencies;

- create a file-and-behavior migration manifest for every JavaScript/TypeScript/Node/Electron path and every heavy non-C++ product implementation;
- quarantine benchmark answers and unaudited/noncommercial data;
- establish pinned builds, artifacts, SBOM, contamination registry, and hardware harness;
- create the dependency lock, local source mirror, offline bundle export/import, immutable prefixes, and network-disabled build gate;
- register Forge-managed storage roots, artifact classes, leases, quarantine, deletion receipts, and initial dry-run capacity baseline;
- preserve a clean reproducible baseline of current capability.

Exit: current claims are reproducible or explicitly reclassified; no hidden training/evaluation leakage; a clean local offline build, smoke suite, activation, rollback, and proof-based storage dry run exist.

Phase 1 - native C++ spine

- C++ application shell and project store;
- Qt 6 Widgets/KDDockWidgets workbench, unified command registry, Diligent/Metal viewport, stable typed selection, and prebuilt shader pipeline;
- 30B-class local multimodal inference candidate on Metal;
- native durable workflow engine;
- ESG, ambiguity ledger, exact feature IR, chunk protocol, canonical serializer;
- prebuilt operator registry, OCCT compiler transaction path, content-addressed incremental execution, persistent workers, and deterministic observations;
- SearXNG C++ client with evidence records.
- native [forge-storage](#) scan/plan/quarantine/purge core with worktree and cache adapters.

Exit: a text/image request creates a small exact C++-native graph, compiles through prebuilt local operators, renders in the native viewport, verifies, saves, restarts, and replays offline without JavaScript/Python in the shipped path.

Phase 2 - manufacturing-grade parts

- complete sketch/constraint foundations and stable topology naming;
- core prismatic, hole/thread, pattern, draft, shell, fillet/chamfer, surfacing, sheet-metal subset;
- exact drawing reconstruction and neutral export round-trip;
- initial machining, molding, sheet-metal, and additive DFM;
- feature-level repair and edit invariant harness;
- native analysis graph plus a bounded live multibody/contact/structural proxy loop tied to stable feature IDs;
- S/M/L long-DAG curricula and hidden tasks.

Exit: representative parts pass validity, dimensions, edit survival, DFM, exact counts, export gates, and at least one live-versus-confirmation physics correlation suite at target reliability.

Phase 3 - long-DAG and benchmark leadership

- XL/XXL chunked graph generation and dependency-slice retrieval;
- systematic SFT and execution-grounded RLVR;
- all eight public benchmark adapters under the contamination firewall;
- champion/challenger automation and failure data factory;
- target-hardware quantization/performance optimization.

Exit: reproducible public and hidden results meet defined thresholds without leakage, and 10×/100× graphs complete/resume exactly.

Phase 4 - assemblies, drawings, and lifecycle

- hierarchical occurrences, mates/joints, configurations, motion/interference;
- BOM, GD&T/PMI, tolerance, inspection, drawing, ECO/revision system;
- supplier/catalog research only through the explicit SearXNG browser tool, plus local file-based handoff packages and same-machine CAD adapters;
- L/XL assembly evaluation and product handoff artifacts.
- after all mapped browser behaviors have proved native replacements: remove tracked JavaScript/TypeScript/Node/Electron code and tests in a dedicated commit, clean exact repository-local Node artifacts, and enable the permanent zero-JavaScript/non-C++-allowlist CI gate.

Exit: a versioned assembly can be edited, rebuilt, documented, inspected, and handed off with reconciled product structure and evidence; the Forge product repository and its local build/output stores contain no JavaScript/TypeScript/Node/Electron implementation or artifacts.

Phase 5 - physics and multidisciplinary engineering

- validated multibody, structural, thermal, CFD, vibration/fatigue, and optimization schemas/adapters using the native local simulation plane;
- 30-60 Hz interactive loops where the declared method and target hardware permit them, with explicit degradation and validity envelopes;
- deterministic confirmation solvers, live-versus-confirmation correlation, solver verification corpus, convergence/sensitivity gates, and revision-safe field visualization;
- mechatronics, routing, systems requirements, digital-twin and field-feedback foundations;
- domain-specific approval and safety classes.

Exit: user-requested live virtual tests and bounded confirmation workflows reproduce reference problems, survive geometry edits with correct invalidation, and improve designs without pretending to replace certification.

Phase 6 - enterprise and broad physical-product platform

- PLM/PDM/ERP/MES/SCADA/BIM/GIS connectors according to demand and safety;
- enterprise identity, policy, audit, retention, deployment, and administration;
- controlled native-CAD host agents and approved proprietary-format adapter paths;
- industrial validation partners and longitudinal field correlation.

Exit: named workflows, domains, and connectors have individual evidence-backed maturity rather than one universal "done" badge.

23. Definition of done for Archie

Archie is not complete because it can make an attractive model, win one leaderboard, or execute a demo. The central product promise is proved only when representative unseen requests reliably complete this chain:

1. ingest mixed evidence without losing source provenance;
2. extract a complete ESG and expose material ambiguity;
3. reason over multiple plausible interpretations when appropriate;
4. select or ask with calibrated, concise engineering justification;
5. generate the exact C++-native feature DAG of whatever length is required;

6. checkpoint and resume graphs beyond a single context window;
7. compile every node incrementally through prebuilt local C++ operators with stable references and no arbitrary code;
8. verify requirements, counts, geometry, editability, manufacturing, assemblies, drawings, and physics as applicable;
9. run requested interactive physics/testing with an explicit rate and validity envelope, then run deterministic confirmation when the decision requires it;
10. invalidate and recompute exactly the analysis and geometry subgraphs affected by an edit;
11. repair the smallest responsible subgraph without collateral change;
12. deliver editable project history, neutral artifacts, documentation, citations, simulation evidence, and reproducibility evidence;
13. protect private data, prohibit outbound actions except the explicit redacted SearXNG browser request, and require approval for consequential local exports;
14. rebuild and execute the activated core application from the pinned local dependency bundle without network access;
15. prove every former Electron/browser behavior has a native C++ owner and C++ acceptance test, then remove all JavaScript/TypeScript/Node/Electron code, tests, manifests, dependencies, and exact repository-local artifacts;
16. pass the permanent zero-JavaScript and non-C++ executable allowlist gate;
17. keep the configured storage headroom through native scan/plan/quarantine/purge workflows while proving no dirty, active, unique, referenced, pinned, rollback, benchmark, provenance, or user artifact is deleted;
18. remove dead clean worktrees and superseded reproducible caches/weights/datasets through exact deletion manifests and append verifiable receipts;
19. reproduce target benchmark and hidden-suite results on the declared hardware/package;
20. remain honest about unsupported operations, uncertainty, and certification.

The invariant is:

Probabilistic in exploration. Exact in specification. Deterministic in commitment. Verified in execution. Explicit in failure. Unlimited in logical feature-tree length.

Appendix A - Canonical long-graph example

This illustrative example shows the difference between a human plan and executable graph detail. Dimensions are examples, not reconstructed truth for a supplied benchmark task.

A.1 Human plan

Create a 3 mm tray, form four perimeter walls, cut six rounded cells, and add six mounting tabs at the specified heights.

This is useful to a reviewer but is not executable.

A.2 Graph hierarchy

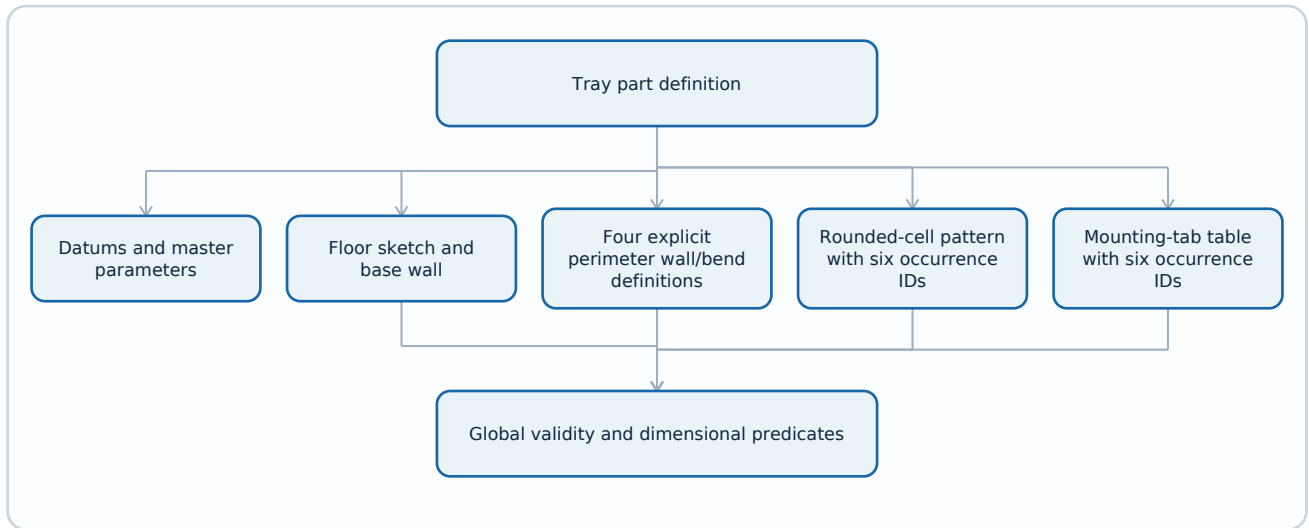


Figure 21. A.2 Graph hierarchy

A.3 Illustrative native C++ builder calls

```

FeatureGraphBuilder graph(project_id, normalized_spec_hash);

const auto xy = graph.datum_plane("datum.xy", Datum::XY());
const auto tray_params = graph.parameter_block(
    "tray.master_parameters",
    {mm("thickness", 3.0), mm("inside_bend_radius", 3.0)});

const auto floor_sketch = graph.sketch("sketch.floor", xy)
    .rectangle("floor.profile", mm(194.875), mm(222.492))
    .constrain_horizontal_vertical()
    .constrain_center_to_origin()
    .fully_constrain();

const auto floor = graph.extrude(
    "feature.floor",
    floor_sketch.region("floor.profile"),
    tray_params.quantity("thickness"),
    Operation::NewBody);

const std::array<WallDefinition, 4> walls{
    WallDefinition{"wall.north", floor.edge("floor.north"), mm(40.0), deg(90.0)},
    WallDefinition{"wall.east", floor.edge("floor.east"), mm(40.0), deg(90.0)},
    WallDefinition{"wall.south", floor.edge("floor.south"), mm(40.0), deg(90.0)},
    WallDefinition{"wall.west", floor.edge("floor.west"), mm(40.0), deg(90.0)},
};

const auto wall_group = graph.sheet_metal_walls(
    "feature.perimeter_walls", floor, walls,
    tray_params.quantity("inside_bend_radius"));

const auto cell_seed = graph.rounded_slot_cut(
    "feature.cell_cut.seed", floor.top_face("floor.top"),
    RoundedSlot{mm(32.0), mm(18.0), mm(6.0)},
    Point2{mm(0.0), mm(0.0)}, CutExtent::ThroughAll);

const std::array<OccurrenceTransform, 6> cell_locations{
    at_xy{mm(-60.0), mm(-35.0)}, at_xy{mm(0.0), mm(-35.0)},
    at_xy{mm(60.0), mm(-35.0)}, at_xy{mm(-60.0), mm(35.0)},
    at_xy{mm(0.0), mm(35.0)}, at_xy{mm(60.0), mm(35.0)},
};

const auto cells = graph.table_pattern(
    "pattern.cell_cuts", cell_seed,
    cell_locations);
  
```

```
const std::array<MountingTabDefinition, 6> tab_definitions{
    MountingTabDefinition{"tab.01", "wall.north", mm(-65.0), mm(18.0)},
    MountingTabDefinition{"tab.02", "wall.north", mm(0.0), mm(30.0)},
    MountingTabDefinition{"tab.03", "wall.north", mm(65.0), mm(18.0)},
    MountingTabDefinition{"tab.04", "wall.south", mm(-65.0), mm(18.0)},
    MountingTabDefinition{"tab.05", "wall.south", mm(0.0), mm(30.0)},
    MountingTabDefinition{"tab.06", "wall.south", mm(65.0), mm(18.0)},
};

const auto tabs = graph.table_feature<MountingTabDefinition>(
    "feature.mounting_tabs", tab_definitions);

graph.require_exact_counts({
    {NodeFamily::PatternOccurrence, 6},
    {NodeFamily::MountingTabOccurrence, 6},
    {NodeFamily::UnresolvedReference, 0},
    {NodeFamily::OpaquePlaceholder, 0},
});

graph.verify(requirements_for_tray);
graph.finalize_or_pause_incomplete();
```

The `table_pattern` and `table_feature` calls are C++ APIs over explicit child records. They do not hide an unbounded loop; the serialized graph contains six stable child IDs and transforms/definitions. The final implementation must serialize all relevant sketch entities, constraints, parameters, references, postconditions, and evidence described by Section 0.

Appendix B - Mandatory exact-length acceptance tests

Test	Construction	Pass condition
LONG-10X-RESUME	graph at least 10× the semantic-node count of the matched short baseline	forced stop at multiple chunk boundaries resumes to identical root/replay fingerprints
LONG-100X-RESUME	graph at least 100× the semantic-node count of the matched short baseline	no summary substitution, loss, duplication, count drift, or context dependence
PATTERN-EXPLICIT	nested patterns with suppression and overrides	exact intended/materialized occurrence counts and addressable child lineage
ASSEMBLY-HIERARCHY	repeated subassemblies and standard components	definitions, occurrences, mates, and BOM reconcile without flattening
EDIT-UPSTREAM	change an early driving dimension	downstream rebuild succeeds or precisely fails; unaffected invariants remain unchanged
TOPOLOGY-SPLIT-MERGE	upstream operation splits/merges referenced faces	stable query rebinds uniquely or fails closed; never arbitrary face selection
CHUNK-CORRUPTION	remove, duplicate, reorder, or alter one chunk	hash/count chain detects corruption before acceptance
RESOURCE-EXHAUSTION	exceed context, memory, or time budget	returns <code>PAUSED_INCOMPLETE</code> with last valid checkpoint, never success
OPAQUE-MACRO	insert "finish remaining holes" or equivalent	parser/schema/policy rejects executable placeholder
NOOP-PADDING	add meaningless features to inflate length	graph-quality gate rejects or removes padding without hiding required intent
DETERMINISTIC-COMMIT	repeat from identical normalized input/package	identical canonical graph hash and accepted geometry fingerprint
MODEL-COMPACTION	compact conversation/reasoning summaries mid-job	executable graph and count ledger remain byte-identical

Appendix C - Source register

Models and runtimes

- [Qwen3-VL-30B-A3B-Instruct model card](#)
- [Official Qwen3-VL-30B-A3B-Instruct GGUF](#)
- [Qwen3 technical overview](#)
- [llama.cpp](#)
- [llama.cpp grammar documentation](#)
- [MLC LLM](#)
- [ONNX Runtime Core ML Execution Provider](#)
- [Apple Metal-cpp](#)
- [Apple Mac Studio technical specifications](#)

Agent and retrieval architecture

- [SearXNG documentation](#)
- [SearXNG Search API](#)
- [SearXNG installation guidance](#)
- [LangGraph overview](#)
- [LangGraph persistence](#)
- [LangGraph interrupts](#)

Geometry, numerics, and simulation

- [Open CASCADE Technology](#)
- [OCCT documentation](#)
- [CGAL documentation](#)
- [libfive](#)
- [PicoGK](#)
- [Manifold](#)
- [Project Chrono](#)
- [Eigen](#)
- [MFEM and GPU support](#)
- [PETSc user guide](#)
- [Gmsh](#)
- [OpenFOAM Foundation](#)
- [G+Smo geometry and simulation modules](#)
- [OCCT modeling algorithms](#)
- [OCCT XDE](#)
- [OCCT STEP translation](#)
- [Qt 6 Widgets](#)
- [KDDockWidgets](#)
- [Diligent Engine](#)

- [Filament](#)
- [OpenUSD](#)
- [OpenUSD introduction, performance guidance, and glossary](#)
- [Pixar OpenSubdiv](#)
- [MaterialX](#)
- [OpenColorIO](#)
- [OpenImageIO](#)
- [OpenEXR](#)
- [OpenVDB](#)
- [Embree](#)
- [CMake Presets](#)
- [CMake FetchContent offline controls](#)
- [vcpkg local and asset caching](#)
- [ccache](#)
- [Git worktree documentation](#)

CAD and engineering benchmarks

- [BenchCAD repository and paper](#)
- [neuralCAD-Edit](#)
- [Drawing2CAD](#)
- [ParaCAD / PHT-CAD](#)
- [Text2CAD and dataset card](#)
- [HistCAD and paper](#)
- [MUSE paper and dataset](#)
- [CADGenBench and input data](#)

Data and public engineering references

- [DeepCAD](#)
- [Fusion 360 Gallery Dataset](#)
- [SketchGraphs](#)
- [ABC Dataset](#)
- [JoinABLE](#)
- [AutoMate](#)
- [MFCAD](#)
- [MFCAD++](#)
- [AAGNet / MFinSeg](#)
- [BRepNet](#)
- [UV-Net](#)
- [ShapeNet](#)
- [PartNet](#)
- [Objaverse / Objaverse-XL](#)

- [Amazon Berkeley Objects](#)
- [Scan2CAD](#)
- [OmniObject3D](#)
- [Matbench](#)
- [matminer datasets](#)
- [NIST CAD/PMI Testing](#)
- [PDEBench](#)
- [AirfRANS](#)
- [DrivAerNet++](#)
- [SimJEB](#)
- [The Well](#)
- [Materials Project API](#)
- [NIST Materials Data Repository](#)
- [NIST JARVIS](#)
- [NASA Technical Standards System](#)
- [NASA Turbulence Modeling Resource](#)
- [NIST CAD/PMI verification results](#)
- [NIST STEP/AP209 examples](#)

High-yield capability catalogs and product references

- [AMD Research and Advanced Development open-source projects](#)
- [NVIDIA open-source projects](#)
- [NVIDIA Kaolin](#)
- [NVIDIA PhysicsNeMo overview](#)
- [NVIDIA Isaac Sim](#)
- [FreeCAD, features, source repository, and source architecture](#)
- [CATIA, CATIA V5, mechanical engineering and Class-A surfacing, and current surface-refinement capabilities](#)

Public product references

- [P-1 AI](#)
- [P-1 AI COMPUTEX 2026 workflow](#)
- [Reuters on Prometheus and complex physical-system manufacturing](#)
- [Reported "artificial general engineer" scope](#)
- [Kyrall](#)
- [Adam](#)

Appendix D - Open decisions that require measured evidence

These are deliberate experiments, not ambiguities that justify stalling the architecture:

1. exact production quantization and KV-cache precision for Qwen3-VL-30B-A3B on the 36 GB Mac;

2. llama.cpp commit/model-package combination that passes multimodal structured-output regressions;
3. measured Qt 6 Widgets/KDDockWidgets/Diligent integration boundaries and final UI module set;
4. sketch solver architecture and source-use admission strategy;
5. which CGAL packages can enter the distributable product;
6. native versus external-process boundaries for complex CAE tools;
7. exact approved SDKs and adapter boundaries for DWG and proprietary native CAD;
8. benchmark metric-specific thresholds where an official score is not normalized to 0-1;
9. data admission decisions for every **CONDITIONAL** source;
10. external branding/name decision given P-1 AI's public use of Archie.

Each decision receives an owner, experiment, success metric, artifact, and deadline in the project tracker. None permits weakening the exact-feature-DAG contract.